How to Write a Research Proposal & Documentation

Author:

komondi@strathmore.edu

Designed for Fourth Year Scientific Research Projects in BSc. Informatics and Computer Science in School of Computing and Engineering Sciences

---

Summary

A research proposal is typically written in the third person and is organized into three main chapters: Introduction, Literature Review, and Methodology. This guide provides a detailed overview of each of these core sections, along with an analysis of the additional subsections not explicitly highlighted in this summary.

Note: Some sections of the proposal, such as the Cover Page (which includes the Research Title), Declaration and Approval, Table of Contents, and Lists of Tables, Figures, Equations, and Abbreviations, will be discussed in detail during the class seminars. **Please refer to the template provided on the classroom platform.** You are encouraged to use this template to begin your write-up - it's available for free download and reuse.

Cited References

How to write the Background of your study - Dr. Rishibha Sachdev

Lesson 1: Choosing a Research Topic

Session Outline:

1. **Choosing a Research Topic:** How to identify, refine, and narrow down a topic for your study.
2. **Writing a Research Problem:** Common mistakes to avoid when formulating your research problem.
3. **Creating and Selecting a Title:** Best practices for developing and choosing an effective research title.

Choosing a Research Topic

1. **Start with the End in Mind: -** Approach your research topic by thinking backward from the final outcome. Consider the kind of contribution you want your research to make.
2. **Select a Relevant Topic: -** Choose a topic that will be of genuine interest to the scientific community and the broader research field, especially considering the expectations at the defense stage.
3. **Important Caution: -** "*Just because it has not been done before is no justification for doing it now.*" — Peter Attiwill, Editor-in-Chief, Forest Ecology and Management.

Key Questions to Ask When Selecting a Research Topic:

1. **Relevance:** Is the topic focused on something new, current, interesting, or compelling?
2. **Impact:** Does it have the potential to provide solutions to significant challenges within an organization, sector, or society?

3. **Awareness of Existing Research:** Are you familiar with the latest findings in this field? (Make sure to review and reference top journals and leading research studies in the area.)
4. **Final Check in Topic Selection:** - Will the results of your research be interesting or useful to others in the scientific community or in a related field?

If the answer to all the guiding questions is **YES**, then you have identified a strong research topic worth pursuing. Begin exploring and gathering relevant information and literature to build a solid foundation for your study.

Common Mistakes in Research Proposal Writing:

These mistakes typically fall into three main areas:

1. **Lack of a Narrative Hook:** Failing to capture interest at the beginning by providing a strong narrative backed by evidence, such as statistics or real-world examples.
2. **Unclear Knowledge Gap:** Failing to clearly identify and demonstrate the gaps in the existing body of literature.
3. **Undefined Significance:** Failing to articulate why addressing these knowledge gaps is important and how it contributes to solving the research problem.

These mistakes often arise from three key areas:

1. **Unrealistic Expectations:** Failing to understand the level of commitment, effort, and time required to successfully complete a final year project. Set realistic expectations from the start.
2. **Poor Supervisor Relationship:** Not establishing a professional, communicative relationship with your supervisor. A strong working relationship can significantly ease problem-solving and project progress.

3. **Lack of Long-Term Focus:** Losing sight of your ultimate goal. Keep the end in mind and consistently work to stay ahead in your planning and execution.

Topic Selection

Let us now have a deep dive into topic selection and creation. What are some of the strategies for Topic Formulation (Wilkinson, 1991)?

1. **Be Brief and Concise:** Avoid unnecessary words that do not add value to your title. Eliminate phrases such as "*An Approach to...,*" "*A Study of...,*" or *"An Investigation of...*"
2. **Choose a Clear Title Structure:** Use either a **single title** or a **double title**.
    a. **Example of a single title:** Optimizing Neural Networks for Real-Time Speech Recognition
    b. **Example of a double title:** Fine-Tuning T5 Models: Enhancing Question Answering Performance in Low-Resource Languages.

In all these examples, you do notice how they're short, direct, and clearly tell you the topic without needing extra phrases like "A Study of..." or "An Investigation into...".

Other things to consider include:

1. **Keep the Title Concise:** The title should be between **12 and 18 words.** Eliminate most articles (e.g., the, a, an) and prepositions where possible. Focus on including the core subject or focus of the study—the fewer words, the better, without losing clarity.
2. **Ensure Research Feasibility:** Choose a topic that can realistically be researched (given available resources, time, and access to data) and should be researched (it addresses a meaningful gap or problem in the field).
3. **Identify Your Interests**: - Focus on areas that genuinely interest or puzzle you within your field of study.

4. **Explore Trending Topics:** - In Computer Science, consider emerging and high-impact areas such as Machine Learning, Edge Computing, Quantum Computing, Internet of Things (IoT), Blockchain, and others. Choose a topic that will sustain your interest throughout the project.
5. **Identify Relevant Keywords:** - Keywords help in refining your focus. For instance, in Machine Learning, common keywords could include K-Nearest Neighbors (KNN), Classification, Regression, Recognition, among others.
6. **Define the Topic Clearly: -** Analyze the selected keywords carefully and narrow down the scope to a specific area of focus. Ensure the topic is neither too broad nor too narrow.
7. **Formulate the Topic:** - Review existing articles, research papers, and journals relevant to your area of interest. This background research will help you sharpen and clarify your topic. Take notes, paraphrase important points, and summarize key ideas from these materials—this will greatly assist in writing your literature review.

More examples of single (standalone) topics and double topics you may explore further are as follows.

Single Titles (Simple, Standalone Titles)

1. Optimizing Edge Computing Architectures for Real-Time Data Processing
2. Blockchain-Based Identity Management Systems for Secure Online Transactions
3. Improving Image Recognition Accuracy Using Deep Convolutional Neural Networks

Double Titles (Main Title: Subtitle)

1. Enhancing Multilingual Chatbots: Improving Low-Resource Language Support Using Transfer Learning

2. Cybersecurity in Smart Cities: Analyzing Threats and Developing Proactive Defense Strategies
3. Energy-Efficient IoT Networks: Designing Sustainable Solutions for Remote Monitoring

In some scenarios, you may have topics where a case study is applied and when formulating case study topics, it is good practice to follow a few important guidelines.

1. First, clearly specify the technology or solution you are applying in your study. This helps define the technical focus of your work.
2. Second, explicitly mention the case you are examining—whether it is a specific location, organization, group, or community—so that the context of your research is well understood.
3. Finally, highlight the particular impact or challenge your study aims to address.

Doing so ensures that your research is positioned as relevant and meaningful to both academic and practical audiences. See the examples below:

1. Optimizing Federated Learning Models for Healthcare Data Privacy: A Case Study of Rural Clinics in Kenya
2. Implementing Smart Contract-Based Land Ownership Verification: A Case Study of Nairobi County's Blockchain Registry System
3. Deploying Reinforcement Learning Algorithms for Personalized Learning Paths: A Case Study of Strathmore University's E-Learning Platform

To-Do: End of Lesson 1

1. Schedule a meeting with your supervisor to discuss the research area you are interested in pursuing.
2. During the meeting, agree on a research topic that aligns with the topic formulation guidelines discussed in class.

3. Once finalized, update your selected topic on the shared Google Sheet by **Thursday, 14th May 2025**.

Lesson 2: Proposal Writing

Acknowledgements

The acknowledgements section of a thesis or research proposal is where you formally express gratitude to the individuals, institutions, and organizations that significantly supported your work. Unlike a dedication, acknowledgements are formal and focused specifically on contributions related to the research.

It is not necessary to mention everyone who assisted you—only those who played an important role in your research process. For example, it would not be appropriate to thank a partner simply for providing personal support, such as preparing meals, unless their contribution is directly related to the project.

Abstract

An abstract is a concise summary of a longer piece of writing. It highlights the key points, clearly describes the content and scope, and presents an abbreviated overview of the work. Typically, abstracts are between **100 and 250 words**, with most being around **200 words**. They often follow specific patterns, which may be determined by the guidelines of the examining academic institution.

The abstract is critical because it often forms the reader's first impression of the work—and in some cases, the only impression. For example, in funding applications, some reviewers, such as members of a foundation's board of directors, may read only the abstract before making funding decisions. As a result, the abstract must not only

summarize the proposal effectively but also make a strong and persuasive impact. It is, therefore, one of the most important elements of a research proposal or thesis.

Significance of the Abstract

1. Enables readers to determine whether the full article is relevant to their interests or research.
2. Assists readers and researchers in recalling essential findings on a specific topic.
3. Provides an overview of main points, enhancing comprehension before engaging with the complete text.
4. Facilitates indexing for efficient retrieval and cross-referencing of articles.

Core Components of an Abstract

1. **Background:** A brief opening that places the research within its broader context.
2. **Objectives:** A concise statement outlining the purpose or goals of the study.
3. **Methods:** A short description of the procedures or approach taken in the project.
4. **Findings:** A summary of key results or anticipated outcomes, often refined at the final stages—such as reporting model accuracy or observed improvements.
5. **Conclusion:** A final statement highlighting the significance or impact of the results—what they reveal, solve, or contribute.

Essential Questions an Abstract Should Address

1. **Purpose**: What motivated this study, research, or project? (Begin with the end goal in mind—such as contributing to graduation or solving a specific problem.)
2. **Approach**: What was done, and by what means? (Or, what will be done and how?)

3. **Results**: What were the key findings? (Or, what outcomes are anticipated?)
4. **Implications**: What is the significance of these findings? What do they reveal or contribute?

Tips for Writing an Effective Abstract

1. Revisit your article or proposal with the goal of identifying its core components.
    - Focus on extracting key elements: purpose, methods, scope, results, conclusions, and recommendations.
    - Use the headings or table of contents to guide the abstract structure.
2. Draft your abstract from memory after reviewing the full document—avoid copying directly.
3. Rephrase ideas in your own words to ensure clarity and originality.
4. Revise your draft to:
    - Strengthen organization and flow
    - Improve transitions
    - Remove unnecessary details
    - Ensure completeness and accuracy
    - Eliminate wordiness
    - Correct grammar, spelling, and punctuation
    - Maintain consistency in tone and voice with the main document

Abstract Example:

Review the abstract below to understand how to craft one effectively. Pay close attention to the title—it is a single, well-structured line that captures the core problem, the proposed solution, and the intended implementation methods.

Improving Question Answering in Maternal Healthcare through Generative AI and Natural Language Processing

In recent scientific research, Question Answering (QA) systems have gained significant attention across various domains, including healthcare. Within maternal healthcare, QA systems offer the potential to deliver timely and accurate responses to queries concerning pregnancy, childbirth, and postpartum care. However, current QA models in this area face limitations, such as scarce training data and difficulty processing complex questions with multiple components.

This study proposes leveraging generative AI models to enhance QA performance in maternal healthcare. The approach integrates pre-trained language models with domain-specific datasets to generate contextually appropriate and informative answers. A fine-tuning pipeline is also introduced, designed to adapt these models using a small, annotated corpus of maternal healthcare questions.

Experimental results demonstrate that the proposed method outperforms existing QA models across key evaluation metrics, including accuracy, precision, and recall. The model's ability to address complex, multi-part questions positions it as a practical tool for both healthcare providers and patients. This research highlights the potential of generative AI in advancing QA capabilities, ultimately contributing to improved access to information, better patient outcomes, and more efficient healthcare delivery.

Keywords: Question Answering, Maternal Healthcare, Generative AI, Pre-trained Language Models, Fine-tuning, Complex Queries, Healthcare Innovation

Note: The final paragraph in the abstract reflects a completed project, where all experiments have been conducted and results analyzed. At the proposal stage, however, your abstract will naturally differ, as the experiments are yet to be carried out. Keep in mind that your research direction may evolve over time. Additionally, ensure that your ***keywords***—which appear at the end of the abstract—are drawn directly from the content of your abstract and are formatted in ***bold and italics*** as shown.

To-Do: End of Lesson 2:

- **Meet with your assigned supervisor** to discuss your initial project ideas. Use insights from class sessions to formulate a well-thought-out research title.
- **Develop a draft abstract** based on your proposed title and review it with your supervisor. Refer to the example provided above for guidance.
- **Document your work** using the Word template shared in class. Ensure all sections are properly filled in as instructed.

End of Session

Please take this time for **questions**, **summaries**, and **clarifications** of any **concepts** or **assigned tasks**. Ensure you leave with a clear understanding of what is expected before the next class.

Lesson 3: Chapter 1: Introduction

The introduction of a research paper or proposal provides background information and sets the context for the study. It shows how the research relates to existing literature and identifies gaps to be explored. In proposals, especially those seeking funding, the introduction must clearly communicate the research goals and significance. A strong introduction includes the research **hypothesis**[1], which guides the research design and can be formulated by asking critical questions about the topic.

1. What is the central research problem?
2. What is the topic of study related to that problem?
3. What methods should be used to analyze the research problem?

[1] A **hypothesis** is a tentative explanation based on limited evidence, used as a starting point for further investigation.

4. Why is this research important, what is its significance, and how will its outcomes affect the funders and the research society as a whole?

Chapter 1 – Research Proposal Sections

1.1 Background Information

The background section provides the context for the research, explaining its importance and setting the stage for the study. It justifies the need for the research and summarizes its objectives. Typically, this is the opening section of a proposal, article, or thesis.

How Should the Background Information Be Structured?

1. **Historical Context & Literature Overview**
    - Briefly outline key developments in the field that led to the current research topic.
    - If the study is interdisciplinary, clarify how different disciplines intersect and which aspects will be examined. (For technology-focused research, specify relevant subfields, e.g., AI, cybersecurity, or renewable energy technologies.)
2. **Identification of Gaps**
    - Summarize existing knowledge on the broad topic.
    - Clearly state unresolved issues or missing links in the literature that the study aims to address.
3. **Significance of Addressing Gaps**

- Explain why filling these gaps matters (e.g., theoretical, practical, or societal implications).

4. **Rationale & Hypothesis**
    - Provide the logical foundation for the study.
    - State the hypothesis or research questions guiding the investigation.

How to Make the Background Engaging

To maintain reader interest (including funders), structure the background as a **compelling narrative** around the research's central theme. Avoid excessive detail or digressions into broad literature reviews. Instead:

- **Build a logical flow**: Each point should connect seamlessly, guiding readers to recognize the gaps your study addresses.

Class Discussion:

Assuming your research is focusing on Sentiment Analysis on Monolingual, Multilingual and Code - Switching Twitter Corpora, how would you structure your background?

- **Context**: Start with the rise of social media (e.g., Twitter/X) as a key platform for public sentiment.
- **Focus**: Narrow down to challenges in analyzing monolingual, multilingual, and code-switched texts.
- **Gaps**: Highlight limitations in current tools (e.g., bias in monolingual models, scarcity of code-switching datasets).

- **Climax**: Emphasize why addressing these gaps matters (e.g., improving cross-cultural communication, AI fairness, credibility in fact checking).

Common Mistakes to Avoid

1. **Length Issues**
    - **Too long**: Overwhelms with unnecessary details.
    - **Too short**: Omits critical context.
    - **Solution**: Be **concise but comprehensive**—cover key developments and gaps without tangents.
2. **Ambiguity**
    - Avoid vague language; assume readers lack deep familiarity with your topic.
3. **Irrelevant Content**
    - Stay focused on the **core problem** (e.g., for sentiment analysis, avoid digressing into general NLP history).
4. **Disorganization**
    - Present ideas **chronologically** or thematically to show field progression clearly.
    - **Example:** Trace sentiment analysis evolution → current multilingual/code-switching challenges → your study's role.

Key Takeaway: A well-structured background tells a **clear, gap-driven story** that aligns with the research's novelty and significance.

Distinguishing Background Information from Literature Review

Many researchers confuse these two sections, but they serve distinct purposes:

1. **Background Information**

- **Purpose:** Establishes the *context* and *significance* of the research.
- **Content:**
  - Broad introduction to the research area.
  - Key developments leading to the current problem.
  - Clear identification of gaps and rationale for the study.
- **Structure:**
  - Thematic + chronological organization to show field evolution.
  - Focused discussion (avoid excessive detail).
- **Referencing: Heavily cited but *concise*—only essential sources to frame the problem.**

2. **Literature Review**
   - **Purpose:** Provides *evidence* supporting the study's hypothesis/objectives.
   - **Placement:** Follows the background (typically after research questions).
   - **Content:**
     - *Comprehensive* analysis of past studies (both broad and specific).
     - Critical evaluation of methodologies, findings, and trends.
     - Explicit links to how prior work justifies the current study.
   - **Depth:** More detailed than the background—expands on all referenced studies.

Practical Tips

- **Background**: Think of it as the *"trailer"* for your research—hook the reader with clarity and urgency.
- **Literature Review**: This is the *"full film"*—prove your study's necessity through rigorous evidence.

Example for Clarity:

- **Background**: Sentiment analysis has advanced in monolingual contexts Smith, (2020), but multilingual and code-switched data remain understudied Lee, (2022), limiting global applicability.
- **Lit. Review**: Smith (2020) used LSTM models for English tweets (accuracy: 85%), while Lee (2022) identified a 20% drop in accuracy for code-switched texts, attributing it to...

Note: Always cite rigorously in both sections, but tailor references to their purpose - background = foundational; lit. review = exhaustive.

1.2 Problem Statement

A **problem statement** is a concise claim that defines the specific issue your research aims to solve. It answers the question: ***What is the problem, and why does it matter?***

Purpose of a Problem Statement

1. **Narrow the Focus**: Transforms a broad concern into a clear, researchable question.
2. **Justify the Study**: Explains why the problem warrants investigation.
3. **Guide the Proposal**: Serves as the foundation for your research objectives and methodology.

A well-crafted problem statement is **brief but impactful** - typically **Half a page or 250 words** for early-stage research, though advanced work may require more detail.

Structure of a Persuasive Problem Statement

1. **The Ideal** – Describe the optimal scenario **(*how things should be*).**

   ***Example*:** *Social media platforms should accurately detect sentiment across languages to foster inclusive communication.*

2. **The Reality** – Explain the current shortfall **(*what's wrong now*)**.

   ***Example*:** *However, most sentiment analysis tools fail with multilingual and code-switched text, misclassifying 30% of non-English tweets (Lee, 2023).*

3. **The Consequences** – Propose your research's role in bridging the gap.

   ***Example*:** *This study will develop a hybrid model to improve accuracy in code-switched data, enabling equitable AI applications.*

Note:

- **Cite evidence** (e.g., "Prior studies show... [Smith 2021]") to ground your problem in existing research.
- Avoid vagueness—quantify the problem where possible (e.g., "70% of rural clinics lack access to...").

1.3 Research Objectives

Research objectives define the **specific goals** your study aims to achieve. They provide clear direction and help measure the success of your work. Objectives should be **action-oriented, measurable, and aligned** with your problem statement.

1.3.1 General Objective

The **general objective** is the **broad, overarching goal** of your research. It summarizes the primary purpose of your study in **one concise statement.**

Examples:

1. *"To develop an end-to-end speech recognition model for Fon and Igbo languages."*
2. *"To design an improved deep learning model for question-answering tasks in maternal healthcare."*
3. *"To develop a secure e-voting application using blockchain technology."*

1.3.2 Specific Objectives

These are **detailed, actionable steps** that break down the general objective into smaller, achievable tasks. They should follow the **SMART criteria** (Specific, Measurable, Achievable, Relevant, Time-bound).

Example - in case you have a topic below:

Improving QA in Maternal Healthcare with Generative AI

General Objective:

To develop an improved question-answering (QA) model for maternal healthcare using generative AI.

Specific Objectives: - This section should be in roman numbers (small).

1. To analyze the current state of Question Answering in Maternal Healthcare Domain (you should also add that the challenges in this section will be reviewed)
    a. **Explanation:** identifying limitations in accuracy and domain coverage. (Literature review + dataset audit)
    b. This will make your first subsection in the literature review.
2. To study and analyze how currently existing works have improved Question Answering in Maternal Healthcare domain
    a. **Explanation:** This will benchmark existing generative AI models (e.g., BERT, GPT) on maternal health QA tasks, measuring F1-score and response relevance. (Comparative analysis)
    b. This will be your related works in the literature review
3. To review the challenges related to currently existing Question Answering Models in Maternal Healthcare.
    a. **Explanation:** This will be the gaps in existing literature.
4. To design and develop an improved Question Answering Model fine-tuned to Maternal Healthcare Questions.
    a. **Explanation:** This will be your Conceptual Framework
5. To test and validate the proposed Question Answering model
    a. **Explanation:** This will be your system from chapter 3 to the end.

1.4 Research Questions

A research question is the specific focus of your study, derived from your research problem and shaped by your study design. By refining your research problem into a clear, actionable idea, you arrive at your research question. Typically, these questions align with and emerge from your research objectives.

Examples: Research questions can be derived from objectives as follows:

Objective: To analyze the current state of QA in the maternal healthcare domain.

Research Question: What is the current state of QA in maternal healthcare?

Objective: To study how existing works have improved QA in maternal healthcare.

Research Question: How have previous QA models advanced question answering in maternal healthcare?

1.5 Justification

Research needs a strong "**why**" – and that justification comes from understanding what's already known and what's still missing. Here's how this study fits in - (*these are just dummy examples*):

1. **Current Knowledge: - this is what we currently know**

   For Example:

   - Studies like Smith et al. (2020) developed effective QA systems for general healthcare...
   - Johnson (2021) created a model that answers cardiology questions with 85% accuracy...

2. **Identified Gaps: - this is what is missing**

   For Example:

   - No systems specialize in maternal health scenarios:
     - Can't handle questions about high-risk pregnancies (Lee, 2022)
     - Struggles with regional language variations (Garcia, 2023)

- Existing models are 20% less accurate for postpartum care questions (Chen, 2021)

3. **The Problem This Causes: - this is why this matters**

   For Example:

   - Expectant women in developing regions often get incomplete/inaccurate information...
   - Healthcare workers waste time searching through unreliable sources...

4. **Our Solution: - this is why this study helps**

   For Example:

   - Building the first QA system specifically for maternal health
   - Incorporating:
     - Regional language support
     - Verified medical guidelines
     - Case-based reasoning
   - Expected to improve answer accuracy by 30% for maternal health queries

Summary: This work will help [target users] by [expected benefit], while advancing research in [academic field].

1.6 Scope

The scope details how in-depth your study is to explore the research question and the parameters in which it will operate in relation to the study and timeframe.

Example:

This study will focus on [specific area] while operating within defined boundaries to ensure feasible completion. The research will cover:

1. **Technical Scope:**
    - Development of a [specific system/tool] using [technology/method]
    - **Example:** *"A maternal health QA system using BERT-based models, focusing on English-language queries about pregnancy complications"*
2. **Data Scope:**
    - Analysis of [data types/sources] from [time period/location]
    - **Example:** "Publicly available maternal health guidelines from WHO (2015-2023) and anonymized patient forums"
3. **Functional Scope:**
    - Implementation of [key features] but excluding [limitations]
    - **Example:** *"Will answer diagnostic questions but not provide treatment recommendations"*
4. **Time Scope:**
    - Conducted over [duration] with milestones:
        - Phase 1: Literature review (Month 1-2)
        - Phase 2: Model development (Month 3-5)

Exclusions:

- This study will not address [related but out-of-scope areas]
- Example: *"Non-English languages or pediatric care questions"*

1.7 Limitations and Delimitations

Delimitations define the boundaries you intentionally set for your study. These are choices you make to narrow your research focus, ensuring it remains manageable and relevant. **For example:**

- Studying only female patients aged 20–35 in urban hospitals
- Analyzing data from 2015–2026 to exclude outdated trends

- Using English-language sources only

Delimitations clarify what your study *will not address*—not because of constraints, but because those aspects fall outside your research goals. They help readers understand the specific lens through which you're examining the problem.

Limitations, on the other hand, are potential weaknesses in your study that arise from factors beyond your control. These might affect the validity or generalizability of your findings. Common examples include:

- A small sample size due to limited participant availability
- Potential bias in self-reported survey responses
- Missing data in historical records that couldn't be recovered

Unlike delimitations, limitations aren't choices—they're challenges you acknowledge upfront. Addressing them openly strengthens your credibility by showing awareness of your study's constraints.

Key Differences Summarized

- **Delimitations** = Intentional exclusions (*"We focus on X and exclude Y because..."*).
- **Limitations** = Unavoidable shortcomings (*"We recognize that Z may affect results because..."*).

Why Both Matter

1. **Delimitations** keep your study targeted and feasible.
2. **Limitations** contextualize your findings and guide future research.

By transparently outlining both, you help readers interpret your work accurately and trust your conclusions.

Real-World Analogy

Think of delimitations as drawing a map ("We'll explore *this* neighborhood only") and limitations as weather conditions ("Rain limited visibility during our survey"). One is planned; the other is unpredictable.

To-Do: End of Lesson 3: - complete by next class

- Use the provided template (Download from Google Classroom) to draft all sections of the proposal till end of Chapter 1:
- Keep it focused:
    - Align all sections with your research problem
    - Be specific in scope/delimitations (avoid vague wording)
- Review before submitting: - Work with your supervisor
    - Check formatting matches the template

Lesson 4: Chapter 2: Literature Review

Definition and Purpose

A literature review surveys and critically evaluates scholarly sources (books, articles, papers and journals etc.) relevant to a research problem. It summarizes, synthesizes, and contextualizes existing work, demonstrating how your research fits within the broader field.

Key Objectives:

- Situate each source's contribution to understanding the research problem.
- Analyze relationships, conflicts, and gaps among studies.
- Offer new interpretations of prior research.
- Prevent duplication by identifying existing scholarships.
- Justify the need for further research.

Structure of a Literature Review

1. **Introduction**
    - Overview of the topic, issue, or theory.
    - Objectives of the review.
2. **Thematic Organization**
    - Group sources by themes (e.g., supporting/opposing/alternative viewpoints).
    - Compare and contrast works, noting similarities and differences.
3. **Conclusion**
    - Highlight the most compelling arguments and contributions.
    - Identify unresolved gaps or contradictions.

Critical Evaluation of Sources

Assess each work by examining:

- **Provenance:** Author's credentials and evidence quality.
- **Methodology:** Appropriateness of data collection, analysis, and reporting.
- **Objectivity:** Balance of perspective and inclusion of contrary evidence.
- **Persuasiveness:** Strength of the author's arguments.
- **Value:** Contribution to the field.

Writing Tips

- **Use Evidence:** Support analysis with citations.
- **Be Selective:** Focus on key points tied to the research problem.
- **Summarize & Synthesize:** Integrate sources thematically.
- **Maintain Your Voice:** Prioritize your analytical perspective.
- **Paraphrase Carefully:** Accurately represent authors' ideas and cite properly.

Common Mistakes to Avoid

- Including irrelevant sources.
- Failing to critically assess research design or biases.
- Overlooking contrary findings or alternative interpretations.
- Relying too heavily on one type of source (e.g., only supportive studies).

Note: Write in the third person.

Class Discussions: Structuring Chapter 2

Tailoring the Literature Review to Your Objectives

Chapter 2's structure should align with your research objectives. Below is an example using the topic:

"Improving Question Answering Tasks in Maternal Healthcare Using Generative AI Models in Natural Language Processing."

Key Principle:

- Group related objectives to avoid redundancy. For instance, Objectives 1 and 2 might merge into a single subsection (e.g., *2.2: Generative AI in Healthcare*).

2.1 Introduction

This section outlines the organization of **Chapter 2 (Literature Review)**, detailing its scope, thematic structure, and the rationale behind the selected sources. It provides a roadmap for readers by:

1. **Contextualizing the Review**
    - Briefly restating the research problem (*e.g., improving QA tasks in maternal healthcare using generative AI*).
    - Explaining the chapter's role in grounding the study within existing scholarship.
2. **Describing the Layout**
    - Thematic grouping of literature (*e.g., AI in healthcare, NLP techniques, ethical considerations*).
    - Justification for merged/combined objectives (*if applicable*).

3. **Highlighting Key Sources**
    - Types of works reviewed (*peer-reviewed articles, datasets, case studies*).
    - Criteria for inclusion (*relevance, publication date, methodological rigor*).
    - Focus of papers/journals that are 3 years old (best case) or 5 years old (worst case)
4. **Transitioning to Subsections**
    - Preview of how each subsection (*2.2, 2.3, etc.*) addresses specific research objectives.

Example Phrasing:

Section 2.2 examines generative AI's applications in maternal healthcare, while Section 2.3 evaluates NLP techniques for QA tasks, culminating in identified gaps (Section 2.5).

2.2 Current State of Question Answering in Maternal Healthcare

Alignment with Objective I: *To analyze the current state of QA in maternal healthcare, identifying limitations in accuracy and domain coverage.*

2.2.1 Overview of QA in Maternal Healthcare

- **Definition**: QA systems in maternal healthcare leverage NLP to provide accurate, timely responses to medical queries (e.g., symptom checks, treatment guidelines) (Smith et al., 2022).
- **Current Approaches**:

    - Rule-based systems (e.g., WHO chatbots) with limited adaptability (Doe, 2021).
    - Generative AI (e.g., GPT-3 fine-tuned for obstetrics) improving dynamic responses (Lee et al., 2023).
- **Efficacy**:
    - **Successes**: Improved access in low-resource settings (e.g., *MamaToto* app in Uganda reduced maternal mortality by 20% (UN, 2022)).
    - **Limitations**: Poor handling of regional dialects and rare complications (Zhang et al., 2023).

2.2.2 Key Challenges ....

(Prose format with citations)

Existing QA systems face:

1. **Accuracy Gaps**: Misdiagnosis risks due to limited domain-specific training data (e.g., misclassifying preeclampsia symptoms (HealthTech, 2023)).
2. **Domain Coverage**: Over-reliance on Western datasets neglects local healthcare contexts (African Journal of Medicine, 2024).
3. **Real-World Integration**: Lack of clinician-AI collaboration frameworks (WHO, 2023).

2.3 Related Works in Maternal Healthcare QA

Alignment with Objective II: *Benchmark existing generative AI models on maternal health QA tasks.*

2.3.1 Jibu Sasa: Web-Based QA Application

- **Description**: A Kenyan platform providing maternal health advice via NLP (Android/iOS).
- **Technology**: BERT-base fine-tuned on Swahili/English medical texts (Acc.: 78% F1-score (KEMRI, 2023)).
- **Gaps**:
    - Limited multilingual support beyond East Africa.
    - No peer-reviewed validation for high-risk pregnancy queries (Mwangi et al., 2023).

(Optional: 1 screenshot of the interface in Appendix A.)

2.3.2 MediBot: Multilingual AI Assistant for Maternal Health

(Dummy Example 2 – Mobile App)

- **Description**: A cross-platform (Android/iOS) chatbot developed in Nigeria, integrating GPT-4 and WHO guidelines for prenatal care.
- **Technology**:
    - **Model**: GPT-4 fine-tuned on Nigerian Pidgin, Yoruba, and English.
    - **Performance**: 82% F1-score on local symptom queries (Nigerian Medical Journal, 2023).
- **Gaps**:

    - Struggles with rare conditions (e.g., placenta accreta).
    - No integration with hospital EHR systems.

2.3.3 MamaCare: Voice-Based QA for Rural Clinics

(Dummy Example 3 – Low-Resource Focus)

- **Description**: A voice-enabled AI tool used in Ethiopian clinics, designed for illiterate users (Amharic/Oromo).
- **Technology**:
    - **Model**: Hybrid of BERT and speech-to-text.
    - **Performance**: 75% accuracy on voice queries (Addis Ababa University, 2023).
- **Gaps**:
    - High latency (>5s response time).
    - Fails to distinguish between urgent/non-urgent queries.

Screenshot: Interface flowchart (Appendix B).

2.3.4 SafeBirthAI: High-Risk Pregnancy Advisor

(Dummy Example 4 – Specialized Use Case)

- **Description**: A web-based tool for clinicians, predicting complications (e.g., eclampsia) using RAG + Llama 2.
- **Technology**:
    - **Training Data**: 10,000+ case studies from Johns Hopkins.
    - **Benchmark**: 88% precision in risk alerts (JAMA, 2024).

- **Gaps**:
    - Requires high-bandwidth internet.
    - Limited to English-language inputs.

Table 2.2: Comparative Analysis of Reviewed Systems

System	Strengths	Critical Gaps
Jibu Sasa	Local language support	Narrow scope
MediBot	Multilingual	No EHR linkage
MamaCare	Voice accessibility	Slow response time
SafeBirthAI	High-risk focus	Infrastructure-dependent

2.4 Gaps in Related Works

Should be written in prose and cited with the relevant sections in the related works. In case you have to refer to some tables, you should cross - reference them

A critical synthesis of the reviewed systems (*Jibu Sasa, MediBot, MamaCare, SafeBirthAI*) reveals persistent limitations in maternal healthcare QA technologies, which this study aims to address:

1. **Limited Clinical Validation**

   Most tools (e.g., *Jibu Sasa*, *MediBot*) rely on narrow benchmark datasets rather than real-world clinical trials, risking inaccurate responses for high-risk scenarios like postpartum hemorrhage (Nigerian Medical Journal, 2023; KEMRI, 2023). *SafeBirthAI*'s focus on predictive alerts lacks peer-reviewed validation across diverse populations (JAMA, 2024).

2. **Language and Regional Bias**

   While *MediBot* and *MamaCare* support local languages, their training data overlooks regional dialects (e.g., Hausa variations in Northern Nigeria) and cultural contexts (e.g., traditional birth practices), reducing efficacy for rural populations (Addis Ababa University, 2023).

3. **Integration Challenges**

   None of the systems interface with Electronic Health Records (EHRs) or clinician workflows, limiting their utility in hospital settings (*Table 2.2*). *MamaCare*'s voice-based system further suffers from latency (>5s), rendering it impractical for emergency use.

4. **Infrastructure Dependencies**

   High-performance tools like *SafeBirthAI* require stable internet, excluding low-resource clinics (WHO, 2023). Conversely, offline-capable systems (*Jibu Sasa*) lack dynamic updating mechanisms, perpetuating outdated medical advice.

5. **Ethical Oversights**

   Only *SafeBirthAI* included bias audits for sensitive queries (e.g., abortion complications). Others (*MediBot*, *MamaCare*) fail to address disparities in maternal care access for marginalized groups (AI Ethics, 2023).

Key Unmet Need: A unified QA model combining *language adaptability* (like *MediBot*), *low-resource optimization* (like *MamaCare*), and *clinical rigor* (like *SafeBirthAI*) remains absent—a gap this research directly targets.

2.5 Conceptual Framework

Definition and Purpose

The conceptual framework for this study visually and theoretically outlines the development of an **improved QA model for maternal healthcare using generative AI**. It integrates key variables, relationships, and processes derived from the literature review (Sections 2.2–2.4) and aligns with the research objectives.

Steps to Develop the Framework

1. **Research Focus**
   - **Topic**: *Improving QA in Maternal Healthcare with Generative AI*.
   - **Research Question**: *How can generative AI models be optimized to address accuracy, language diversity, and clinical integration gaps in maternal healthcare QA systems?*
2. **Key Variables and Concepts**
   - **Input Variables**:

        - *User Queries*: Maternal health questions (e.g., symptom checks, prenatal advice).
        - *Data Sources*: Medical literature (WHO guidelines), EHRs, local language datasets.
    - **Processing Variables**:
        - *AI Model*: Fine-tuned generative LLM (e.g., Llama 2 with RAG).
        - *Validation*: Clinician feedback loops, bias audits.
    - **Output Variables**:
        - *Responses*: Accurate, culturally adapted QA outputs.
        - *Evaluation Metrics*: F1-score, latency, user satisfaction.
3. **Theoretical Foundation**
    - Builds on:
        - *Human-AI Collaboration Theory* (Klein et al., 2022): Ensures clinician oversight.
        - *Low-Resource NLP Models* (Ruder et al., 2023): Addresses language/dataset gaps.
4. **Visual Representation**
    - There are various ways to visually represent a conceptual framework, depending on your preference and the complexity of your research. Some common formats include flowcharts, **diagrams**, or schematic representations. You can use software like Microsoft Word, PowerPoint, or **specialized diagramming tools** to create your visual representation.
5. **Relationships Between Variables**
    - Fine-tuning the AI model on local datasets (*Input*) improves response accuracy (*Output*).

- Clinician feedback (*Processing*) reduces bias in high-risk scenarios (*Output*).

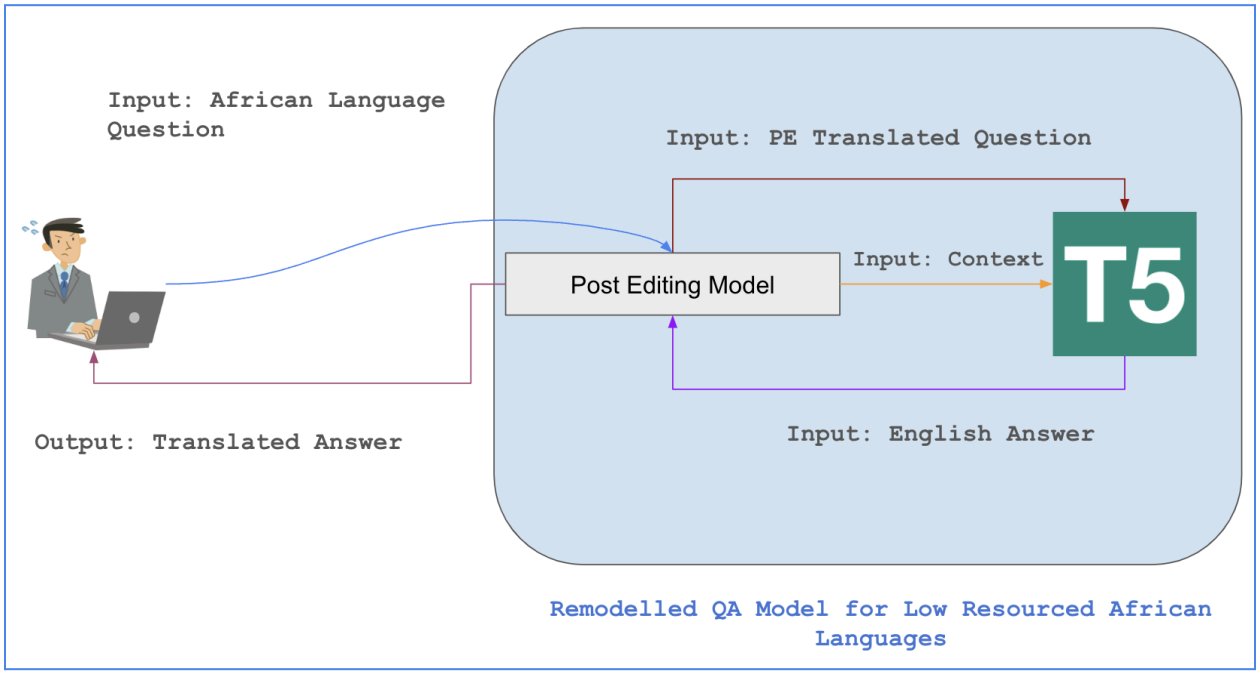
6. **Flexibility and Refinement**
    - The framework will evolve during model testing (Chapter 5) based on real-world data.

Example: Applied to Maternal Healthcare QA

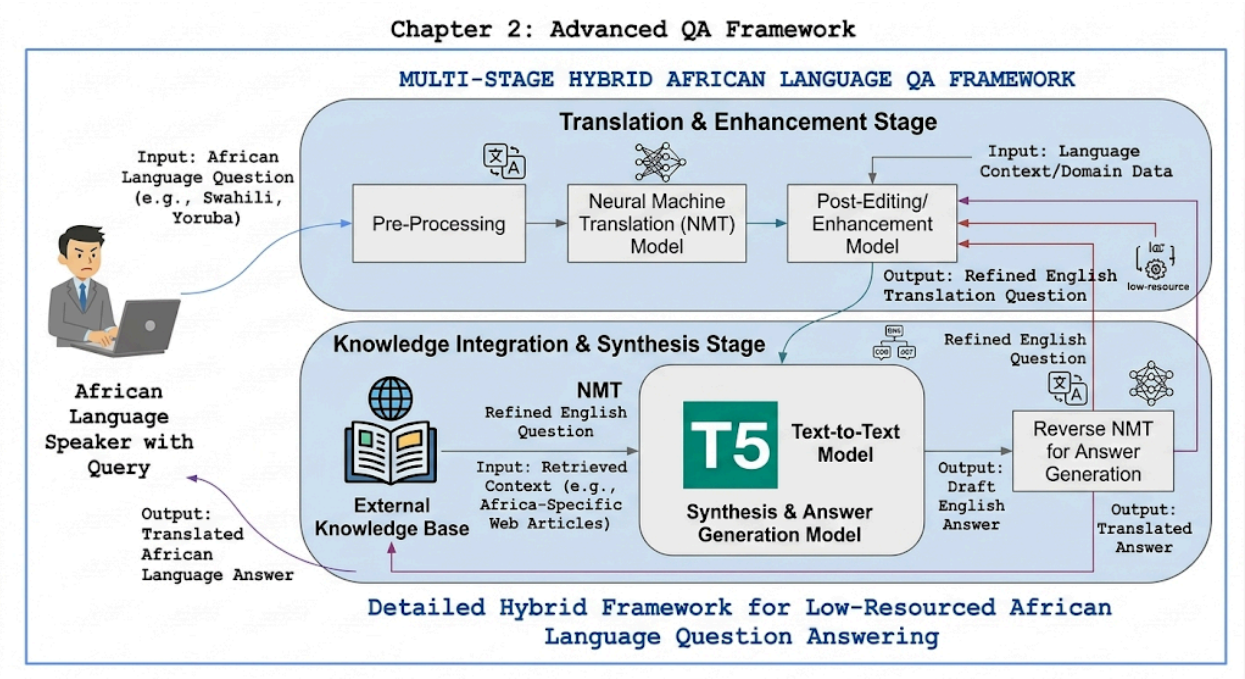
Narrative Flow:

1. A user (e.g., midwife in Kenya) submits a voice query in Swahili: *"Je, mimba ya miezi 6 inahitaji vitamini gani?"* (*"What vitamins are needed at 6 months pregnancy?"*).
2. The system:
    - Detects language/dialect (*Input*).
    - Retrieves WHO guidelines and local clinic protocols via RAG (*Processing*).
    - Generates a response validated by a clinician (*Output*).
3. Feedback from the midwife improves the model's future accuracy (*Iteration*).

Figure 2.1: Conceptual Framework Diagram



Generated using Gemini 3.5 Flash - PRO



Key Considerations

- **Alignment with Objectives**:
    - *Objective IV* (Design): Directly addressed by the AI model and validation modules.
    - *Objective V* (Test): Framework includes feedback loops for iterative improvement.
- **Supervisor Collaboration**: The final diagram will be refined with advisor input to ensure theoretical/practical coherence.

To-Do: End of Lesson 4: - complete by next class

- With reference to your objectives and supervisor's input, complete chapter 2. We still need to see your references growing in the reference section.

Lesson 5: Chapter 3: Methodology

3.1 Introduction

This chapter presents an overview of the methodological approach adopted in the study. It outlines the chapter's structure and briefly introduces key components discussed in subsequent sections. Keep this introduction **concise**, as it serves to frame the chapter rather than provide detailed content.

Note:

You must clearly state the Development Paradigm guiding your project — either

Structured Systems Analysis and Design (SSAD/SSADM) or **Object-Oriented Analysis and Design (OOAD)** — as covered in the section at the end of this session **(end of chapter 3).**

Your choice will shape the overall methodology, since some methods align better with SSAD/SSADM while others suit OOAD. You may consult references such as ChatGPT/Gemini or other academic materials to compare these paradigms.

- You can also use the **experimental approach** in your study. You will need to follow the rest of the document as if to fit your approach. Discuss with your supervisor.

For AI Projects, this section is adapted as follows

3.1 Introduction (For AI and IoT Projects)

- The example given below is for AI, but can also be adapted to IoT
    - After this section, there is an example for non AI/IoT projects - probably shorter than AI/IoT sections.
    - Normal systems such as projects in Blockchain/Pure Software/Algorithms will not use the format below. - check the section after.

This section outlines the chapter while introducing the **experimental design** used in the AI-based study. Unlike traditional software engineering, AI projects emphasize **data-driven experimentation**, **model training**, and **validation**.

The chapter covers stages including **data acquisition**, **processing**, **model training**, and **validation** — ensuring a systematic approach toward developing an AI solution for **maternal healthcare Question Answering (QA)**.

3.2 Research Paradigm

This project adopts an **experimental research paradigm**, where hypotheses are validated through **controlled data analysis** and **model training**. The core steps include: - (you should present this section in prose/paragraphs)

- Defining measurable objectives (e.g., improving accuracy of automated responses in maternal healthcare QA).
- Collecting and preprocessing structured and unstructured healthcare data.
- Training and evaluating machine learning models to verify performance.

Example:

In maternal healthcare QA, the system could be trained to **answer patient FAQs** using data from clinical guidelines. An experiment might test whether adding additional context (such as patient history) improves the accuracy of generated answers.

3.2.1 Data Acquisition

Datasets will be sourced from:

- Public health repositories (e.g., **WHO maternal health datasets**, anonymized hospital records).
- Synthetic data generation tools (e.g., **Synthea**) if real-world data is limited, or manually curated QA datasets.

Example Dataset Contents:

Patient_ID	Age	Blood_Pressure	Previous_Complications	Risk_Level
001	32	120/80	None	Low
002	28	140/90	Preeclampsia	High

Additionally, for QA systems:

Question	Answer - (Gold)
What are symptoms of high blood pressure during pregnancy?	Headaches, vision changes, swelling.
When should I visit the hospital for early labor signs?	If you experience regular contractions, bleeding, or water breaking.

Important:

Ensure compliance with data privacy regulations (e.g., **HIPAA**, **GDPR**) when handling patient data.

3.2.2 Data Processing

Data processing will involve:

- **Cleaning:** Handling missing values (imputation or removal), managing outliers (e.g., capping), correcting inconsistencies.
- **Transformation:** Normalizing numerical data (e.g., Min-Max scaling), encoding categorical variables (e.g., One-Hot Encoding).
- **Augmentation (if required -** when your dataset is either imbalance or really small for training**):** Generating synthetic samples to address class imbalance (e.g., using **SMOTE** for high-risk pregnancy cases).

Example:

The "Previous_Complications" column (text data) may be transformed into **binary features** — e.g., a "Preeclampsia" column with values 1 (present) or 0 (absent).

For QA systems:

Preprocessing may also involve **tokenizing text**, **removing irrelevant terms**, and **indexing questions and answers** for faster retrieval or fine-tuning language models.

3.2.3 Model Training

The chosen model (e.g., **Random Forest**, **Neural Network**, **Transformer-based QA model** such as **BERT**) will be trained using:

- **Stratified k-fold cross-validation** to maintain balanced representation (e.g., for risk categories or question types).
- **Hyperparameter tuning** (e.g., **GridSearchCV**) to optimize performance.
- **Feature importance analysis** (where applicable), to identify key predictors (e.g., blood pressure, age).

Example:

A **Transformer-based QA model** (e.g., **BioBERT**) could be fine-tuned to answer patient questions based on a corpus of maternal healthcare guidelines and validated by comparing its generated answers to expert-reviewed responses.

3.2.4 Model Validation and Testing

The model will be evaluated through:

- **Metrics:** Precision, Recall, F1-score (especially important in imbalanced datasets), and **AUC-ROC** for probabilistic outputs.
- **Testing:** On a held-out test set (typically 20%) not used in training. This should not appear anywhere in training.
- **Explainability:** Using tools like **SHAP** or **LIME** to interpret predictions — critical in healthcare settings.

Example:

If a QA model achieves **90% F1-score** in correctly answering safety-critical questions (e.g., when to seek emergency care), it can be considered reliable for deployment in **maternal healthcare QA** applications. - [*In medical environments, we always need* ***human in the loop*** *to make reliable decisions and not using AI fully to determine the outcome on human lives*]

Example: User (asks questions/assign tasks) -> AI (get results) -> human in the loop (review - approve, reject or modify) -> Show User the results.

Note:

- The Numbering of your document subtitles should now adapt automatically to the numbering format of the template that was given in class.

3.2 System Development Methodology

Note:

You should structure the subtopics in this section based on the **specific content and scope of your project**. It is essential to work closely with your supervisor when developing this section, as your methodology may significantly differ from that of other projects.

In this section, clearly **justify your choice of system development methodology**. Explain how the selected methodology supports the nature of your solution and aligns with your project goals.

You may select from various methodologies depending on the type of system you are developing. For example:

- If using an **iterative and user-focused** approach, you may choose **Agile**, and should further specify which Agile framework (e.g., **Scrum**, **Kanban, incremental**).
- If focusing on **user-centered innovation**, **Design Thinking** may be appropriate.
- For **prototyping and fast delivery**, you might adopt **Rapid Application Development (RAD)**.
- If aiming for **continuous integration and delivery**, **DevOps/MLOps** practices can be integrated.
- If following a **structured and phased** process, consider a **Modified Waterfall** approach.
- For projects focused on **waste reduction and efficiency**, **Lean Development** principles may apply.

Ultimately, ensure that your choice of methodology is **well-reasoned** and that you explain how it will guide the **design, implementation, and evaluation** of your solution.

Example 1: *For a QA System in Maternal Healthcare (AI-based Software Project)*

For this project, the **Agile methodology**, specifically the **Scrum framework**, was selected to manage the iterative development of the Question Answering (QA) system

for maternal healthcare. Agile is well-suited to AI-driven projects where model performance needs continuous refinement through feedback and experimentation.

By using Scrum, the project team can deliver working components in short sprints — for example, progressively improving the QA model's ability to answer questions related to antenatal care, high-risk symptoms, or nutritional advice.

Additionally, the flexibility of Agile allows rapid incorporation of healthcare expert feedback after each sprint, ensuring the system remains accurate and clinically relevant.

Example 2: *For an IoT-based Maternal Healthcare Monitoring System*

This project adopts a **Modified Waterfall methodology** to guide the development of the IoT-based maternal healthcare monitoring system. The system involves both **hardware components** (wearable sensors) and **software services** (data processing, alert generation), which benefit from a structured, phased development approach.

The Modified Waterfall allows for clear documentation of **hardware integration**, **data transmission protocols**, and **cloud-based analytics**, while also supporting limited iterations during testing phases.

This approach ensures that each phase — from sensor calibration, to network setup, to real-time alert validation — is fully tested before deployment in a sensitive healthcare environment.

What informs the choice of a Development Approach/Methodology?

SSAD / SSADM (Structured Systems Analysis and Design Methodology)

SSAD/SSADM is **process-oriented** and **document-driven** — it works well with **linear or phased methodologies** where the system is designed upfront with clear documentation of requirements, data flows, and system structure.

Suited Methodologies:

✅ **Waterfall / Modified Waterfall**

- Matches the structured, sequential nature of SSAD/SSADM.
- Phases: Requirements → Design → Implementation → Testing → Deployment.

✅ **Rapid Application Development (RAD)** (with structured prototyping)

- Works if you're building prototypes based on **well-documented user requirements** and using **structured data models**.

✅ **V-Model (Verification & Validation Model)**

- Very structured — ensures validation at each stage, complements SSAD's rigorous documentation.

Use Case Example:

Building a hospital record system where data flows and processes must be clearly defined and validated before implementation.

OOAD (Object-Oriented Analysis and Design)

OOAD focuses on **modeling systems as objects** (with attributes and behaviors), and supports **iterative**, **flexible**, and **modular** development — best suited to **incremental and adaptive methodologies**.

Suited Methodologies:

✅ **Agile** (Scrum, Kanban, XP)

- Strong synergy: OOAD encourages **modular, incremental builds** — Agile delivers **working software in iterations**.

✅ **DevOps/MLOps**

- OOAD models (classes/objects) align well with **CI/CD pipelines** and **automated testing** in DevOps practices.

✅ **Lean Development**

- OOAD supports building **minimal viable product (MVP)** architectures which can be expanded gradually — ideal for Lean.

✅ **Design Thinking** (paired with OOAD for innovative solutions)

- OOAD naturally supports **prototyping** and **iterative user feedback**, which complements **Design Thinking.**

Use Case Example:

Developing an AI-powered maternal healthcare chatbot/intelligent conversational

agent where components (conversation engine, QA module, database models) are built as **independent objects**, refined iteratively.

Summary Table:

Development Paradigm	Suited Methodologies
SSAD / SSADM	Waterfall, Modified Waterfall, RAD (structured), V-Model
OOAD	Agile (Scrum, Kanban, XP), DevOps, Lean, Design Thinking

---

3.2.1 Justification of the Methodology

In this section, you should provide a **well-reasoned rationale** for selecting the chosen system development methodology. Clearly explain **why the methodology is appropriate** for your specific project — considering factors such as project complexity, flexibility needs, stakeholder involvement, and the nature of the system being developed (e.g., AI-based, IoT-based, or traditional software).

Your justification must be **supported by relevant literature** — cite authoritative sources (books, journal articles, conference papers, or best-practice guidelines). The goal is to demonstrate that your choice is **not arbitrary**, but informed by established knowledge and aligned with best practices in system development.

Example structure for students:

"The Agile Scrum methodology was selected for this project because it supports iterative development and continuous integration of feedback, which is essential when building AI-driven Question Answering systems for maternal healthcare (Beck et al., 2001; Schwaber & Sutherland, 2020). Since model accuracy and relevance require frequent validation by domain experts, the flexibility and collaborative nature of Scrum align well with project needs. Similar approaches have been successfully applied in healthcare AI projects (Smith et al., 2022)."

3.3 Methodology Diagram

In this section, you will present a **diagrammatic representation** of your chosen methodology. The diagram should be **appropriately referenced** — as you do not own the rights to external diagrams, you must properly **cite the source** of the diagram used.

After presenting the diagram, you should provide a **discussion of each stage of the methodology**, clearly explaining how each stage will be **applied to your specific project**.

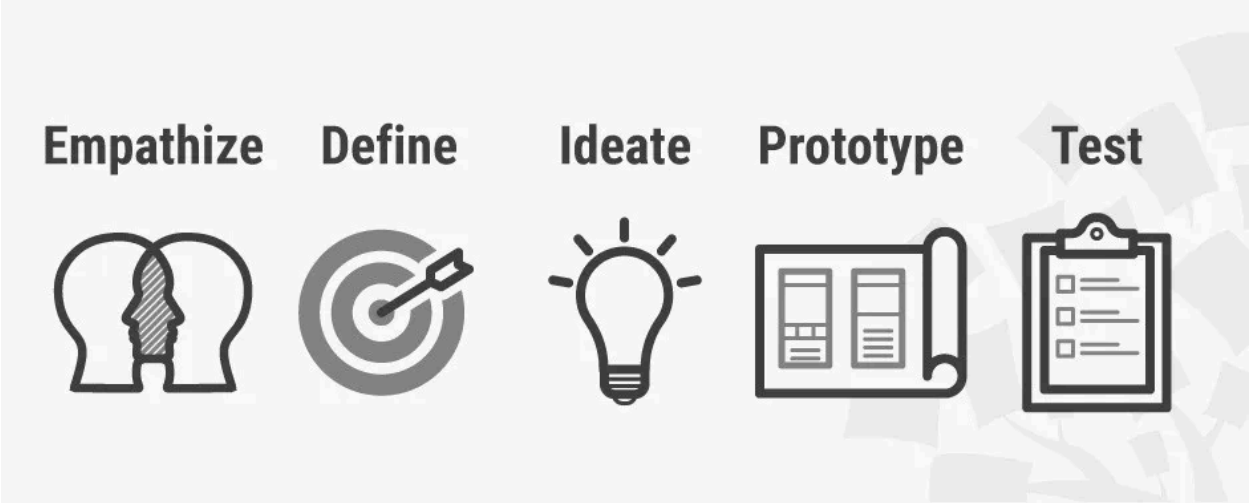
Note: - Follow the Explanation in Class on how this section Changes and has two different formats.

Two common methodology examples include:

- **Design Thinking**

- **Prototyping** (as part of **Agile Development**)

Example 1: Design Thinking



3.3.1 Empathize

In this stage, the goal is to **understand the needs, challenges, and context of the users**. It is critical because a deep understanding of user needs ensures that the solution will be both **usable** and **valuable**.

For this project, relevant data will primarily come from **secondary sources** such as published maternal healthcare studies, healthcare guidelines (e.g., WHO, MoH), and existing QA systems in healthcare.

You must explain **how this stage will inform your specific project** — for example, identifying common questions pregnant women ask in QA systems or understanding what types of alerts are most useful in IoT maternal monitoring.

3.3.2 Define

In this stage, you will **clearly define the problem statement** and outline key **model or solution parameters** based on insights from the Empathize stage.

This helps to ensure that the project is solving the **right problem** and that development is focused on clearly defined objectives.

For example, in an AI QA project, you might define the categories of healthcare questions your system will cover (e.g., nutrition, danger signs, antenatal care schedules).

3.3.3 Ideate

This stage involves **brainstorming and exploring potential solutions** to the problem. It encourages **creative thinking** and helps identify the most promising design approaches.

You will also specify the **design paradigm** chosen for your project (e.g., **OOAD** or **SSAD/SSADM**), and explain **why this paradigm was selected**.

The chosen paradigm will influence the type of design diagrams presented in Chapter 4 (refer to the provided table for guidance).

For example, if you choose **OOAD**, your diagrams may include **Class Diagrams**, **Sequence Diagrams**, and **Use Case Diagrams**.

3.3.4 Prototype

In this stage, you will build an **early version** of the solution to test concepts and gather feedback.

Clearly explain your **development approach** — will you use an **iterative approach** (Agile), a phased approach, etc.?

Also describe the **tools and technologies** you will use — for instance:

- Programming languages (e.g., Python, JavaScript)
- Frameworks (e.g., TensorFlow, React, Node.js)
- IoT platforms (e.g., Arduino, Raspberry Pi)
- QA frameworks (e.g., Hugging Face Transformers for NLP-based QA systems).

Justify **why these tools and techniques are appropriate** for your project.

3.3.5 Test

In this stage, you will conduct **rigorous testing** of your solution to ensure it meets the defined requirements and performs reliably.

Explain the **testing approaches** you will use and why they are suitable:

- **Unit Testing:** Testing individual components for correctness.

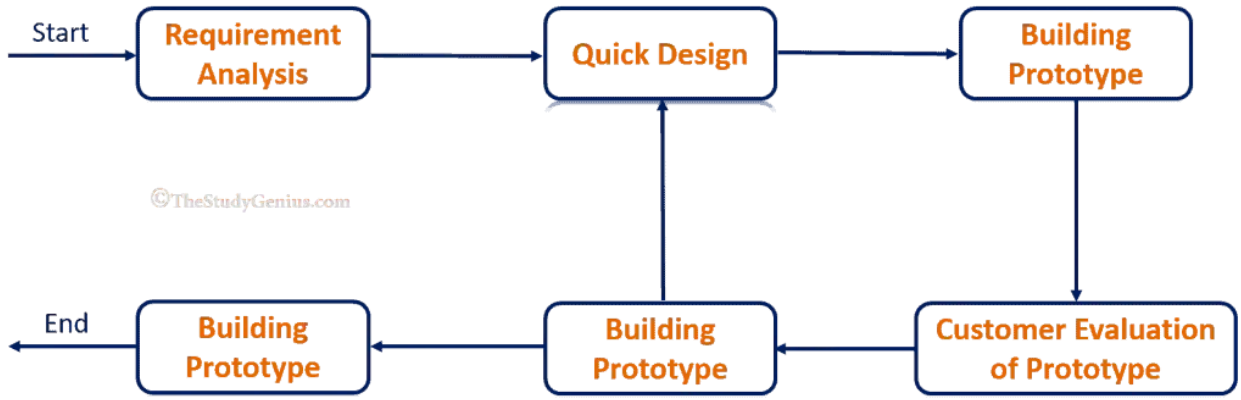
- **Black Box Testing:** Testing the system from a user's perspective without knowledge of internal code.
- **White Box Testing:** Testing internal logic and structure of the code.

You should also explain how you will evaluate **system performance**, for example:

- For QA models: **Accuracy testing, F1-score, precision, recall**.
- For IoT systems: **Sensor accuracy, alert response time, reliability**.

Finally, describe the **test cases** you will design to cover different usage scenarios.

Example 2: Prototyping (as part of Agile Development)



3.3.1 Prototype Planning (Equivalent of "Empathize + Define")

In Prototyping under Agile, the first step is to **understand the user needs** and **define the scope of the prototype**.

You will begin by gathering **user requirements** through secondary sources (published research, guidelines, previous systems) or direct stakeholder input (e.g., interviews with healthcare providers).

Clearly **define what the prototype will demonstrate** — for example:

- For a QA system: answering a core set of maternal health questions.
- For an IoT system: capturing and displaying vital signs from wearable sensors.

You must explain **why these functions are the priority** and how they align with project goals.

3.3.2 Initial Prototype Development (Ideate + Prototype)

This is where you **design and build the first version of the system** (often a minimal viable product or MVP).

You will:

- Describe your **design approach** — iterative and incremental.
- Mention the **design paradigm** (OOAD or SSAD/SSADM), and **why you chose it**.
    - If using OOAD, expect to create **Class Diagrams, Sequence Diagrams, Use Case Diagrams**.
    - If using SSAD/SSADM, expect to use **Data Flow Diagrams, Entity-Relationship Diagrams**.

Then outline your **development tools** and **tech stack**, for example:

- Programming languages
- Frameworks (e.g., Django, Flask, React)
- ML libraries (for QA models)
- IoT platforms (for IoT systems).

3.3.3 Iterative Refinement (Agile Sprints)

Agile-based Prototyping involves **repeated cycles (sprints)** where:

- The prototype is demonstrated to users/stakeholders.
- Feedback is collected.
- The prototype is refined and improved.

Explain how your team will conduct **sprint reviews** or **iteration cycles** — e.g.:

"After each 2-week sprint, the QA system will be tested with a sample of clinical questions. Feedback from domain experts will guide the next sprint's focus."

3.3.4 Testing

Describe your **testing approach** and types of testing used during Prototyping:

- **Unit Testing**: To verify correctness of individual components.
- **Integration Testing**: To ensure system components work together.
- **System Testing**: To test the prototype in an end-to-end scenario.

- **User Acceptance Testing (UAT)**: To gather feedback from end-users or healthcare experts.

Also specify performance testing:

- For QA systems: **accuracy of answers, precision, recall**.
- For IoT systems: **sensor accuracy, latency of data transmission, reliability of alerts**.

Clearly explain **why these testing methods are appropriate** for an **Agile prototyping approach** — which focuses on **iterative improvement** rather than exhaustive testing in the first release.

3.3.5 Final Prototype Evaluation

In this phase, evaluate whether the **prototype meets key project goals** and is ready for:

- Deployment
- Further development
- Scaling to full system

Describe:

- How test results will be documented.
- How stakeholder feedback will inform next steps.

- How success will be measured (e.g. user satisfaction, model accuracy thresholds, system stability).

Points to Note:

In some cases, your supervisor or examiner may require the Analysis and Design stages to be presented as separate, dedicated sections — rather than embedded within the overall methodology flow.

If this is the case, you can structure your Methodology chapter as follows:

- **Section 3.4: Analysis Diagrams**

  Present and explain all diagrams used to analyze system requirements (e.g., Data Flow Diagrams, Use Case Diagrams, Entity-Relationship Diagrams).
- **Section 3.5: Design Diagrams**

  Present and explain diagrams related to system architecture and design (e.g., Class Diagrams, Sequence Diagrams, Component Diagrams).

When using this approach, remember to **cross-reference these sections** within your main methodology stages (Empathize, Define, Ideate, Prototype, Test) — for example:

"Detailed analysis diagrams are presented in Section 3.4."

"The system design is further elaborated in Section 3.5."

Additionally, complete the chapter with:

- **Section 3.6: Deliverables**

  Outline the key outputs or products of the project (e.g., working prototype, QA model, IoT dashboard)

- **Section 3.7: Tools and Techniques**

  List and justify the tools, frameworks, platforms, and techniques used throughout development.

What informs the choice of Diagrams that you should draw for either OOAD or SSDM/SSADM?

OOAD	SSADM/SSDM	Belongs To
Use Case Diagram	Context Diagram	System Analysis
Use Case Descriptions *(Optional)*	DFD Level 0	System Analysis
Sequence Diagram	DFD Level 1	System Analysis & Design
Activity Diagram	Process Specifications	System Analysis
Class Diagram	ERD	System Analysis & Design
State Machine Diagram *(Optional)*	Structure Chart *(Optional)*	System Design
Database Schema	Database Schema	System Design
UI Mockups / Wireframes	UI Mockups / Wireframes	System Design
System Architecture	System Architecture	System Design
—	Network Topology *(Optional)*	System Design

This marks the end of Chapter 3. The first options of doing Chapter 3 are the best options.

References

All references should be formatted using the **APA referencing style (version/edition 7)**. If you have followed the correct referencing process from the start (see the shared video on how to use mendeley), your reference list should be **automatically generated** by your chosen reference management tool (might be zotero or others).

Important guidelines:

- **Avoid undated references** (e.g., "n.d."), unreliable websites, and non-academic sources such as **Wikipedia**.
- **Prioritize peer-reviewed journal articles**, **conference papers**, **books**, and **official publications** (e.g., WHO reports).

It is highly recommended to use reference management tools such as **Zotero**, **Mendeley**, or similar software to ensure consistent and accurate referencing throughout your document.

Appendices

An **Appendix** contains supplementary material that is not critical to the main text but helps provide **greater depth or additional context** to the research. It may also include material that is too detailed or bulky to fit into the main body of the report.

Examples of suitable content for the appendices:

- **Additional diagrams or images** that complement the Literature Review or Methodology sections (these should be cross-referenced in the main body of the report).
- **Gantt Chart** or **Work Plan** showing the project timeline up to project completion.
    - You may use tools such as **Google Sheets**, **Microsoft Project**, or dedicated Gantt Chart tools — choose tools that help you present a **clear and insightful timeline**.
    - **Note:** The project timeline should run up to the end **of November 2026** (with presentations scheduled before final exams - Reading week).

Formatting:

Use a **clear numbering system** for the appendices:

- **Appendix 1**, **Appendix 2**, and so on.
- Each appendix should be clearly titled.

Example:

- *Appendix 1: Gantt Chart*
- *Appendix 2: Feature Extraction in Convolutional Neural Networks (CNNs)*

Semester II: Documentation & System Development

Recall

Semester I Assessment and Transition

Semester I contributes 35% of the final unit grade: proposal progress (5%), written proposal (15%), and proposal oral defense (15%). Students should confirm that all three marks appear correctly on the AMS, compare them with the results and feedback already communicated, and report only genuine discrepancies through the official channel with supporting evidence.

Before Semester II implementation, students must resolve written, defense, classroom, and supervisor feedback; strengthen Chapters 1–3; and preserve alignment from the problem and gap through the objectives, research questions, literature review, methodology, implementation, testing, and evaluation. A feedback-resolution matrix should record the issue, source, affected section, action, and status. The revised work and implementation plan must be reviewed with the supervisor.

Writing the Final Documentation

Use the past tense for completed activities, the present tense for established knowledge and what the document, figures, or tables show, and the future tense only for work that is genuinely incomplete. Objectives may remain in infinitive form, but the

report must explain how each was achieved. Revise the abstract last so it reflects the actual methods, implementation, results, conclusions, and contribution.

Responsible Use of AI

AI may support brainstorming, literature discovery, data analysis, coding, testing, methodological review, and language improvement. It does not replace the student's judgement, authorship, accountability, or supervisor guidance.

Students must verify AI-assisted facts, citations, code, analyses, and interpretations against original sources or direct testing; protect confidential and personal data; disclose material AI use; maintain an AI-use log; respect copyright, bias, fairness, and reproducibility; and understand and defend every retained output. AI must never be used to fabricate citations, data, participants, experiments, results, metrics, screenshots, findings, or approvals.

To-Do: End of the Recall Session

- Check all Semester I marks on the AMS and report genuine discrepancies through the official channel.
- Resolve feedback using a feedback-resolution matrix and correct Chapters 1–3.
- Update proposal language as activities are completed and revise the abstract last.
- Confirm alignment among the problem, gap, objectives, research questions, methodology, implementation, testing, and evaluation.
- Maintain an AI-use log and verify and disclose material AI assistance.
- Review the revised documentation and Semester II implementation plan with the supervisor.

Chapter 4: System Analysis and Design

4.1 Introduction

Chapter 4 presents the analysis and design of the proposed system. It explains what the system must do, the quality conditions it must satisfy, the data and interfaces it depends on, and the diagrams used to communicate the system's structure and behaviour.

Keep the introduction concise. Briefly outline the sections covered, identify the analysis and design diagrams included, and restate the development paradigm or combination of approaches justified in Chapter 3—for example, Object-Oriented Analysis and Design (OOAD), Structured Systems Analysis and Design (SSAD/SSADM), or a justified data-centric, experimental or hybrid approach. The selected diagrams must fit the project rather than being included merely because they appear in a list.

4.2 System Requirements

System requirements describe the capabilities, qualities, interfaces, data and constraints that the complete system must satisfy. They are not limited to the minimum hardware or software configuration needed to install an application. Requirements should be clear, relevant, feasible, testable and traceable to the problem, objectives, system features and evaluation plan. The requirements-engineering guidance in ISO/IEC/IEEE 29148:2018 applies across different project sizes, products and development methodologies.

You may introduce the section using a statement such as: “The system requirements identified for this project are presented in the following subsections.”

📌 Writing Good Requirements

- Give each requirement a unique identifier, for example FR-01, NFR-01, DR-01 or IR-01.
- Write one clear requirement per statement and avoid vague words such as fast, secure, user-friendly or efficient unless they are supported by measurable criteria.
- Identify the relevant stakeholder, source or rationale where this is useful.
- State how the requirement will be verified through inspection, demonstration, testing, measurement or user acceptance.
- Maintain alignment among requirements, objectives, diagrams, implemented features and test cases.

Where practical, maintain a compact requirements catalogue containing the requirement ID, statement, priority, source or rationale, and verification method. Do not invent thresholds simply to make a requirement appear measurable; justify them using stakeholder needs, the intended operating environment, authoritative guidance, benchmarks or experiments.

4.2.1 Functional Requirements

Functional requirements describe what the system must do under specified conditions. They define services, behaviours, business rules, inputs, outputs and interactions that allow users or external systems to accomplish particular tasks.

Examples of correctly framed functional requirements include:

- FR-01: The system shall allow an authorised user to create an account and authenticate before accessing role-specific functions.
- FR-02: The system shall allow a user to search for and retrieve records using the defined search criteria.
- FR-03: The system shall notify the affected user after a specified transaction has been completed or rejected.

In your own documentation, list and explain only the functions required by your project. If you organise the system into modules—such as authentication, file transfer, data collection, reporting or administration—describe the actor, trigger, expected behaviour, output and important exceptions. The requirement should state what is required; detailed implementation evidence belongs in the implementation chapter.

4.2.2 Non-Functional Requirements

Non-functional requirements describe how well the system must operate and the constraints under which it must work. Select only the quality characteristics relevant to the project. ISO/IEC 25010:2023 provides a current product-quality model that can help students identify suitable quality areas.

Possible areas include:

- Performance efficiency, including response time, throughput and resource use.
- Security and privacy, including authentication, authorisation, confidentiality, integrity and accountability.

- Reliability and availability, including recovery from failures and continuity of service.
- Usability and accessibility for the intended users and operating context.
- Compatibility and interoperability with required platforms or services.
- Maintainability, testability and modifiability.
- Safety, fairness, explainability or human oversight where the project can affect people or other critical decisions.

Use measurable, project-specific statements. Suitable patterns include:

- Under [defined workload and environment], at least [X%] of [specified transactions] shall complete within [Y seconds], as measured using [identified test].
- Only users assigned the required role shall access [specified data or function], as verified through access-control tests.
- The system shall recover from [defined failure] within [specified period] without losing [identified critical data].

For AI/ML projects, model quality, robustness, fairness and explainability may form part of the quality and evaluation requirements. For IoT projects, relevant requirements may include latency, sensor accuracy, connectivity, power consumption and reliability. These examples must still be justified and measured within the actual project context.

4.2.3 Data Requirements

Data requirements apply to all project types, not only AI/ML or IoT. They describe the data the system needs, produces, protects and retains.

Discuss the following where applicable:

- Required inputs, outputs, data sources and ownership.
- Data formats, structures, validation rules and permitted values.
- Data quality, completeness, accuracy, provenance and handling of missing or duplicate records.
- Storage, retention, backup, archival and deletion.
- Privacy, confidentiality, consent and access restrictions.
- Transformations, preprocessing and audit information needed to reproduce the work.

Adapt the discussion to the project. AI/ML projects should address datasets, labels, sampling, train/validation/test separation, representativeness and bias. IoT projects should address sensor types, units, sampling frequency, calibration and missing readings. Information systems should describe records, validation and retention, while cybersecurity projects may focus on logs, telemetry, indicators and provenance.

4.2.4 Interface and Integration Requirements

Interface and integration requirements are also general. They describe how users, external services, software components and hardware devices interact with the system.

Discuss the following where applicable:

- User-interface and accessibility requirements.
- APIs, third-party platforms and external data services.

- Authentication or institutional identity services.
- Hardware devices, sensors, gateways and peripheral equipment.
- Communication protocols, message formats and compatibility constraints.
- Timeouts, retries, validation and error handling when an integration is unavailable or returns invalid data.

For example, an IoT project may describe sensor-to-gateway and gateway-to-cloud communication, while an AI project may describe the interface through which a trained model receives input and returns predictions. A conventional information system may instead focus on web, mobile, payment, messaging or institutional-service integrations.

🔗 Requirements Traceability

Every important requirement should link back to an objective or stakeholder need and forward to a feature, diagram and test. When a requirement changes, review the affected design and evaluation evidence. Discuss significant changes with the supervisor.

Discussion of Diagrams and Their Purposes

System analysis and design diagrams help students visualise, examine and communicate different aspects of a proposed solution. They support:

- Understanding and validating system requirements.
- Defining the system boundary and external interactions.
- Representing data, processes, behaviour and solution structure.
- Identifying omissions, inconsistencies, risks and design alternatives.

- Communicating with supervisors, users, developers and other stakeholders.
- Supporting implementation, testing, evaluation and future maintenance.

Use the diagram-selection table already provided in this guide to identify the diagrams appropriate to your project and chosen paradigm. The table is a guide, not a checklist requiring every diagram. Confirm the final selection with the supervisor.

OOAD projects will normally use relevant UML diagrams. SSAD/SSADM projects will normally use context diagrams and data-flow diagrams. AI/ML, IoT, blockchain, cybersecurity and other specialised projects may require a justified combination of UML, structured-analysis, data-pipeline, block, network, architecture or state-oriented diagrams.

📌 Important Distinction: Context Diagram and DFD Level 0

A context diagram represents the entire system as one process and shows its interactions with external entities. A DFD Level 0 decomposes that system into its major internal processes, data stores and data flows. They are related, but they are not the same diagram.

4.3 System Analysis Diagrams

This section presents the diagrams used to analyse the problem, requirements, actors, processes, data and system boundary. The selected diagrams should be consistent with the paradigm justified in Chapter 3.

For every diagram, use the following presentation pattern:

- Narrative before the diagram: explain what the diagram represents, why it is relevant, its scope and the requirements or objectives it addresses.
- Diagram: use the correct notation and show only the information needed for that view.
- Figure caption: number and caption every diagram according to the course formatting guidance.
- Interpretation after the diagram: explain the important flows, relationships, decisions and implications instead of merely repeating labels visible in the figure.

All labels must remain readable at normal document size. Use consistent names across diagrams and keep the editable source files. A diagram that cannot be read, explained or traced to the project requirements does not provide useful evidence.

4.3.1 Use Case Diagram — Worked Example

A use case diagram provides a high-level view of how external actors interact with the system and which services the system exposes. Identify the system boundary, actors, use cases and associations. Use include, extend and generalisation only when their UML meanings are justified.

The use-case example included in this guide demonstrates the expected narrative–diagram–interpretation sequence. It is an example of a presentation, not a diagram that every student should copy. Each student must develop a diagram based on the actors and functions of the actual project.

Points to Note

- Explain the narrative and relevance of the diagram before presenting it.
- Keep external actors outside the system boundary and use clear verb-based names for use cases.
- Do not add databases, internal code modules or technologies as actors unless they genuinely represent an external interacting system.
- Caption the diagram and interpret its important relationships after the figure.
- Revisit the course formatting tutorial and discuss any uncertain notation with the supervisor.

4.4 System Design Diagrams

System design diagrams describe how the proposed solution will be organised to satisfy the analysed requirements. They may show components, classes, data structures, interfaces, deployment, states, networks, hardware blocks, user interfaces or other implementation-oriented views.

Use the same narrative–diagram–interpretation sequence applied in Section 4.3. Each design diagram should make a clear contribution to the project and remain consistent with the analysis diagrams, requirements and terminology.

4.4.1 Logical Database Schema — Worked Example

For projects that use a conventional database, Chapter 4 should normally present a logical database schema rather than a physical storage design. Show the main entities

or tables, important attributes, primary and foreign keys, relationships and cardinalities. Explain significant normalisation or integrity decisions where relevant.

A physical schema contains implementation-specific details such as exact DBMS data types, indexes, partitions, storage configuration and performance tuning. Include these details only when they are necessary to justify the design or evaluation and have been discussed with the supervisor.

Do not force a relational schema onto every project. A **NoSQL** system may instead show collections or documents and explain embedding and referencing decisions. An AI/ML project without a transactional database may require a data-pipeline or feature-store design, while an IoT project may need a telemetry-storage and data-flow design.

The logical-schema example in this guide is the second worked example. Students should use the same style and academic language for whichever design diagrams are appropriate to their projects.

🛠️ Class Guidance: Diagram Quality and Tools

This is guidance for preparing Chapter 4 and is not a heading that students must copy into the final report. Use tools that support the required notation and allow legitimate, clean export:

- **Visual Paradigm** — suitable for formal UML, DFD, ERD and architecture modelling; its AI diagramming assistant can generate and refine model drafts.
- **StarUML** — suitable for UML, ERD, DFD, C4 and other software-modelling views.

- **PlantUML** — useful for reproducible diagram-as-code workflows, especially use case, sequence, class, activity, component, deployment and state diagrams.
- **draw.io/diagrams.net** — a strong general-purpose choice for context diagrams, DFDs, architecture, block diagrams and network topologies.
- **Microsoft Visio** — suitable for process, UML, ERD and network diagrams where the required licence is available.
- **Figma or Penpot** — better suited to UI wireframes, screen flows and interactive prototypes.
- **Mermaid Chart** — useful for AI-assisted and text-based flowcharts, sequence diagrams, class diagrams, ER diagrams and architecture views.

Tool pricing, trials and AI availability change, so confirm the current terms before relying on a tool. Export using SVG or PDF where supported, or a clear high-resolution PNG. Keep the editable source file. Do not hide, crop or bypass a watermark; use a properly licensed, institutionally available or open tool.

Student access can support design verification as well as coding. For example, the Polypane student offer supports responsive and accessibility checks, while selected testing tools in the GitHub Student Developer Pack can help verify browser behaviour. Follow the redemption and usage guidance in the Build Phase; retain editable designs and evidence of the issues found and corrected.

🤖 Responsible AI-Assisted Diagramming

AI can help generate a first draft, convert a narrative into diagram code, suggest missing interactions or improve layout. It does not determine whether the project

requirements are correct, and it does not remove the student's responsibility to understand and defend the model.

- Verify every actor, use case, process, relationship, data store, component and label against the approved project requirements.
- Do not accept invented functionality merely because an AI-generated diagram looks professional.
- Record the tool, model name where disclosed, exact prompt, important iterations, retained output, corrections and verification in the AI-use log.
- Do not upload confidential, personal, proprietary or identifiable project data to a public AI tool without authorisation and appropriate safeguards.
- Use AI output as a draft that must be reviewed and corrected, not as evidence that the design is valid.

To-Do: End of the Chapter 4 Session

This activity compares your own modelling decisions with AI-assisted diagram generation. The comparison—not the attractiveness of the first AI output—is the main evidence being assessed.

Common Use-Case Narrative

An online project-supervision portal allows a student to authenticate through the university identity service, upload a project document, submit a revision and request a supervision meeting. A supervisor reviews submissions, provides feedback, approves a submission or requests a revision, and confirms meeting requests. A project

coordinator assigns supervisors, monitors submission status and schedules proposal or final defences. When a submission, review decision, meeting confirmation or defence schedule is recorded, the system sends a notification through an external notification service. Users may access only the functions permitted by their assigned roles.

Use This Same Instruction with Each AI Tool

Using only the supplied narrative, produce a UML use case diagram. Show the system boundary, external actors, use cases and associations. Use include, extend or generalisation only where logically justified. Do not invent requirements that are not present in the narrative. Return an editable diagram or diagram source where the tool supports it.

What You Must Do

- Step 1 — Read the narrative and draw a UML use case diagram yourself using an appropriate diagramming tool. Complete the manual diagram before consulting an AI diagram generator.
- Step 2 — Research at least three AI-assisted diagramming tools or model-based workflows. Compare their notation support, editability, privacy, export quality, cost or access, and reproducibility.
- Step 3 — Select two AI-assisted options and give each the exact same narrative and instruction shown above.
- Step 4 — Record the product, model name if disclosed, access date, exact prompt, number of refinement attempts and any manual corrections. If the underlying model is not disclosed, state that instead of guessing.

- Step 5 — Compare your manual diagram with both AI-generated diagrams using the criteria below.
- Step 6 — Correct the strongest diagram manually before using it in Chapter 4. You must understand and be able to defend every element that remains.
- Step 7 — Discuss the final diagram and your comparison with the supervisor.

Comparison Criteria

- Coverage and traceability to the supplied narrative.
- Correct identification of actors, use cases and the system boundary.
- Correct UML notation and use of relationships.
- Absence of invented or unsupported functionality.
- Readability, organisation and consistency.
- Editability and reproducibility of the output.
- Time, effort and number of iterations required.
- Privacy, transparency and responsible-use considerations.

✅ Required Submission

- Your manually produced use case diagram.
- The two AI-generated diagrams.
- A concise comparison matrix using the stated criteria.
- The corrected final diagram and a short justification for the approach selected.
- The relevant AI-use log entries, including prompts and verification performed.

The purpose of this exercise is not to prove that AI is better than manual modelling. It is to determine where AI assists the modelling process, where it introduces errors, and

what human judgement is required to produce a clear, accurate and defensible system design.

Chapter 5: System Implementation, Testing and Evaluation

Note: As you read these notes, please refer to the shared sample documentations in your Google Classroom.

5.1 Introduction

Chapter 5 explains how the approved design was implemented, made available in a suitable environment, tested and evaluated. It should show what was actually built, how the components work together, the evidence produced during implementation, and the extent to which the completed solution satisfies the objectives and requirements.

The organisation of this chapter will not be identical for every project. Select and number only the subsections that fit the project type, then agree on the final structure with the supervisor. The routes below cover general software and information systems, AI and machine learning, IoT and embedded systems, cybersecurity, blockchain and other specialised or hybrid projects. They are guides—not headings that every student must copy. Chapter 5 should finish with evaluated results, discussion and limitations; the final conclusions, recommendations and future works belong in Chapter 6.

📌 Writing Perspective and Evidence

Write mainly in the past tense because implementation and testing describe completed work. Use the present tense when explaining what the finished system, table or figure shows. Write in the third person and support claims with traceable evidence such as configuration records, repository history, test results, logs, measurements, figures and supervisor-verified demonstrations. Screenshots may support the discussion, but screenshots alone are not proof that a requirement was met.

5.2 Implementation Environment

The implementation environment describes where the solution was developed, tested and deployed, together with the hardware, software, services and configuration required to reproduce or use it. Distinguish the development environment from the test, staging, pilot or production environment. Do not state that a system is publicly available unless it has actually been deployed for public access; describe a local prototype, controlled pilot or private deployment accurately.

State the minimum configuration that was actually tested and justified. Avoid copying outdated platform examples or vendor marketing specifications. Versions, service tiers and hardware thresholds should come from the implemented solution, measurements, official documentation or controlled experiments.

5.2.1 Hardware and Deployment Environment

Describe the client devices, servers, cloud resources, storage, network connectivity and specialised hardware used by the project. Explain both the development or training

equipment and the minimum environment required to run the finished solution. Where the project uses a managed cloud service, record the relevant service, region where material, instance or tier, storage, network assumptions and scaling limits without exposing credentials.

Use a compact table such as **Table 5.1.** Replace the guidance text with project-specific values and evidence. A stated minimum must be supported by a test, measurement, datasheet or provider configuration rather than guesswork.

Table 5.1: Implementation environment and minimum requirements

Item or layer	Minimum tested specification or version	Purpose and justification	Evidence or source
Client device	State processor, memory, storage, operating system and connectivity.	Explain the intended user tasks and the tested minimum.	Device test, measurement or official documentation.
Server or cloud runtime	State compute, memory, storage, service tier and network.	Explain workload, availability and scaling assumptions.	Deployment configuration and benchmark.
Specialised hardware	State the device, sensor, gateway, accelerator or peripheral.	Explain its role, interface and selection reason.	Datasheet, calibration record or experiment.

Additional Evidence for IoT and Embedded Projects

For an IoT or embedded project, identify each sensor, actuator, microcontroller, gateway, communication module, power component and external service that materially affects the solution. Explain its role, interface, operating range, calibration or accuracy where relevant, and why it was selected. Show the end-to-end chain from

sensing or input, through local processing and communication, to the action, alert or stored record.

Use one clear, properly attributed image for each distinct component only where the image helps identification. A part number or model is often more useful than a decorative picture. Table 5.2 provides a suitable structure.

Table 5.2: IoT or embedded component evidence

Component	Image or model/part number	Role and connection	Key specification and justification
Sensor or input device	Insert one clear image or state the model/part number.	State the variable, unit, interface and connection.	State the range, accuracy, calibration and selection reason.
Actuator, controller or gateway	Insert one clear image or state the model/part number.	State the control, processing or communication role.	State the power, protocol or performance specification and selection reason.

5.2.2 Software, Services and Configuration

Identify the operating systems, runtimes, frameworks, libraries, databases, application servers, package managers, container tools, mobile platforms, cloud services and external APIs required to build and run the project. Record the exact versions or version ranges that were tested, their purpose, and the reason they were selected.

Discuss Docker, npm, Composer, Firebase, messaging services, email services or similar technologies only when they were genuinely used. Explain the configuration and

role of each relevant dependency rather than listing product names. If containers are used, document the images, environment variables, volumes, ports and deployment differences; the Docker Compose production guidance is a useful starting point. Never expose passwords, API keys, tokens, private certificates or personal data in the report, screenshots or repository.

5.2.3 Deployment and Reproducibility

Provide a concise, reproducible account of how the solution is installed, configured, started and verified. Include the dependency or environment files, database migrations, seed or sample data where lawful, model or firmware version, build commands, deployment steps, and a simple health check or demonstration procedure. State any assumptions, paid services, licences, restricted datasets or network dependencies that another authorised person would need.

🧭 Select the Route That Fits the Project

- General software and information systems — emphasise application modules, database implementation, interfaces, access control, integrations, deployment and user workflows.
- AI and machine learning — emphasise dataset provenance, data preparation, experiment design, model training and selection, evaluation, model packaging, inference and monitoring.
- IoT and embedded systems — emphasise component assembly, circuit and firmware implementation, protocols, calibration, power, connectivity, gateway or cloud integration and end-to-end operation.

- Cybersecurity — emphasise the authorised test environment, assets and threats, control implementation, detection or protection logic, safe attack simulation, evidence handling and repeatable evaluation.
- Blockchain or distributed systems — emphasise network choice, smart-contract or protocol implementation, identities and keys, transaction flow, deployment, cost, security and performance.
- Other or hybrid projects — define an evidence path that matches the objectives, system boundary and evaluation plan, then confirm it with the supervisor.

5.3 System Implementation

Describe implementation in an order that matches the architecture, modules, data or experimental pipeline shown in Chapter 4. For each major component, link the relevant requirement and design, explain the chosen technology and implemented behaviour, show how it integrates with other components, and identify evidence that it works. Record material deviations from the approved design and justify them. Do not turn the chapter into a code listing; use short extracts only when they explain a non-obvious algorithm, interface, configuration or safety decision.

5.3.1 Implemented Modules, Integration and User Workflows

For a general software or information-system project, organise implementation around the modules and the user's tasks. A suitable sequence is database implementation, application modules, integration and user workflows, followed by testing and discussion. Use headings that describe the actual system and adapt the numbering with your supervisor. The example below uses a maternal healthcare appointment and

information portal; the wording is illustrative and should be replaced with evidence from the student's implementation.

🗃️ Database and Data Implementation

Explain how the Chapter 4 data design was implemented: tables or collections, important fields, relationships, validation rules and access permissions. Identify any imported records or sample data and explain their preparation. Show a small, anonymised sample or relevant database extract, then explain how it supports a requirement.

✍️ Example: "The database stored user accounts, appointment slots and bookings. Each booking linked a user to an available slot and recorded its status. Validation prevented incomplete bookings, while access permissions restricted users to the records permitted by their role." Adapt this paragraph to the actual schema and explain how duplicate or conflicting bookings were handled.

⚙️ Application Modules and Business Rules

Describe each major module's purpose, inputs, processing and outputs, together with the technology used and the related requirement. For the maternal portal, these might include registration, appointment booking, approved information pages and staff administration. Explain how authentication, permissions and booking rules were implemented. Include short code extracts only where they clarify an important decision.

🔗 Integration and User Workflows

Follow one complete task through the interface, application logic and database. In the portal example, a user signs in, selects an available appointment, submits a booking and receives confirmation. Explain how the booking is stored and becomes visible to authorised staff. Where reminders or external services are used, describe the connection and what happens if delivery fails. Use numbered, captioned screenshots to show meaningful steps, including validation or error messages.

🧪 Testing and Results

Use the test-case format in Section 5.4.2 of these notes to compare expected and actual outcomes. For the portal, test successful booking, missing details, an unavailable slot, unauthorised access and cancellation. Add performance and recovery tests where required. For user acceptance or usability evaluation, report who participated, the tasks attempted and the findings. Discuss whether the evidence supports the objectives; a completed booking test does not by itself establish that users found the portal easy to use.

5.3.2 Data and Model Implementation for AI/ML Projects

For an ML project, organise the chapter around the data-to-model workflow. Elvis's Chapter 5 illustrates this clearly: Dataset Description comes before Model Training and Evaluation, followed by Functional Testing and Results and Discussion. You may therefore use "5.3 Dataset Description" and "5.4 Model Training and Evaluation" in your own report, then renumber the remaining relevant sections. The headings in these

class notes show alternative project routes; select and adapt them with your supervisor.

📊 Dataset Description

Introduce each dataset by name and source, explain why it suits the research objective, and describe what one record contains. Report the actual size, language, topic or class distribution, ownership or licence, inclusion and exclusion criteria, and any privacy restrictions. Explain the contribution of each source when combining datasets. Include a small, captioned sample and a summary of the final dataset; discuss what they show.

✍️ Maternal Healthcare Question Answering — illustrative wording: “The dataset comprised maternal healthcare questions paired with reference answers and their supporting source passages. Each record contained a question, reference answer, source identifier and topic label. The topic labels were used to examine coverage across antenatal, childbirth and postnatal questions.” Adapt this description to the actual records, cite the sources, and add the measured counts and languages. This example does not establish that a dataset has been collected or clinically reviewed.

🧹 Data Preparation and Partitioning

Explain how the raw data became usable model input: missing or duplicate records, corrections, exclusions, feature extraction or tokenisation, and the final training, validation and test split. State counts as well as proportions and justify the split. Keep related records together where needed to prevent leakage; fit learned preprocessing on training data only. Use augmentation or encoding only when appropriate to the task.

For the maternal QA example, describe how question–answer pairs were checked against their source passages, how incomplete records were handled, and how questions were formatted for the selected model. Keep duplicate questions and their paraphrases in the same partition. Explain any document-level grouping and whether evaluation tests new questions from known sources or questions from unseen sources. Show one permitted sample before and after preparation, with a short explanation of the change.

⚙️ Model Training and Selection

Introduce the baseline and candidate models, then explain their inputs, outputs and selection rationale. Record the model version, important hyperparameters, random seeds, training environment, stopping rule and saved artefacts. Justify cross-validation when used; the number of folds must suit the data. Select the final configuration using validation evidence and reserve the test set for final evaluation. Retain configuration files and experiment logs so the reported run can be reproduced.

For a fine-tuned maternal QA model, compare the original pretrained model with the fine-tuned version under the same evaluation conditions. Explain how questions and any supplied context become input and reference answers become training targets. Report the actual learning rate, batch size, epochs and token limits. If the project instead uses retrieval-augmented generation, describe passage preparation, indexing, retrieval and prompting; report model training only if it occurred.

🧪 Model Evaluation and Application Testing

Choose evaluation measures for the task. For maternal QA, explain the selected answer-quality measures and inspect examples of correct, incomplete and unsupported answers. Exact Match or token-level F1 may support comparison with reference answers where suitable, but text overlap alone does not establish clinical correctness. Describe any qualified domain review actually conducted, including the rubric and sample evaluated. Report measured results and limitations before drawing conclusions.

Then show how the selected model works in the application: a user submits a question, the system processes it, and an answer is displayed. Test valid, empty, out-of-scope and unsupported questions, response time, and the intended fallback or referral behaviour. Present expected and actual outcomes using the test-case format in Section 5.4.2 of these notes. In the discussion, connect model results and application tests to the research objectives. Use actual outputs; the example provides a writing sequence, not findings to copy.

5.3.3 Hardware, Firmware and Integration for IoT/Embedded Projects

For an IoT or embedded project, follow the physical-to-software sequence: component assembly, firmware and data acquisition, communication and integration, then testing and discussion. You may use “5.3 Hardware and Firmware Implementation” with these as subsections, adapting the remaining numbering to your project. The example below uses a maternal healthcare monitoring prototype. Describe the devices and behaviour actually implemented, and treat the example wording as illustrative.

🔌 Component Assembly and Connections

Build on the component table in Section 5.2 by explaining how the selected sensors, controller, gateway, power supply and any actuators were connected. State the relevant pins, interfaces and power requirements. Include a labelled photograph of the assembled prototype and relate it to the circuit or architecture in Chapter 4. Explain and justify any changes made during implementation.

✍️ Example: “The prototype connected the selected sensor to a microcontroller, which collected readings and forwarded them to a monitoring dashboard through a gateway. Each transmitted record contained a device identifier, timestamp, measured value and unit.” Replace the generic component names with the actual devices and explain the connections. Distinguish simulated inputs, bench measurements and any authorised participant measurements.

⚙️ Firmware and Data Acquisition

Explain how the firmware initialises the device, samples the sensor, checks readings and prepares records for transmission. State the sampling interval, units, calibration procedure and handling of missing or invalid readings. For the monitoring example, describe how a reading is compared with a reference instrument or controlled input and how measurement error is recorded. If alerts use thresholds, justify their source and show the implemented decision rule; a prototype threshold is not evidence of clinical validity.

📡 Communication and System Integration

Trace a reading from the device through the chosen protocol, gateway or server to storage and display. Explain message format, device authentication, buffering and retry behaviour where implemented. In the example, show how the dashboard associates a reading with the correct device, displays its timestamp and indicates stale data. Describe what happens when the sensor disconnects, the network fails or power is restored, including whether queued readings are recovered without duplication.

🧪 Component and End-to-End Testing

Test individual components before the complete workflow. For the monitoring prototype, present calibration error, reading-to-display delay, successful message delivery and recovery after disconnection, with the test conditions and repetitions. Evaluate power use or communication range where relevant to the requirements. Use Section 5.4.2's test-case format for expected and actual behaviour, including alerts and failures. Retain circuit photographs, serial logs and message traces, and explain what each item demonstrates. Discuss the limits of the tested conditions and any remaining faults.

5.3.4 Cybersecurity, Blockchain and Other Specialised Projects

Cybersecurity projects should define the authorised laboratory or target scope, threat assumptions, datasets or traffic, control or detection implementation, tool configuration and safeguards used to prevent harm. Never test a third-party system without explicit authorisation. Blockchain projects should document the network, contracts or protocol components, deployment addresses or identifiers where safe,

account and key-management approach, transaction lifecycle, oracle or off-chain dependencies and relevant cost or performance decisions.

Projects outside the listed categories should build an equivalent implementation narrative around their actual artefacts and evidence. The guiding question is not “Which template can be copied?” but “What must a reader know and verify to reproduce, operate and evaluate this solution?”

5.4 Testing and Evaluation

Testing should demonstrate whether the implemented system satisfies its requirements and whether the project objectives were achieved. Begin with the test strategy, then present test cases, results, interpretation, defects and retesting. The general concepts in ISO/IEC/IEEE 29119-1:2022 can help distinguish test planning, design, execution, reporting and incident management.

5.4.1 Test Strategy and Acceptance Criteria

- Define the purpose, scope, environment, test data, responsibilities and completion criteria before presenting results. Select complementary test levels and techniques because they answer different questions: unit testing checks individual components, integration testing checks interfaces, system testing checks the complete solution, and user acceptance or usability testing checks fitness for intended users where applicable.
- Use black-box, white-box or grey-box techniques according to the available knowledge and test objective; they are perspectives, not substitutes for test

levels. Include positive, negative, boundary and exception cases. Add performance, reliability, compatibility, accessibility, security or recovery tests when the corresponding requirements exist.

- Map every important test to a requirement, feature or objective. Record the tool and version, configuration, repetitions, workload and acceptance threshold. A test verdict must compare the observed result with a predefined expected result; it must not be assigned because the system appeared to work once.
- For web-security testing, the OWASP Web Security Testing Guide provides project-specific test ideas. Use authorised targets, protect evidence and report vulnerabilities responsibly. An automated scanner or an SQL-injection screenshot is not a complete security evaluation; combine suitable review, negative testing and manual verification.

5.4.2 Test Cases, Defects and Retesting

Present repeatable test cases and their observed outcomes. Table 5.3 is a suitable template. Test data must be lawful, non-sensitive where possible, and sufficiently described for another authorised evaluator to repeat the test. Replace the illustrative rows with cases from the actual project.

Table 5.3: Test cases and results

Test ID	Traceability	Scenario, data and conditions	Expected result	Actual result, evidence and verdict
TC-001	FR-01 / Authentication	Valid and invalid credentials; include	Authorised users gain permitted access; invalid	Record the observed result, evidence

		boundary and negative inputs.	input is rejected safely.	reference and Pass/Fail.
TC-002	NFR-01 / Performance	State the workload, device and network conditions, tool and repetitions.	State the measurable threshold defined in the requirement.	Report the measured result, evidence reference and Pass/Fail.

Where many tests exist, summarise them by module or requirement and place detailed logs in an appendix or repository. Record failed tests, defect identifiers, corrective changes and retest outcomes. Do not delete or conceal a failed result merely because a later version passed.

5.4.3 Project-Type-Specific Evaluation

- General software and information systems — report functional coverage, user acceptance or usability evidence, performance, reliability, security and data-integrity results that correspond to the requirements.
- AI and machine learning — compare against a meaningful baseline; report task-appropriate metrics, class-wise or subgroup results where relevant, confusion or error analysis, variation across runs or folds, and final held-out test performance. Explain false positives, false negatives or other consequential errors.
- IoT and embedded systems — report component and circuit tests, calibration or measurement error, communication range or packet delivery, latency, power behaviour, recovery from disconnection and end-to-end scenario results.

- Cybersecurity — report coverage, detection or prevention effectiveness, false positives and false negatives, time to detect or respond, resource overhead and safely reproduced attack scenarios where authorised.
- Blockchain or distributed systems — report smart-contract or protocol correctness, access and key controls, transaction success, cost, latency, throughput, finality and behaviour under defined failure or network conditions.
- Human-centred or decision-support systems — include usability, accessibility, task completion, user acceptance and appropriate human-oversight evidence in addition to technical performance.

5.4.4 Results Presentation and Interpretation

Use tables for precise comparisons and figures for patterns, distributions, trends or error analysis. A figure should show an insight, not merely that a notebook or test tool was opened. Number and caption each table and figure, state the test conditions, and discuss what the result means for the requirement, objective and intended user.

Separate the expected result from the observed result. Report sample size, repetitions and variability where they affect interpretation. Do not generalise beyond the evaluated environment or population, and do not fabricate results, logs, screenshots, users or measurements.

5.5 Version Control and Implementation Evidence

Provide the project repository link on a readable label such as Project repository and ensure the supervisor can access it. The repository should make development history

and reproducibility visible. GitHub repository guidance explains the role of repositories, branches and collaboration, while GitHub flow provides a simple branch-based workflow.

Verified students should check GitHub Education before purchasing development services. GitHub Pro and the included Codespaces and Actions allowances can support private collaboration, reproducible environments and automated tests. The Build Phase explains how to apply the relevant offers at each sprint. Record what was actually used and monitor the account's limits.

- Include a clear README with the problem, setup, configuration, run and test instructions; meaningful commits linked to implemented work; justified branches and pull requests where used; issues or another traceable task record; dependency and environment files; release tags or versions for important milestones; and an appropriate licence or access statement.
- Use a suitable .gitignore and remove credentials, secret keys, personal data, large private datasets and generated build files. If such material was committed, removing the visible file is not enough; rotate the exposed credential and follow the platform's remediation guidance.
- Repository screenshots may support the report, but they do not replace an accessible repository and its history. Explain how the selected workflow supported versioning, experimentation, review or collaboration rather than merely listing branch names.

5.6 Author Oversight and Use of AI-Assisted Tools

Where AI-assisted tools were used, include a short declaration naming the tool and provider, explaining what it assisted with and how its output was checked, and confirming that the student remained responsible for the work. Keep the declaration factual and specific to the actual project; replace the example details with the actual tool and workflow.

✍️ Example

OpenAI Codex was used to assist with code review and language refinement in selected sections of this report. Its suggestions were checked against the source code, test results and cited sources. The student made the final decisions and takes full responsibility for the submitted work.

5.7 Evaluation Results, Discussion and Limitations

Organise this section around the research objectives, important requirements or evaluation questions—not around the order in which screenshots were collected. For each objective, state the implemented output, the relevant evidence, the acceptance criterion, the observed result and what the result permits the study to conclude. Use exact measurements, sample sizes, repetitions and uncertainty where they matter.

Discuss the results rather than repeating tables. Explain why important outcomes occurred, compare them with the selected baseline, Chapter 2 literature or justified expectations where appropriate, and analyse failures and consequential errors. A general software project may discuss requirement coverage and user outcomes; an AI

project may discuss baseline differences, error patterns and generalisation; an IoT project may discuss calibration, communication and field conditions; and a security or blockchain project may discuss coverage, overhead, cost and behaviour under defined threats or failures.

State limitations honestly and explain their likely effect on the findings. Relevant limitations may involve data quality or representativeness, participants, devices, infrastructure, deployment duration, operating conditions, security coverage, cost, access, external services or evaluation design. Describe any mitigation already applied, but do not convert an unresolved limitation into a stronger claim. The final synthesis, recommendations and future work belong in Chapter 6.

To-Do: End of the Chapter 5 Session

- Agree on the final Chapter 5 structure with the supervisor and retain only the project-type subsections that apply.
- Complete the implementation-environment tables using tested versions, minimum specifications, justifications and traceable evidence.
- Document the implemented modules, data or experimental pipeline and integrations, and trace them to the Chapter 4 requirements and design.
- Prepare the test strategy, acceptance criteria and project-specific test cases, including expected results, actual results, evidence, defects and retests.
- Present and interpret the important results using suitable tables, figures, measurements, error analysis and limitations.

- Add the accessible repository link, README, version history, relevant branches or pull requests, environment files and reproducibility evidence.
- Add a brief declaration of AI-assisted tool use, where applicable.
- Verify that the report and repository contain no private data, credentials, fabricated evidence or unsupported claims.
- Review the completed chapter, working system, repository and evaluation evidence with the supervisor before submission.

Chapter 5 should allow an informed reader to understand what was built, reproduce the relevant environment and evaluation, follow the evidence, and judge whether the solution meets its stated objectives and requirements.

Chapter 6: Conclusions, Recommendations And Future Works

6.1 Introduction

Chapter 6 closes the chain from the original problem and objectives to the implemented solution and the evidence reported in Chapter 5. It should provide a concise answer to what the project established, what should be done on the basis of the findings, and what credible work remains.

Do not introduce new experiments, features, results or citations in this chapter. Every conclusion, recommendation and future-work proposal must be traceable to evidence, a limitation or an unresolved question already discussed. Keep the sections distinct:

conclusions state what the study established; recommendations propose justified action; future works identify research or development that remains to be investigated.

6.2 Conclusions

Begin by briefly restating the problem, purpose and completed solution. Then answer the research questions or objectives in the same order used throughout the report. For each objective, state the strongest relevant result and what it means without reproducing the whole results chapter. End by identifying the project's contribution and the conditions or boundaries within which the conclusion holds.

✅ Match the Conclusion to the Evidence Route

- Software and information systems — conclude using implemented capabilities, requirement coverage, user acceptance or usability, performance, reliability, security and data-integrity evidence.
- AI and machine learning — state the selected model or approach, fair baseline comparison, final held-out or external evaluation, important error or subgroup findings, uncertainty and the limits of generalisation. For high-stakes uses, state the role of human oversight.
- IoT and embedded systems — conclude using sensing or control accuracy, calibration, latency, packet delivery, power and recovery evidence under the tested operating conditions.
- Cybersecurity — conclude within the authorised threat model using detection or prevention effectiveness, false alarms, coverage, response time and resource-overhead evidence.

- Blockchain or distributed systems — conclude using protocol or smart-contract correctness, transaction success, cost, latency, throughput, finality and the tested security and network assumptions.
- Hybrid projects — combine the relevant evidence routes and report the weakest important component as honestly as the strongest; a successful interface does not compensate for an unevaluated model, sensor, security control or protocol.

Avoid broad claims such as “the project was successful” unless the defined acceptance criteria support them. Do not claim novelty, superiority, safety, clinical readiness or real-world scalability from a single demonstration or an unfair comparison.

6.3 Recommendations

Recommendations should follow directly from the results and limitations and should be written for a named audience such as users, the adopting organisation, system maintainers, developers, researchers or policy makers. Each recommendation should state the action, the responsible audience, the evidence-based reason and any condition or priority that affects implementation.

💡 Write Recommendations That Can Be Acted On

- Recommend deployment or adoption only within the environment, population and safeguards that were evaluated.
- Recommend infrastructure changes only when measurements show a bottleneck in compute, memory, storage, network, power or response time.

- Recommend data collection, calibration, security hardening, user training, human review or monitoring when a documented error pattern or limitation justifies it.
- Prioritise recommendations and separate essential safeguards from optional improvements.
- Do not disguise an unimplemented feature, an unsupported model change or a personal preference as a finding-based recommendation.

6.4 Future Works

Future works should extend the research rather than repair missing promised work. Select a small number of credible directions that arise from the limitations, error analysis or unanswered questions. For each direction, explain the rationale, the factor that would change, the data or resources required and how the extension should be evaluated. Prioritise the most valuable directions instead of listing every possible technology.

🚀 Examples of Evidence-Based Directions by Project Type

- Software and information systems — longer deployment studies, broader usability or accessibility evaluation, integration with additional systems, stronger recovery testing or evaluation in a new organisational context.
- AI and machine learning — larger or more representative data, external validation, fairness or subgroup analysis, calibration, explainability, robustness, drift monitoring, alternative approaches under a controlled comparison, or deployment evaluation with meaningful human oversight.

- IoT and embedded systems — additional calibration, wider field conditions, power optimisation, communication alternatives, hardware revision, durability testing or longer reliability studies.
- Cybersecurity — broader authorised threat coverage, adversarial or evasion testing, additional environments, longer monitoring, reduced false positives or independent security validation.
- Blockchain or distributed systems — larger network trials, alternative consensus or scaling designs, contract audits, privacy improvements, governance evaluation or sustained cost and performance measurement.

A proposed direction is not automatically valuable because it uses a newer framework, larger model or more expensive device. Link every future work item to the evidence and explain how success would be measured. Confirm scope and feasibility with the supervisor.

To-Do: End of the Chapter 6 Session

- Write a conclusion that answers the problem, research questions and objectives using only evidence already reported.
- State the project contribution and the conditions, population, environment or threat model within which the findings hold.
- Draft audience-specific, actionable recommendations and link each one to a result or limitation.
- Draft a prioritised set of future works, each with a rationale, required evidence and proposed evaluation method.

- Remove new results, citations, unsupported superiority claims and repeated Chapter 5 detail.
- Check the full alignment chain from the problem and objectives to implementation, evaluation, conclusions, recommendations and future works.
- Review Chapter 6 and the final evidence boundaries with the supervisor before submission.

A strong Chapter 6 is brief but decisive: it tells the reader what the evidence supports, what should happen next, and what remains genuinely open for future investigation.

Proposal Submission Guidelines

Proposal Submission will be done online as communicated on **Dr. Khakata's E-Learning Platform.** Please read the instructions carefully to ensure your submission is complete and compliant with all requirements.

Before Submission:

1. **Document Formatting**

- ☐ Revisit the Formatting Guidelines Here and the Required Template Here
- ☐ Ensure your proposal follows the required structure and formatting standards.

2. **Cover Page Requirements**

Confirm that your cover page contains:

- ☐ Project Title
- ☐ Student Name
- ☐ Admission Number
- ☐ ICS Group
- ☐ Supervisor Name
- ☐ Date of Submission (Month & Year)
- ☐ Project Type (**Capstone** or **Internal**)

Example 1:- CAPSTONE PROJECT

A Personalized Conversational Training System for Sales Agents Using Large Language Models

[Capstone Project]

Wangari Kimberly Wanjiru

159745

ICS 4B

Supervisor Name

Kevin Ochieng' Omondi

An Informatics and Computer Science Project Proposal Submitted to the School of Computing and Engineering Sciences in Partial Fulfillment of the Requirements for the Award of a Degree in Bachelor of Science in Informatics and Computer Science

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

June 2026

Example 2:- INTERNAL PROJECT

A Personalized Conversational Training System for Sales Agents Using Large Language Models

[Internal Project]

Wangari Kimberly Wanjiru

159745

ICS 4B

Supervisor Name

Kevin Ochieng' Omondi

An Informatics and Computer Science Project Proposal Submitted to the School of Computing and Engineering Sciences in Partial Fulfillment of the Requirements for the Award of a Degree in Bachelor of Science in Informatics and Computer Science

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

June 2026

3. Declaration and Approval Section

- ☐ Both the student and the supervisor must sign the Declaration and Approval page.

4. Supervisor & Coordinator's Approval

- [ ] Ensure your supervisor has approved your work for submission on the ICS 4 CS II PROJECT - Proposal Approval Register 2026 .
- [ ] Supervisors MUST additionally append their signature on the approved proposal
- [ ] The Coordinator MUST approve your work on the shared Google Sheet before submitting.

5. **Similarity / Plagiarism Report**
   - [ ] Extract and attach the one-page Similarity/Plagiarism Index Report as an Appendix at the end of the proposal (You may just attach as an additional page).
   - [ ] If the report is not available, upload the signed proposal document as instructed.
6. **File Naming Convention**
   - [ ] Before uploading, rename your document using the format:
     - StudentName_AdmissionNumber_SupervisorSurname_Prof.SupervisorSurname.pdf
     - **Example:**
     - JohnDoe_123456_Prof.VOmwenga.pdf
7. **Uploading**
   - [ ] Upload the signed proposal document using the designated submission links provided on the E-Learning platform.
     - Check for your ICS Group Link
   - [ ] Verify that the correct version has been uploaded before the deadline.
8. **Proposal Defence Dates**
   - [ ] **Proposal Defence Period: 13th – 16th July 2026**
     - Students should ensure they are available during this period.
     - Also note that this will be done online (more information will be provided)

9. **Post-Submission Activities**
    - ☐ Git and GitHub classes will commence immediately after proposal submissions. Attendance is encouraged as these sessions contribute to Semester I coursework assessment.

Final Checklist Before Uploading

- ☐ ✅ Approved project template used
- ☐ ✅ Correct formatting applied
- ☐ ✅ Cover page complete
- ☐ ✅ Project type indicated (Capstone/Internal)
- ☐ ✅ Student signature included
- ☐ ✅ Supervisor signature included
- ☐ ✅ Similarity report attached
- ☐ ✅ Supervisor and Coordinators approval obtained
- ☐ ✅ Correct filename used
- ☐ ✅ Final PDF uploaded

Important: Documents that do not meet the minimum formatting, approval, signing, and submission requirements may be returned for correction or may not be considered complete submissions. Please take time to verify every requirement before uploading.

Thank you and all the best with your proposal submissions.

Unit Assessment, Milestones & To Do List

Milestones & To Dos

The table below lists the milestones you should be meeting on a weekly basis during the course of your project. These milestones should appear on your work as well once completed in class.

No.	Tasks Name	Due Date	Status
1	Supervisor Selection and allocation	May 7, 2026	DONE
2	Document Formatting and Template Creation	May 8, 2026	DONE
3	Topic creation, and selection		DONE
4	Abstract		DONE
5	Chapter 1: Introduction		DONE
6	Chapter 2: Literature Review		DONE
7	Chapter 3: Methodology		DONE
8	Proposal Submission	Jun 12, 2026	DONE
9	Proposal Marking and Revision	Jul 31, 2026	DONE
10	Proposal Oral Defense		DONE
10	Chapter 4: System Analysis and Design	Aug 17, 2026	ONGOING
11	Documentation Progress		PENDING

No.	Tasks Name	Due Date	Status
12	Documentation Due		PENDING
13	Project Final Defense	Nov 23, 2026	PENDING

Have a look at CS Project II – Proposal Defense Presentation Guidelines 2026

ML Fundamentals - Deep Dive

From examples and simple numbers to models that work on new data

Designed for CS Project II students moving from an approved proposal into implementation and evaluation. No previous machine learning knowledge is assumed. Basic arithmetic is sufficient; the small calculation is explained step by step.

Session Outline

The central idea: A model learns by adjusting internal numbers using examples. A useful model must then perform well on appropriate examples it has not practised on.

- Session 1 — Building the foundations (suggested 90 minutes): Lessons 1–6. Begin with rules and examples, follow one learning update, and distinguish learning from memorisation.
- Session 2 — Making sound project decisions (suggested 90 minutes): Lessons 7–10, classroom activities, and the CS Project II checklist. Discuss model size, data, synthetic examples, and evaluation.

Learning outcomes: Explain how models learn; distinguish learning types, training, and inference; use validation appropriately; interpret a confusion matrix and QA metrics; assess model size, data quality, synthetic data, and bias; and justify whether a model meets a project's needs.

Teaching note: All library rules, dataset sizes, scores, and numerical experiments below are invented teaching examples unless explicitly attributed to a source. They are not Strathmore University policies or measured research results. The diagrams are original explanatory illustrations. The animal pictures are AI-generated, photo-style illustrations for teaching, not photographs collected as research data.

Our Two Running Examples

Example 1 — Image classification: A system receives one photograph and chooses between two labels: cat and dog. Begin with a single clearly visible animal. Later,

examine poor lighting, unfamiliar animals, unusual backgrounds, and images containing neither class.

Example 2 — Question answering (QA): A system answers questions from a fictional library guide. Our first passage is: “The library closes at 8 p.m. on weekdays. Students may borrow three books.” The question “When does the library close on weekdays?” should produce “8 p.m.”

Important distinction: Classifying a photograph means selecting a category. Answering a question may mean selecting words from a passage or generating a response. These tasks share learning principles but need different outputs, training targets, and evaluation methods.

Lesson 1: From Written Rules to Learning from Examples

1.1 What Does “Learning” Mean Here?

Imagine describing every possible cat to a programmer: ears, fur, eyes, tails, body shape, and every possible camera angle. A rule such as “pointed ears means cat” will fail for some dogs. The difficulty is not writing an if-statement; it is finding rules that handle the variety.

Machine learning uses examples to fit a rule or mathematical function. People still choose the task, data, model, and learning procedure. Training changes parts of the function so its predictions better match a chosen objective. The computer does not need human-like awareness to do this.

Image example: Rather than manually describe every ear shape, provide many photographs with reliable cat/dog labels. A model can learn combinations of visual patterns associated with each label.

QA example: A fixed lookup can answer an exact stored question. A learned system may handle “What time does it shut?” as well as “When does it close?” It must still

receive suitable evidence or have relevant learned information; flexible wording does not guarantee factual correctness.

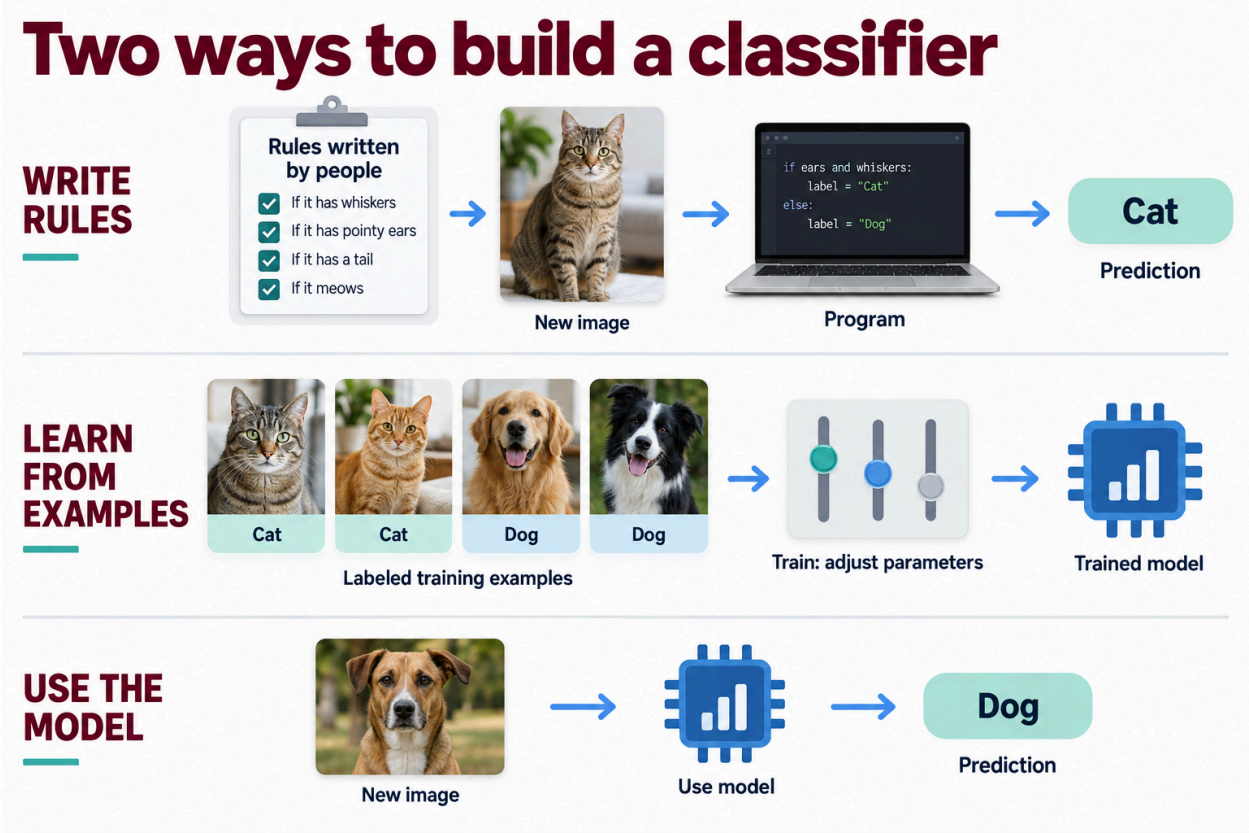


Figure 1. Handwritten rules and learning from examples. Both approaches require human decisions; the learned approach fits parameters from data. The handwritten rules shown are deliberately simplistic.

1.2 Where Do AI, ML, and Deep Learning Fit?

Artificial intelligence (AI) is a broad area concerned with computer systems performing tasks associated with intelligent behaviour. Machine learning (ML) is one approach within AI. Deep learning is a branch of ML using neural networks with multiple processing layers. Not all ML uses deep learning: linear models and decision trees are useful examples.

Generative AI describes systems that produce content, such as text or images. It describes a kind of capability, rather than a separate size category. A cat/dog classifier normally selects a label; a generative image model creates an image.

1.3 Types of Learning: Where Does the Feedback Come From?

A useful question is: what tells the model that it should change? The answer helps distinguish the main learning approaches. “Supervised” does not mean a teacher watches every calculation. It means training examples supply desired outputs (Google, n.d.-a).

Supervised learning — learn from examples with answers

Think of practice questions with a marking guide. Each input has a target: the answer the model should produce. For images, a person checks each photograph and labels it cat or dog. For our library QA system, an example contains a passage, a question, and a verified answer such as “8 p.m.” The model predicts, compares with the target, and adjusts.

Classification predicts a category, such as cat or dog. Regression predicts a numerical quantity, such as an animal’s weight in kilograms. Both can use supervised learning. Generating a written answer is another kind of output; it is not automatically a simple classification problem.

Unsupervised learning — look for structure without supplied answers

Think of sorting a mixed box of photographs without labels. A clustering method groups examples that look similar according to its chosen representation. It might group photographs by animal shape, but it could instead group them by background or colour. People must inspect what the groups mean; cluster 1 is not automatically “cats.”

For the library, clustering unlabelled questions might reveal groups about opening hours, borrowing, and membership. This helps organise questions or identify common needs. It does not, on its own, teach the system the correct answers. “No answer labels” does not mean “no human choices”: people still select the data, representation, and method.

Semi-supervised learning — use a few labelled examples and many unlabelled ones

Imagine 200 checked cat/dog photographs and 5,000 photographs without labels. A method can learn from the checked examples and use information in the others. One approach assigns tentative model-generated labels, called pseudo-labels, to selected unlabelled examples. These labels can be wrong and must not be treated as verified truth.

For QA, a small set of checked passage–question–answer examples can be combined with unlabelled library text or questions using a suitable training method. Merely adding unanswered questions to a folder is not semi-supervised learning. The training procedure must actually use the unlabelled material.

Self-supervised learning — make a practice target from the data itself

Think of covering part of a sentence and trying to recover it. In “The library closes at eight ___,” the original text supplies “p.m.” as the target. A next-token model similarly learns to predict the following token from earlier ones. No person needs to write a separate answer label for every position.

For images, one approach hides patches of a photograph and trains a model to reconstruct them. These practice tasks can help learn useful representations. A model that reconstructs photographs is not automatically a reliable cat/dog classifier, just as predicting words does not guarantee correct library answers. Further task training or suitable prompting and evaluation may be needed.

Semi-supervised and self-supervised are related ideas, but the names answer different questions: semi-supervised describes a mixture of labelled and unlabelled data; self-supervised describes how targets are created. A system can use both across its training stages.

Reinforcement learning — learn actions from rewards

Think of learning a game: an agent acts, observes what happens, and receives a reward. It tries to improve its decisions to obtain greater total reward over time. A reward is a feedback score, not necessarily the correct answer for every step.

A library assistant could learn when to retrieve another passage, ask for clarification, or answer, using carefully designed feedback for supported, useful responses. An image-related example is an agent learning where to move a camera to obtain a clearer view before classification. A fixed cat/dog classifier is normally taught more directly with labelled images.

Reward design matters. Rewarding only fast replies might encourage guessing; rewarding only user approval may favour pleasing but unsupported answers. Human feedback does not always imply reinforcement learning: people can also label supervised examples or provide preferences for other training methods.

How the approaches fit together

These are ways of learning, not competing model sizes. A QA model might use self-supervised pretraining, supervised fine-tuning on checked answers, and later reward-based training. Transfer learning means reusing previous learning for another task; fine-tuning describes further parameter training. Neither replaces the question of where the new feedback comes from.

Check your understanding: Labelled cats and dogs → supervised. Grouping unlabelled library questions → unsupervised. A few verified labels plus many unlabelled images → semi-supervised. Predicting a hidden word using the original sentence → self-supervised. Learning retrieval decisions from rewards → reinforcement learning.

A simple library opening-hours database lookup may still solve the task without ML. Choose the learning approach because it serves the problem, not because its name sounds advanced.

Lesson 2: Inputs, Labels, Features, and Numbers

2.1 Start with One Example

An example is one case used for learning or evaluation. Its input is what the model receives. Its target is the desired output used during training or assessment. A dataset is a collection of examples. A prediction is what the model actually produces.

Image example: Input = a photograph; target label = cat; prediction = dog. The target and prediction are different things. A wrong label can teach the wrong association even when the training software works correctly.

QA example: Input = the library passage plus a question; target = the correct answer or its location in the passage. The target is not normally handed to the model as part of its input. Doing so would give away the answer.

2.2 What Does the Computer Receive?

An image is stored as numbers. A small greyscale image has one brightness value per pixel; a common colour image has red, green, and blue values. A 100 × 100 colour image therefore contains 30,000 channel values before any additional processing. These are input values, not 30,000 learned parameters.

Text is divided into tokens: pieces that may be words, parts of words, or punctuation. A tokenizer assigns token IDs. Those IDs are looked up or transformed into numerical vectors called embeddings. A larger token ID does not mean a more important word. Token boundaries depend on the tokenizer.

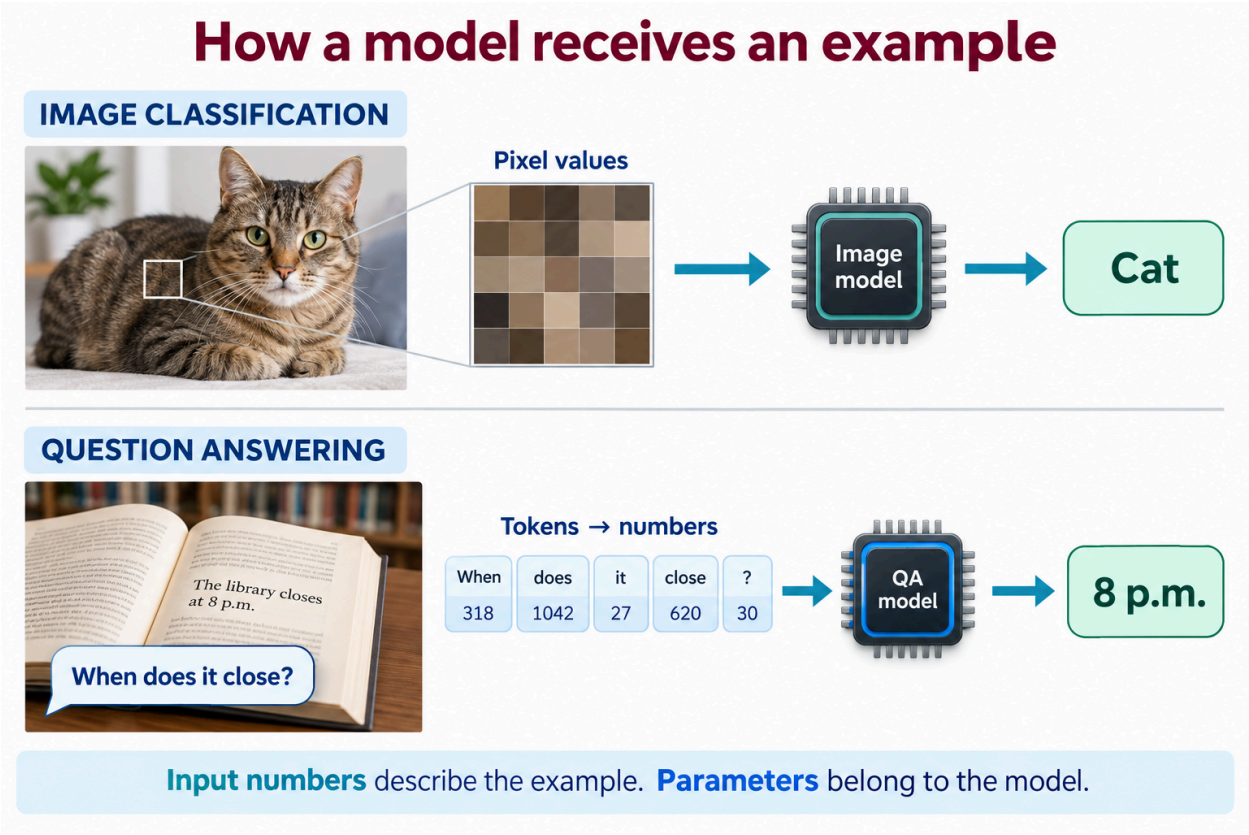


Figure 2. Images and text reach models as numbers. Input values and learned parameters have different roles. Token IDs are illustrative; both the passage and question must be represented for the QA model.

A feature is information used to make a prediction. In a simple system, people might define features such as average brightness. Neural networks can learn intermediate features from pixels or token representations. Features are useful only if they help with the real task.

Important caution: Resizing, normalising, and tokenising must be consistent between training and use. Any preprocessing that learns statistics, such as a mean or vocabulary fitted for an experiment, must be fitted on training data rather than the held-out test set.

Check your understanding: Are 1,000 photographs the same as 1,000 pixels? No. The first counts examples; the second counts elements within image inputs.

Lesson 3: What Is Inside a Model?

3.1 Think of Adjustable Numbers

A useful starting picture is a machine with adjustable dials. Its structure determines how inputs are processed. Its parameters are the internal numbers adjusted during learning. In a neural network, many parameters are weights and biases. A weight controls how strongly an input contributes; a bias can shift the result.

A tiny mathematical model could compute: score = weight × input + bias. A larger model combines many operations. A neural-network unit combines values and usually applies a nonlinear function; layers connect these units. Nonlinearity helps represent relationships that repeated straight-line operations alone cannot express.

Architecture means the arrangement of the model's operations. A training algorithm describes how parameters are fitted. A trained model combines an architecture with its fitted parameter values. A saved checkpoint records a training state that can be loaded again.

3.2 Parameters Are Not Settings Chosen in the Same Way

Parameters are learned during training. Hyperparameters guide the process or define the model: examples include learning rate, batch size, layer count, and training limits. Students choose or tune hyperparameters using training and validation results. They do not use final test performance as a repeated tuning signal.

Image example: A visual model may learn patterns useful for distinguishing ears, fur, and shapes, but it can also learn misleading background patterns. A convolutional neural network processes local image regions using shared filters; other architectures, including vision transformers, are also possible.

QA example: A text model combines information about the question and passage. Transformer attention lets one position use information from other positions. In our

library example, useful relationships include “when,” “closes,” “weekdays,” and “8 p.m.” Attention is a computation, not proof of human understanding.

Check your understanding: If two models have the same architecture, must they behave identically? No. Different data, training choices, or parameter values can produce different behaviour.

Lesson 4: How a Model Learns — One Adjustment at a Time

4.1 The Learning Loop

For the supervised neural-network examples in this lesson, training repeatedly follows five steps: make predictions; compare them with targets; calculate a loss; calculate how parameter changes affect that loss; and update parameters. Other ML methods may learn through different procedures.

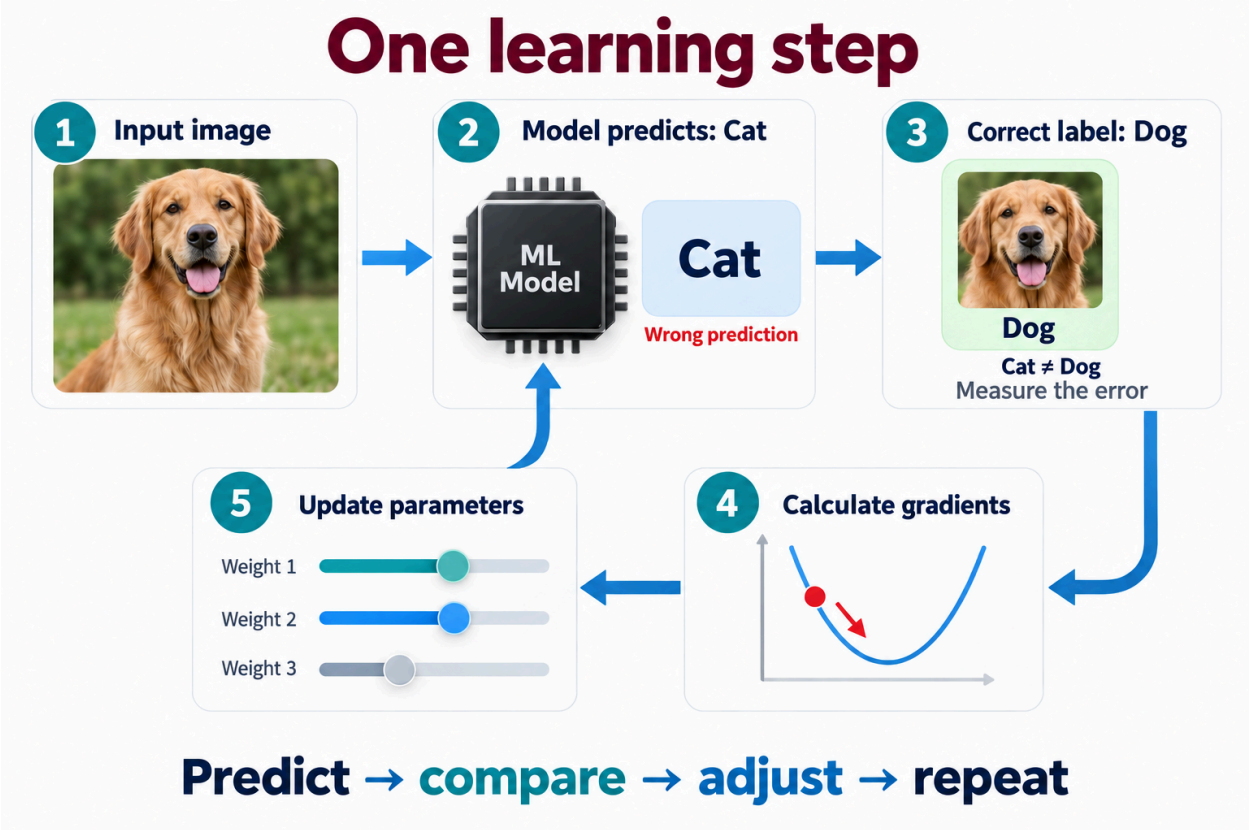


Figure 3. The training loop. The target is used to assess the prediction; the optimizer changes parameters using the calculated gradients.

Loss is a numerical penalty for predictions that differ from the training target. The precise formula depends on the task. A gradient describes how loss changes as a parameter changes. Backpropagation calculates gradients through a neural network. An optimizer, such as gradient descent, uses them to update parameters. The learning rate helps control the size of an update.

4.2 A Tiny Worked Example

This deliberately simplified example demonstrates an update, not a real cat/dog classifier. Let $x = 1$ represent one made-up image feature, let the target $y = 1$ stand for cat, and let the model produce a raw score $s = w \times x$. There is no bias in this example. A raw score is not a probability.

Start with weight $w = 0.2$. The prediction is $s = 0.2 \times 1 = 0.2$. Use squared-error loss: $L = (s - y)^2$. The loss is $(0.2 - 1)^2 = 0.64$.

For this formula, the gradient with respect to w is $2(s - y)x$. Therefore, the gradient is $2 \times (0.2 - 1) \times 1 = -1.6$. Students do not need to derive this formula here; it tells the optimizer which direction changes the loss.

Choose learning rate 0.1. Update rule: new weight = old weight − learning rate × gradient. Thus, the new weight is $0.2 - 0.1 \times (-1.6) = 0.36$.

Predict again: $s = 0.36$. The new loss is $(0.36 - 1)^2 = 0.4096$. The error on this example has decreased. One internal number changed; the model did not receive a new human-written rule.

Important caution: An improvement on one example does not establish a useful classifier. A real system must learn across many examples and be evaluated on new ones. Classification commonly uses a loss such as cross-entropy rather than this teaching example's squared error.

4.3 Apply the Same Idea to Both Tasks

Image example: The model may assign a low probability to the correct class. A classification loss penalises this, and updates can increase the correct class probability. Confidence in a wrong answer can incur a larger penalty than a less confident wrong answer.

QA example: An extractive model can predict the start and end positions of "8 p.m." in the passage. Training penalises incorrect positions. A generative QA model can instead learn to predict the next answer token from the question, passage, and preceding tokens. Predicting plausible words is not itself a guarantee of truth.

Activity — Try the next update: Using $w = 0.36$, $x = 1$, $y = 1$, and learning rate 0.1, calculate the next weight. Suggested answer: gradient = −1.28; new weight = 0.488; new loss = 0.262144.

Check your understanding: What actually learned? The parameter values changed according to the objective. The example does not demonstrate consciousness or perfect understanding.

Lesson 5: Training and Inference Are Different

5.1 Practice Time and Use Time

Training changes parameters. Inference uses a trained model to produce an output. The same model can answer thousands of questions without changing its weights. Saving a chat history or retrieving a document does not, by itself, constitute parameter training.

Image example: During training, labelled photographs provide feedback. During inference, a new photograph enters and a class prediction leaves. The correct label may be unknown until a person checks it.

QA example: At inference time, the system receives the question and relevant passage. Giving it a revised passage that says “6 p.m.” can change its answer without any weight update. The context changed. The model was not necessarily retrained.

5.2 Batch, Step, and Epoch

A batch is a group of examples processed together. A training step commonly refers to one parameter update. An epoch is one pass through the training dataset. With 1,000 examples and batch size 100, one epoch contains 10 updates in a simple setup with one update per batch. Twenty epochs involve 200 updates and 20,000 example presentations, but still only 1,000 distinct examples.

Important caution: Gradient accumulation and other training arrangements can change the relationship between batches and optimizer steps. For large text corpora, training may be described using tokens and steps rather than complete epochs.

Repeated practice can help optimisation, but it does not create new independent evidence. Hours are also an unreliable comparison: the same work can take different times on different hardware.

Check your understanding: If a model uses a document in a prompt, has it necessarily learned that document permanently? No. It may simply be using information available in the current context.

Lesson 6: Learning, Memorising, and Knowing When to Stop

6.1 Three Different Jobs for Data

Training set — practice material: used to fit parameters. Validation set — development check: used to compare settings, choose checkpoints, and decide when to stop. Test set — final assessment: reserved for evaluating the selected approach after development decisions are made.

An illustrative split is 700 training, 150 validation, and 150 test examples out of 1,000. This is not a mandatory ratio. The split must suit the task, available data, and the kind of future use being claimed. Small datasets may need carefully designed cross-validation and an independent final assessment where feasible.

Figure 4. Separate the data before training. Validation informs development; the final test set must not become repeated practice material.

6.1.1 Validation and Evaluation: Related, but Different

Evaluation means measuring how well a model or system performs. Validation is evaluation during development to guide decisions. Final test evaluation estimates performance after those decisions are fixed. The same metric, such as accuracy, can be measured on training, validation, and test data; the data's role is what changes.

Think of practice work, a mock exam, and a final exam. Training fits parameters using practice work. Validation helps choose how to prepare. The test provides a separate final assessment. Repeatedly using the final exam answers to change the model would undermine that assessment.

Image example: Train two candidate cat/dog models on the training set. Compare their validation results, including cat and dog errors, and select one. Fix the model and decision threshold before measuring its performance on the test images.

QA example: Use validation questions to choose a retrieval setting or an instruction that helps the system avoid unsupported answers. Then evaluate the chosen system on held-out questions, including questions whose answers are absent. Trying many prompts against the final test questions is still test-set tuning, even if no weights change.

A metric is a numerical summary of performance. Loss is the quantity used to guide training; the reported task metric may be different. Lower training loss does not automatically mean more correct answers on new data. Lesson 10 works through the task metrics.

Cross-validation is useful when development data is limited. For example, divide development data into five folds, train on four and validate on the fifth, then repeat until every fold has served as validation. Average the results and examine variation. Fit preprocessing separately within each training fold, and keep related animals, source documents, or paraphrases together. Cross-validation does not make leakage harmless or remove the need for an honest final assessment.

6.2 Underfitting, Useful Learning, and Overfitting

Underfitting means the model has not captured enough useful structure. Performance is poor even on training examples. Possible causes include an unsuitable model, weak inputs, difficult optimisation, or insufficient training.

Generalisation means performing usefully on appropriate unseen examples. Overfitting means fitting peculiarities of the training set that do not transfer well. A model can improve on training examples while getting worse on validation examples (Google, n.d.-b).

Image example: Every training cat appears on a sofa and every dog on grass. A model may exploit the background. Test it with cats on grass and dogs on sofas to reveal the shortcut.

QA example: Many training questions have the answer “8 p.m.” A model that repeats this answer can fail when the passage says “6 p.m.” or asks how many books a student may borrow. Test whether the answer follows the supplied evidence.

6.3 Why More Training Is Not Always Better

Consider these invented checkpoint results: epoch 2 — training loss 0.65, validation loss 0.68; epoch 5 — 0.40 and 0.45; epoch 10 — 0.20 and 0.30; epoch 20 — 0.05 and 0.50. Among these checkpoints, epoch 10 has the lowest validation loss even though epoch 20 has the lowest training loss.

5 Lower training loss does not guarantee better use

Invented teaching values. Lower loss is better; the last checkpoint is not always best.

Figure 5. An illustrative learning curve, not a measured experiment. The marked checkpoint is best among the displayed validation results; the final checkpoint is not automatically best.

Early stopping monitors a chosen validation measure and stops when improvement has stalled for a specified patience period. Save the best checkpoint and restore it if that is

the selection rule. A single noisy fluctuation need not trigger stopping. The measure may be validation loss or a task metric chosen in advance (TensorFlow, n.d.).

Important caution: Longer training is not inherently harmful. If validation performance is still improving, further training may help. Conversely, high training and validation errors do not prove that more epochs are the solution. Inspect data, labels, model suitability, learning rate, and the task definition.

Other responses to overfitting include obtaining more varied data, appropriate augmentation, simpler models, and regularisation. Regularisation discourages overly specific solutions; weight decay and dropout are examples. These methods need validation rather than automatic application.

6.4 Data Leakage: Accidentally Giving Away the Answers

Image leakage: Splitting photographs randomly may put near-identical frames of the same animal in both training and test sets. If the claim concerns new animals, group images by animal or capture session before splitting. Keep transformed copies of an image in the same partition as the original.

QA leakage: Paraphrases of the same question-answer pair can cross partitions. If the claim concerns unseen documents, split by document or source before generating questions. A claim about new questions over a fixed library collection may need a different design: state exactly what is unseen.

Use only the training partition to create training augmentations or synthetic examples. Never generate training material from held-out test answers. Repeatedly changing the model after inspecting test failures turns that test set into development data; acknowledge this and obtain a new independent evaluation when needed.

Activity — Choose and diagnose: Which displayed checkpoint would you select using validation loss? Epoch 10. What if all test photographs are duplicates of training

photographs? The reported result would not demonstrate generalisation to independent photographs.

Lesson 7: Small, Large, Base, and Fine-Tuned Models

7.1 Model Size Is About Capacity, Not Just File Size

Parameter count is a common way to describe model size. A model with 10 million parameters has 10 million fitted numbers; one with 1 billion has 1,000 million. More parameters can support more complex patterns but do not guarantee better results for a particular task.

There is no universal boundary separating “small” and “large” across all ML. The terms depend on the model family and context. Architecture, training data, objective, and training quality all matter.

A larger parameter count usually increases storage and computing requirements under comparable conditions. Actual speed and memory also depend on hardware, numerical precision, input length, and architecture. Quantisation stores numbers with fewer bits; it can reduce storage without reducing parameter count.

Image example: A compact pretrained visual model may meet the project’s accuracy and phone-speed requirements. A larger model that gains little accuracy but cannot run on the intended device may be less suitable.

QA example: A smaller model with the correct library passage may answer the scoped question better than a larger model given no reliable source. This is a hypothesis to evaluate, not a promise that smaller models always win.

7.2 “Base” Has More Than One Meaning

Training-stage meaning: A base model often means a pretrained model before further task-specific or instruction-following adaptation. It can be small or large. A base

language model may continue text fluently without reliably following a user's instructions.

Family-size meaning: Some model families use "Base" as a named architecture size alongside "Small" or "Large." Read the model documentation to determine which meaning applies. Do not infer capabilities from the name alone (Google, n.d.-a).

A baseline is different: it is a reference method used for comparison. It could be a majority-class predictor, keyword lookup, or an existing model. A "base model" is not automatically the right experimental baseline.

7.3 Different Ways to Build on Existing Work

- Training from scratch: Start without pretrained task knowledge in the parameters and fit a model using the available training process. This can be expensive for deep models; a small conventional ML model may still be practical.
- Pretraining: Learn reusable patterns from a broad dataset before adapting to a downstream task. In text, self-supervised prediction is common.
- Transfer learning: Reuse knowledge from a model trained for another setting. For image classification, a pretrained feature extractor can be paired with a new classifier.
- Fine-tuning: Continue training some or all parameters using task-relevant data. Freezing layers means keeping those parameters unchanged while other parts learn.
- Instruction tuning: A form of additional training using instructions and desired responses to improve instruction-following behaviour.
- Prompting: Change the instructions or context supplied at inference time. Ordinary prompting does not update model weights.

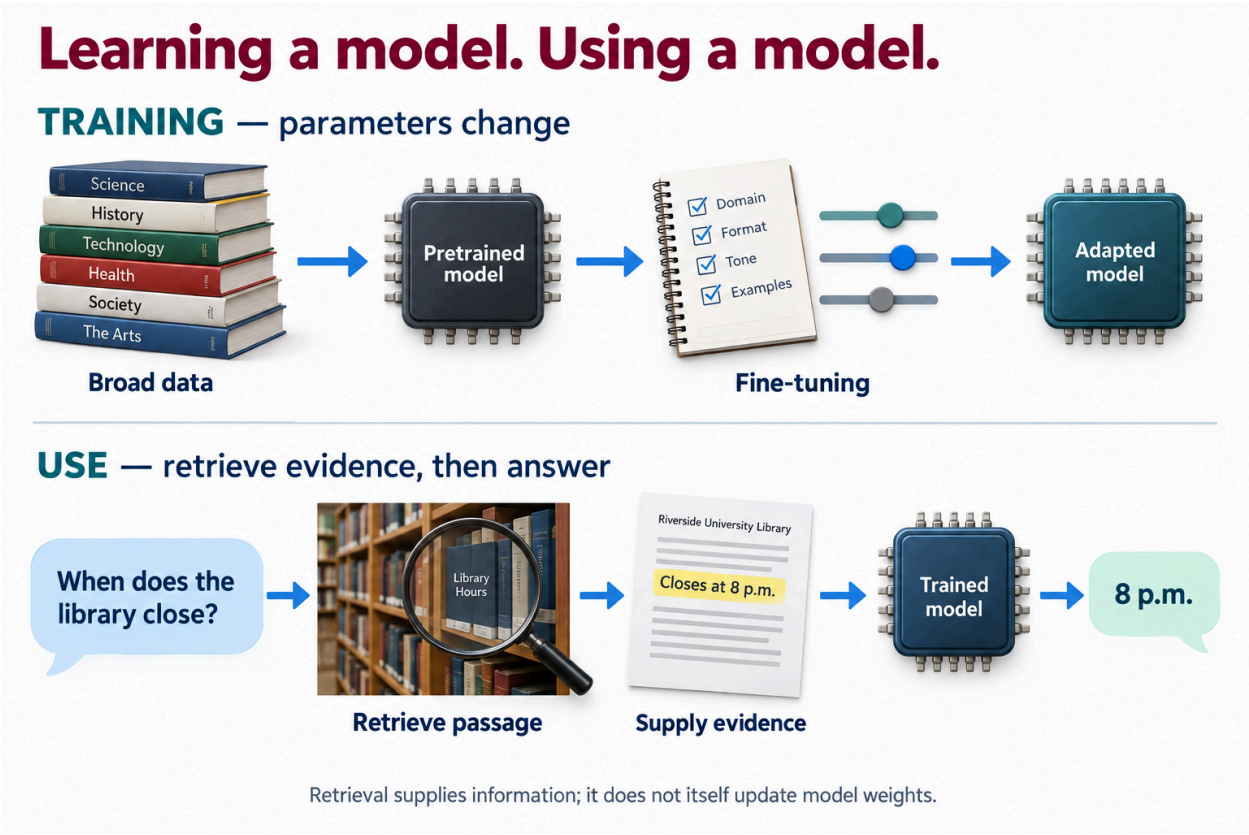


Figure 6. Training changes parameters; inference uses them. Retrieval supplies external evidence at use time and does not itself train the answering model.

7.4 Question Answering and Retrieval

Extractive QA selects an answer span from a passage. Generative QA writes an answer, potentially in different words. Both require suitable evaluation; neither is guaranteed correct merely because a passage is supplied (Hugging Face, n.d.-a).

Retrieval-augmented generation (RAG) first finds relevant material, then provides it to a generative model with the question. For the library example, retrieve the opening-hours paragraph before answering. If retrieval finds an outdated rule, the answer may be wrong despite fluent writing.

A useful diagnostic separates retrieval from answering: Did the correct passage reach the model? If yes, did it use the passage correctly? A retrieved document store and a model's weights are different places where information can reside.

Check your understanding: Can a small model be a base model and later be fine-tuned? Yes. Size and training stage are different descriptions.

Lesson 8: Dataset Size — How Much Is Enough?

8.1 Count Meaningful Variety

More relevant, varied, correctly labelled data can improve learning. More duplicated, irrelevant, or wrongly labelled data can waste resources or teach unwanted patterns. Dataset size should therefore be reported alongside what the examples represent.

Image example: Ten thousand frames from one short cat video do not represent ten thousand independent cats. A smaller collection covering different animals, cameras, backgrounds, and lighting may provide more useful variety. Whether it performs better must be tested.

QA example: Ten thousand paraphrases of one opening-hours question provide limited coverage. A useful collection also covers borrowing limits, exceptions, conditions, ambiguous questions, and questions the guide cannot answer.

For images, report image count, class count, and relevant independent groups such as animals or capture sessions. For text, report documents, question-answer pairs, and tokens where appropriate. File size in megabytes alone does not describe the learning content.

8.2 Quality, Coverage, and Balance

- Correctness: Are labels and answers trustworthy? Check disagreements and ambiguous examples rather than assuming all supplied labels are correct.
- Relevance: Does the data resemble the intended use? Studio animal photographs may not represent blurry phone photographs.
- Coverage: Are difficult cases and important user groups represented? QA data should include supported, unsupported, and ambiguous questions.

- Balance: Do some classes or question types dominate? Preserve a realistic evaluation distribution while also reporting performance for important minority cases.
- Independence: Are duplicates and closely related examples controlled? A large row count can conceal a small amount of independent information.

Important caution: Oversampling a minority class repeats evidence; it does not create new independent observations. Augmentation may introduce useful variation, but it cannot guarantee coverage of missing real-world conditions.

8.3 A Practical Way to Estimate Data Needs

Define the task and baseline first. Train with progressively larger subsets of the training data, keeping validation conditions comparable. Plot validation performance against training-set size. This is a data-size learning curve; it answers a different question from loss plotted against epochs.

If performance continues to improve, more suitable data may help. If it plateaus, investigate labels, coverage, model limitations, or the difficulty of the task. A plateau is evidence about the current setup, not proof that all further data are useless.

No universal number of images or QA pairs guarantees success. Data needs depend on task complexity, variation, model choice, pretraining, required reliability, and evaluation uncertainty. Choose a feasible pilot, inspect errors, and discuss the collection plan with the supervisor.

Check your understanding: Does training for 100 epochs turn 500 examples into 50,000 independent examples? No. It increases exposure to the existing examples.

Lesson 9: Synthetic Data — Useful Examples or Repeated Mistakes?

9.1 What Counts as Synthetic Data?

Synthetic data are artificially created examples rather than direct observations of the intended real-world cases. They may come from a simulator, a rules-based generator, or another ML model. Synthetic does not automatically mean incorrect, independent, realistic, or private.

Data augmentation usually transforms an existing example: slightly crop a photograph, adjust brightness, or carefully paraphrase a question. A simulator can render a new scene with controlled lighting. A generative model can create an image or draft a question-answer pair. These methods are related, but their sources and risks differ.

Image example: A modest brightness adjustment can expose a classifier to useful variation. A crop that removes the animal makes the original label inappropriate. A generated "dog" image might contain unrealistic anatomy or a telltale texture that becomes a shortcut.

QA example: From "Students may borrow three books," generate "How many books can a student borrow?" and verify the answer "three." A generated answer of "five" is a wrong training target. A paraphrase that removes "on weekdays" from an opening-hours question may also change its meaning.

9.2 When Can It Help?

Synthetic examples can target a known gap, vary wording, simulate conditions that are expensive to capture, or provide practice for a specialised task. The benefit depends on realism, correctness, coverage, and the training mixture. Generating a large number of similar examples can leave the original gap unresolved.

Research example: Gunasekar et al. (2023) trained a coding language model using selected web material together with synthetic educational material. This demonstrates a useful application of synthetic data; it does not establish that every generated dataset will improve every task.

9.3 What Can Go Wrong?

- Inherited errors: A generating model can supply wrong labels, unsupported answers, stereotypes, or omissions. The learner may reproduce these patterns.
- Limited diversity: Thousands of generated variations may repeat a narrow set of situations or writing styles.
- Mismatch with reality: Artificial images or questions may be easier than real inputs, producing misleading development scores.
- Contamination: Using test questions, test answers, or closely related material to generate training examples undermines independent evaluation.
- Privacy assumptions: Generated data may reproduce sensitive details from its source. “Synthetic” is not proof of anonymisation or permission to use the material.

Model collapse describes degradation that can occur across generations when models learn from generated material and lose aspects of the original data distribution, including uncommon cases. Shumailov et al. (2024) studied recursive training and this loss of information. This is distinct from ordinary overfitting within one training run; it is not an inevitable outcome of every use of synthetic data.

9.4 A Practical Synthetic-Data Experiment

Begin with an independent real validation set and real test set. Identify a concrete weakness in the training data. Generate or transform examples only from permitted training sources. Check labels, factual support, diversity, duplicates, and unwanted artefacts. Record the generator, prompts or rules, filtering, and mixture proportions.

Compare a real-data baseline with one or more real-plus-synthetic mixtures under documented training budgets. Use validation results to choose a mixture. Evaluate the selected approach on the untouched real test set and inspect difficult cases. Where feasible, compare more real data and ordinary augmentation as alternatives.

Important caution: There is no universally correct real-to-synthetic ratio. If a larger mixture requires more training work, report that difference; otherwise, an apparent data benefit may partly reflect extra computation. Synthetic-only test results cannot establish performance on the real target population.

Activity — Audit a generated pair: Passage: “Three books may be borrowed.” Generated question: “How many books may be borrowed for two weeks?” Answer: “Three.” What is wrong? The question introduces a loan period that the passage never states. Repair the question or obtain supporting evidence.

Lesson 10: What Makes a Model Good or Bad?

10.1 Good for Which Task, Users, and Conditions?

A good model meets a defined need with credible evidence. It performs usefully on independent examples, handles important cases, and fits practical constraints. A poor model may memorise, exploit shortcuts, fail on minority cases, invent answers, or be too slow or expensive for its purpose.

A model is not good merely because it is large, new, trained for many hours, or successful on a few demonstrations. Define acceptance criteria with the supervisor before selecting the final model. There is no single score that makes all models good.

10.2 Image Classification: Read the Confusion Matrix

Accuracy is the fraction of predictions that are correct. In an invented test set of 95 cats and 5 dogs, always predicting cat gives 95% accuracy but finds no dogs. A high overall score can therefore hide complete failure on an important class.

A confusion matrix is a count of actual labels against predicted labels. It shows which classes the model confuses. In the separate teaching example below, there are 100 images: 50 cats and 50 dogs. Each photograph contains one animal and receives one prediction.

Choose cat as the positive class and dog as the negative class. “Positive” is simply the class we are counting as the event of interest; it does not mean good, correct, or preferable. Here, “negative” means dog only because this example has exactly two classes.

Read the rows as the actual animal and the columns as the model’s prediction.

Actual ↓ / Predicted →	**Cat (positive)**	**Dog (negative)**
Cat: 50 actual	40 True positives	10 False negatives
Dog: 50 actual	5 False positives	45 True negatives

True positive (TP): An actual cat is predicted as cat. The model found the chosen positive class correctly. There are 40.

False negative (FN): An actual cat is predicted as dog. The model missed a cat. There are 10.

False positive (FP): An actual dog is predicted as cat. The model raised a false “cat” result. There are 5.

True negative (TN): An actual dog is predicted as dog. The model correctly rejected “cat.” There are 45.

“True” or “false” says whether the prediction was correct. “Positive” or “negative” says what the model predicted. Read both words together. The correct cells lie on the diagonal; the other cells are errors. Software may reverse the axes, so always read the labels.

10.2.1 Turn the Counts into Useful Metrics

Accuracy — How often was the model correct overall? (TP + TN) ÷ all examples = (40 + 45) ÷ 100 = 85%.

Precision for cat — When it said “cat,” how often was it right? TP ÷ (TP + FP) = 40 ÷ 45 = 88.9%. The denominator is predicted cats.

Recall for cat, also called sensitivity or true positive rate — Of all actual cats, how many did it find? TP ÷ (TP + FN) = 40 ÷ 50 = 80%. The denominator is actual cats.

Specificity, or true negative rate — Of all actual dogs, how many did it correctly call dog? TN ÷ (TN + FP) = 45 ÷ 50 = 90%.

False positive rate — Of all actual dogs, how many were wrongly called cat? FP ÷ (FP + TN) = 5 ÷ 50 = 10%. This is not the same as the fraction of predicted cats that are wrong.

False negative rate — Of all actual cats, how many were missed? FN ÷ (FN + TP) = 10 ÷ 50 = 20%. It is 1 − recall; false positive rate is 1 − specificity.

F1 score — A single summary balancing precision and recall through their harmonic mean. F1 = 2TP ÷ (2TP + FP + FN) = 80 ÷ 95 = 84.2%. It is not their ordinary average and does not use true negatives.

Support — The number of actual examples of a class: 50 cats and 50 dogs here. Always report counts as well as percentages. Finding 4 of 5 cats and 400 of 500 cats both gives 80% recall, but the amount of evidence is very different.

Balanced accuracy — The average recall across classes. Here it is (80% + 90%) ÷ 2 = 85%. In the earlier 95-cat/5-dog example, always predicting cat gives balanced accuracy of (100% + 0%) ÷ 2 = 50%, exposing the failure hidden by 95% ordinary accuracy.

These definitions follow standard classification evaluation practice (scikit-learn developers, n.d.). If a denominator is zero, the metric may be undefined; explain the reporting convention rather than quietly interpreting it as reliable performance.

10.2.2 Which Mistake Matters, and Where Is the Cut-off?

Suppose a system sends predicted cat images to a cat-only catalogue. False positives add dogs to the catalogue, so precision matters. If the goal is to find as many cats as possible for later human review, missed cats are costly, so recall matters. Neither goal removes the need to report both types of error.

A threshold is the cut-off used to turn a score into a label. For example, predict cat when its score is at least 0.50. Lowering the cut-off generally produces more cat predictions: recall cannot decrease on a fixed scored dataset, but false positives can increase. Raising it generally reduces cat predictions and can miss more cats. Scores need not be well-calibrated probabilities.

Choose the threshold on validation data according to the task's error costs, then fix it for the final test. There is no universally best 0.50 threshold or universally best F1 score.

For more than two classes, the matrix grows: three classes produce a 3 × 3 matrix. Compute each class's precision, recall, and F1 by treating that class as positive and the others as negative. Macro-F1 averages class F1 scores equally; weighted-F1 weights them by support. A single average can still hide a weak class.

Inspect results by lighting, background, camera, and other relevant conditions. Also test inputs outside the intended classes: a forced cat/dog prediction on a car is not

meaningful animal identification. Consider an appropriate rejection or human-review process.

10.3 Question Answering: Correct, Supported, and Useful

QA needs its own evaluation because an answer is often text rather than one of two labels. A cat/dog confusion matrix cannot by itself score every possible sentence.

Exact match (EM) checks whether a prediction matches an accepted reference answer after stated text-normalisation rules. If 16 of 20 held-out answers match, EM is 80%. Specify how case, punctuation, and alternative correct answers are handled. A valid paraphrase may still fail exact match.

Token-overlap F1 compares the words or tokens in a predicted answer with those in a reference. This is different from cat-classification F1: it counts shared answer tokens, not cats and dogs. Standard QA evaluators define their own normalisation and aggregation rules (Hugging Face, n.d.-b).

Worked example: Reference = “three books”; prediction = “books.” With simple whitespace tokenisation, one of the one predicted tokens is correct, giving precision 1/1 = 100%. One of the two reference tokens was recovered, giving recall 1/2 = 50%. F1 = 2 × 1 × 0.5 ÷ (1 + 0.5) = 66.7%, while exact match is 0. The overlap score gives partial credit even though the answer misses the requested number.

Word overlap is not factual understanding. “The library does not close at 8 p.m.” overlaps strongly with “The library closes at 8 p.m.” but reverses the meaning. Evaluate generated answers with a clear human-review rubric as well.

For each question, record whether the answer is correct, supported by the passage, and complete without invented details. Separately check whether the system handles absent answers appropriately. For example, the weekday passage gives no Sunday hours: “The passage does not specify Sunday hours” is appropriate; inventing a Sunday closing time is not.

Report each rubric dimension separately. In an invented review of 20 answered questions, 17 factually correct answers gives correctness of 85%, while 15 fully supported answers gives evidence support of 75%. A statement can happen to be correct without being justified by the supplied passage. Define the scoring rules before reviewing and double-check disagreements.

10.3.1 A Confusion Matrix for the Decision to Answer

A useful separate binary task is deciding whether the supplied passage contains enough information to answer. Let positive mean answerable; the predicted positive means the system chooses to answer.

True positive: The answer is present and the system attempts an answer. False negative: The answer is present but the system abstains. False positive: The answer is absent but the system attempts an answer. True negative: The answer is absent and the system abstains.

For example, among 20 answerable and 10 unanswerable questions, suppose the system answers 18 of the first group and wrongly attempts 3 of the second. TP = 18, FN = 2, FP = 3, TN = 7. Answerability precision is 18/21 = 85.7%; recall is 18/20 = 90%; correct abstention on unanswerable questions is 7/10 = 70%.

This matrix evaluates the decision to answer, not answer correctness. An attempted answer to an answerable question can still be wrong. Evaluate the actual answers as well. Reporting only accuracy among answered questions can reward a system that avoids almost every question; also report coverage, the fraction it answers: 21/30 = 70% here.

For RAG, first check retrieval: for example, on 20 questions with known supporting passages, did a sufficient passage appear among the top three retrieved results? If it did for 16, the retrieval hit rate at 3 is 80%. State this definition explicitly. Then assess

the answers. This separates “the evidence was not found” from “the evidence was found but used badly.”

10.4 Confidence, Cost, and Reliability

A confidence score is not proof of correctness. Calibration concerns whether stated probabilities align with observed frequencies of correctness over comparable cases. A chatbot’s statement “I am 95% certain” is not automatically a calibrated probability.

Measure response time on the intended hardware, memory requirements, and operational cost where relevant. Compare models on the same evaluation examples and conditions. A small difference on a tiny test set may reflect sampling variation rather than a dependable improvement.

Run repeated training trials where feasible and report variability, not just the best run. Keep the dataset split, model version, settings, code version, and evaluation procedure traceable. After deployment, monitor for changed inputs or rules; useful performance can decline when conditions change.

Check your understanding: Can a model with lower overall accuracy still be preferable? Yes, if it better meets the agreed task requirements, such as detecting an important minority class or meeting a strict response-time constraint. Explain the trade-off with evidence.

10.5 Bias: One Word with Several Meanings

A bias is a systematic tendency, rather than a one-off mistake. In ML, the word has several meanings. Always say which one is intended.

A model’s bias parameter is an adjustable offset. In score = weight × input + bias, a bias of 0.3 shifts the score by 0.3. This is a mathematical parameter, not evidence that the model treats people unfairly.

Inductive bias means the assumptions that help a learning method generalise. For example, an image model may be designed to reuse the same pattern detector across different parts of a photograph. Assumptions can be useful, but can fail when they do not fit the task.

Statistical bias refers to systematic error arising from simplifying assumptions. A model that is too restrictive may repeatedly miss a real pattern and underfit. Variance describes how much the fitted model changes with the training sample. These ideas connect to underfitting and overfitting, but they are not the same as social unfairness.

10.5.1 Bias in Data and in Real Use

Selection or coverage bias: The data leaves out important situations. If most training animals are photographed in bright light, the classifier may perform poorly on dark images. If library QA is trained only on formal English questions, it may struggle with everyday wording, spelling variation, or other intended languages.

Label or measurement bias: The recorded targets or measurements systematically misrepresent the task. A labeller who repeatedly calls small dogs “cats” teaches a wrong association. Outdated library answers teach yesterday’s rules even when the training code is correct.

Shortcut learning: The model relies on an easy but unreliable clue. Cats always on sofas and dogs always on grass can teach “sofa means cat.” A QA system may assume every “when” question is answered by “8 p.m.” without using the actual passage.

Social or harmful bias concerns patterns that produce unjust disadvantages or reinforce stereotypes about people. In a library service, systematic failure on questions from an intended language group could create unequal access. A performance gap should prompt investigation of data, design, and context; an overall score cannot establish fairness (Google, 2025).

Class imbalance is unequal class counts, such as 95 cats and 5 dogs. It can encourage majority-class predictions, but is not automatically proof of harmful bias. A balanced cat/dog dataset can still omit dark images, and a naturally imbalanced dataset may reflect real use. Examine coverage and error costs, not only counts.

10.5.2 How Students Can Investigate Bias

Start by identifying the users and conditions the project claims to serve. Check training coverage, label quality, and errors for those conditions. Compare class and subgroup metrics with their sample counts. For example, 90/100 correct bright images and 12/20 correct dark images gives 90% versus 60% accuracy; the smaller dark-image sample also carries more uncertainty.

For QA, compare supported-answer rates for formal and everyday phrasing of equivalent questions, while keeping related paraphrases in the same data split. Review whether failures come from retrieval, understanding the question, or generating the answer.

Respond to a diagnosed cause: collect missing examples, correct labels, improve retrieval, or revise the model and threshold. Re-evaluate on independent data. Balancing a dataset or adding synthetic examples does not automatically remove bias. Do not collect sensitive personal attributes without an appropriate justification and safeguards.

Check your understanding: Is every error bias? No. A single wrong cat prediction may be an isolated error. A repeated pattern of failure under a particular condition suggests a systematic problem worth investigating.

Classroom Discussion and Project Application

Activity: Choose the Better Model

Image scenario (invented): Model A has 99% training accuracy, 74% validation accuracy, and a 20 MB file. Model B has 92% training accuracy, 88% validation accuracy, and a 7 MB file. Both were evaluated on the same independent validation set. Which deserves further investigation?

Suggested response: B has better evidence of generalisation in this comparison and lower storage needs. A's larger training–validation gap warrants investigation. Selection still requires class-level errors, evaluation uncertainty, target-device measurements, and final independent testing.

QA scenario (invented): A large model answers 18 of 20 familiar questions correctly but invents answers for 8 of 10 unanswerable questions. A smaller model with retrieved evidence answers 17 of the 20 correctly and appropriately abstains on 9 of the 10 unanswerable questions. Which is better for a library help service?

Suggested response: The smaller system may better meet a requirement to avoid unsupported answers, but the sample is too small for a broad claim. Expand evaluation and inspect retrieval, missed answers, latency, and costs. This compares complete systems, so the difference cannot be attributed to model size alone.

Activity: Read the Errors, Then Choose an Action

Using the cat-positive matrix, explain why a dog labelled cat is a false positive. Then switch the positive class to dog. Suggested answer: That same error becomes a false negative for dog; the prediction has not changed, only the class being counted.

A QA system answers "8 p.m." to every question. What should the evaluation include? Suggested answer: Varied answerable questions, unanswerable questions, evidence support, and answer correctness. A high score on repeated closing-time questions would be weak evidence.

CS Project II: Evidence Students Should Prepare

- Problem and objective: State who needs the system and what measurable improvement is expected. Explain why ML is appropriate and identify a simpler baseline.
- Data: Record provenance, permissions, collection method, labels, counts, independent groups, class balance, missing cases, and any synthetic-data process.
- Split design: Explain what is held out, why it represents the intended claim, and how duplicates, related examples, preprocessing, and generated examples are controlled.
- System analysis and design: Show the input-to-output pipeline, model role, data preparation, external services or retrieval, human decisions, and handling of failure cases.
- Implementation: Record the architecture or pretrained model, version, parameter count where relevant, frozen/trainable parts, training settings, environment, and version-controlled code.
- Experiments: Report the baseline, controlled comparisons, training and validation curves, checkpoint-selection rule, repeated-run variability where feasible, and computing budget.
- Evaluation: Present task-appropriate metrics, test-set size and composition, representative errors, subgroup or difficult-case results, and practical performance.
- Limitations and responsible use: Explain unsupported inputs, uncertainty, privacy, bias, and needed human oversight. Safety-critical applications require a much stronger validation and oversight plan than these classroom examples.
- Supervisor review: Validate scope, dataset suitability, method, evaluation criteria, and readiness with the supervisor. Update the report alongside implementation.

Traceability example — Image project: Objective to classify new animal photographs → classifier feature → test on held-out animals → class-level metrics and error examples. QA project: Objective to answer questions from a guide → retrieval and answering features → held-out questions including unsupported cases → correctness, evidence support, and abstention results.

Exit Questions

- What changes during neural-network training? The trainable parameter values.
- Why can training loss decrease while validation loss increases? The model may be fitting patterns specific to the training examples.
- Does “base” always mean small? No; it may describe training stage or a named architecture size.
- Does a million generated examples guarantee improvement? No; correctness, diversity, relevance, contamination, and real-data evaluation matter.
- What makes a model good? Evidence that it meets clearly defined requirements on appropriate independent data and within practical constraints.

Quick Reference Glossary

Example: One input case, with a target where applicable. Dataset: A collection of examples. Label/target: The desired output. Feature: Information used for prediction. Prediction: The model’s output.

Parameter: An internal number fitted through training. Weight: A parameter controlling a contribution. Bias: An adjustable offset in a model; this meaning differs from social or dataset bias. Hyperparameter: A setting guiding model structure or training.

Loss: The numerical training penalty. Gradient: How loss changes with a parameter. Backpropagation: A method for computing gradients through a neural network. Optimizer: The rule that uses training information to update parameters. Learning rate: A setting influencing update size.

Batch: A group of examples processed together. Step: Usually one optimizer update. Epoch: One pass through a training dataset. Checkpoint: A saved model/training state. Inference: Producing outputs with a trained model.

Generalisation: Useful performance on appropriate unseen cases. Underfitting: Failure to capture enough useful structure. Overfitting: Fitting training-specific detail that does not transfer. Leakage: Information entering development that should have remained unavailable for independent assessment.

Token: A unit of text determined by a tokenizer. Embedding: A numerical vector representation. Pretraining: Broad initial learning for later reuse. Fine-tuning: Additional parameter training for adaptation. RAG: Retrieval of relevant material to support generated answers.

Synthetic data: Artificially created examples. Augmentation: Transformations of existing examples intended to preserve useful meaning. Baseline: A reference method for comparison. Calibration: Agreement between predicted probabilities and observed correctness frequencies.

Evaluation: Measurement of performance. Validation: Evaluation used to guide development. Confusion matrix: Counts of actual versus predicted classes. Precision: Correct positives among predicted positives. Recall: Positives found among actual positives. Specificity: Negatives correctly rejected. F1: Harmonic mean of precision and recall.

References and Further Reading

These notes use original classroom examples to explain established concepts. The sources below provide definitions, practical guidance, and research evidence. Research findings are limited to their studied settings; they are not guarantees for a student's project.

Google. (n.d.-a). *Machine learning glossary*. Google for Developers.

Google. (n.d.-b). *Overfitting*. Google for Developers.

Google. (2025). Fairness: Types of bias. Google for Developers.

Gunasekar, S., Zhang, Y., Aneja, J., Mendes, C. C. T., Del Giorno, A., Gopi, S., Javaheripi, M., Kauffmann, P., de Rosa, G., Saarikivi, O., Salim, A., Shah, S., Behl, H. S., Wang, X., Bubeck, S., Eldan, R., Kalai, A. T., Lee, Y. T., & Li, Y. (2023). *Textbooks are all you need* [Preprint]. arXiv.

Hugging Face. (n.d.-a). *Question answering*. Transformers documentation.

Hugging Face. (n.d.-b). Metric card for SQuAD. Evaluate documentation.

scikit-learn developers. (n.d.). Metrics and scoring: Quantifying the quality of predictions. scikit-learn documentation.

Shumailov, I., Shumaylov, Z., Zhao, Y., Gal, Y., Papernot, N., & Anderson, R. (2024). AI models collapse when trained on recursively generated data. *Nature, 631*, 755–759.

TensorFlow. (n.d.). *Overfit and underfit*.

Build Phase – Development Sprints

Dear Students,

The development phase will proceed through structured sprints and regular progress checks, alongside continued work on Chapter 5 and the earlier chapters.

A sprint is a defined period of focused project work with **clear tasks, deliverables and a progress review**. Each sprint should produce a demonstrable improvement in your project and evidence of what has been completed, tested and documented. These sprints apply to **all projects, regardless of project type**; the specific implementation and evaluation evidence should be appropriate to your approved objectives and methodology.

At the beginning of each sprint, agree on realistic priorities with your supervisor. At the end, be prepared to demonstrate your work, explain your decisions, discuss challenges and identify the next tasks. **Maintain your documentation alongside development** throughout this phase.

Sprint I: Data Preparation and Development Environment Setup

The first sprint establishes a reliable foundation for implementation. This applies to **all projects, regardless of project type**. By the end of this sprint, I expect to see:

- A **properly configured development environment**, with the required tools, dependencies and configuration in place.
- A **clean and professionally organised codebase**, with a clear folder structure and instructions for setting up and running the project.

- All project code maintained using **Git version control**, with meaningful commits that show your development progress. Keep passwords, API keys and other sensitive information out of the repository.
- Relevant data, databases, APIs, files, hardware or other **resources prepared and checked**, as applicable. For data-driven projects, document the data source, preparation steps and any access or ethical requirements.
- **Appropriate use of automation** for repeatable setup, data preparation, testing or development management, where it supports your workflow.
- **Corrections from earlier chapters completed**, with proposal-related content revised into project documentation and aligned with the system you are actually developing. Chapter 4 should reflect the analysis and design guiding your implementation.

Use student access before paying for development tools. Apply through GitHub Education using your academic email and the required enrolment evidence, then redeem the relevant Student Developer Pack offers in your own accounts. Keep the verification, expiry dates and account recovery details available.

Put the benefits to work. Use GitHub Pro for private project collaboration, the included Codespaces allowance for a repeatable development environment, and eligible JetBrains student tools where useful. GitHub Copilot Student can support coding and explanations within its current allowances; review and test its output. Monitor usage and stop unused development environments.

Be prepared to demonstrate that the project runs in your development environment, show the repository and prepared resources, and explain how this foundation supports the next stage of implementation.

Sprint II: Core Functionality, Integration and Initial Testing

Due: next week, at the sprint progress review.

In this sprint, you will move from preparation to a working implementation of the core project functionality. Build on the foundation established in Sprint I and prioritise the features or technical components that directly address your approved objectives. By the end of this sprint, I expect to see:

- A clear set of sprint **tasks linked to your objectives**, showing what has been completed, what remains in progress and any issues affecting delivery. Validate the scope and priorities with your supervisor.
- A **working core workflow** that can be demonstrated from input through processing to output. The implementation should perform a meaningful project task; screens, mock-ups or isolated components alone do not demonstrate completion of the core workflow.
- The **main components needed for that workflow integrated**, as applicable: the interface, application logic, database, APIs, model, devices or other project components. Demonstrate that data moves correctly between them and that the resulting output is usable.
- **Initial testing** of the implemented functionality, covering expected behaviour, invalid inputs and relevant failure conditions. Keep a record of the test performed, expected result, actual result and any corrective action taken.
- **Initial evaluation evidence** appropriate to your project. For AI/ML projects, demonstrate a baseline model or pipeline using a justified validation approach and relevant metrics, keeping evaluation data separate from training. For information systems and software projects, demonstrate core user tasks, data validation and access controls where applicable. For IoT projects, demonstrate device communication and record relevant reliability or latency observations. For cybersecurity or blockchain projects, demonstrate the core mechanism in a

controlled environment and record relevant security, correctness or performance evidence.

- **Updated Chapter 5 documentation** explaining what has been implemented, the tools and technologies used, integration decisions, and initial testing or evaluation results. Include relevant screenshots, outputs or test records, and update Chapter 4 diagrams where justified design changes have occurred.
- An **up-to-date Git repository** with meaningful commits, current setup and run instructions, and a reproducible version of the system to be demonstrated. Document known limitations and outstanding defects honestly.

Use the student tools to verify progress. Run repeatable tests through GitHub Actions within the account's included allowance, and retain the results alongside the relevant commit. Use an AI coding assistant to help explain failures or propose tests, then check the assertions and behaviour yourself. A successful automated check does not replace the working demonstration or evaluation evidence.

At the progress review next week, be prepared to demonstrate the **working core workflow**, show your test and evaluation evidence, and explain how the implementation addresses your objectives. Present the outstanding tasks and a realistic plan for completing them in the next sprint.

Sprint II is an intermediate implementation milestone. You are expected to have a working core and initial evidence of its correctness; further functionality, refinement and comprehensive evaluation will continue. Sprint III will focus on approaching completion, including **hosting and serving the project** through an appropriate online platform, such as Cloudflare or AWS, where applicable. During Sprint II, identify deployment requirements and dependencies so that you are ready for that stage.

AI-Assisted Research Workflow

The diagram below illustrates the code, training and evidence cycle for AI/ML projects. Other projects should adapt the same principles of version control, repeatable testing and verified AI assistance to their own implementation.

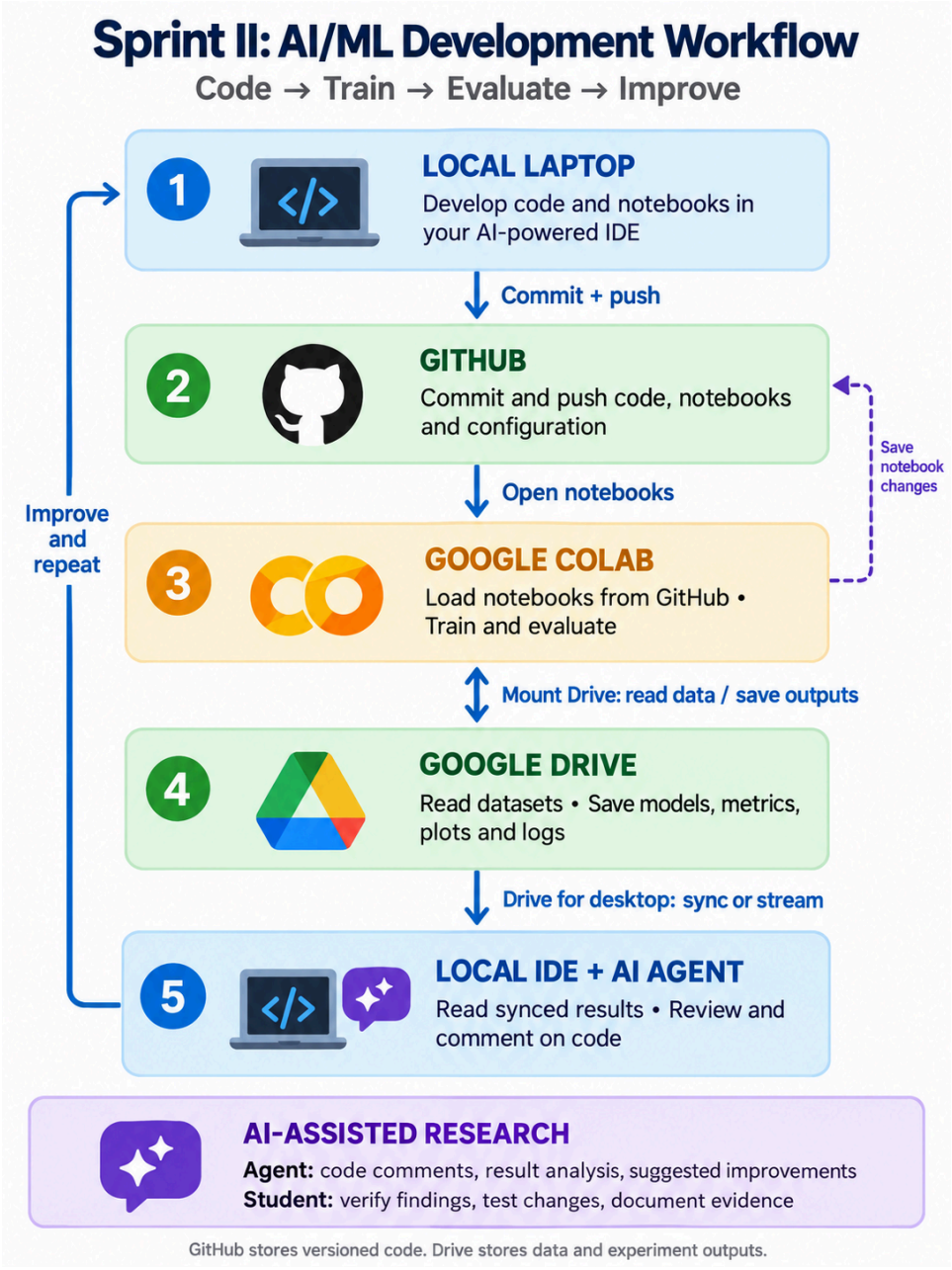


Fig 1: Sprint II — AI/ML development workflow.

How to Work Through the Cycle

1. Define the question and establish a baseline. Begin with a specific question linked to an approved objective: what should improve, and how will improvement be measured? Implement a simple baseline and agree on suitable metrics, data splits and constraints with your supervisor before comparing alternatives.

2. Develop locally and version the work. Keep application code, reusable training modules, notebooks and configuration in your laptop's codebase. Ask the IDE agent to explain unfamiliar code and add useful comments about assumptions, design decisions and potential failure points. Review these comments for accuracy. Commit and push the tested changes to GitHub; keep credentials and sensitive data out of the repository.

3. Run a reproducible experiment in Colab. Open the notebook from GitHub and load any supporting modules from the intended code version. Mount Google Drive in the Colab runtime and read the prepared datasets. Record the Git commit, dataset version, dependencies, parameters, random seeds and run identifier. **Fit preprocessing on training data only**; use validation data for development decisions and reserve the test set for final evaluation.

4. Preserve the evidence in Drive. Save model checkpoints, preprocessing artefacts, configuration, metrics, plots, predictions and logs in a clearly named folder for each run. Include enough information to connect every output to its code and data versions. Save notebook edits back to GitHub so that the recorded code matches the experiment. Files left only in the temporary Colab runtime should not be treated as a permanent record.

5. Bring results back to the local IDE. Use Google Drive for desktop to make the experiment folders available on your laptop, and confirm that synchronisation is complete before analysis. This local connection is separate from mounting Drive inside

Colab. Give the AI agent the relevant code and readable outputs, such as metric files, logs and plots; access to a model file alone does not explain its performance.

6. Use the agent to investigate, then verify. Ask it to compare the baseline and current run, identify error patterns, review code for leakage or implementation faults, and suggest a testable explanation. Require it to point to the actual files and results supporting each observation. **Treat its explanations as hypotheses**: check them against the code, data, experiment evidence and credible literature. Never turn an unverified AI statement or invented citation into a research finding.

7. Test an improvement and document the decision. Make a controlled change, rerun the experiment under comparable conditions and record whether the evidence supports improvement. Preserve unsuccessful runs and explain limitations rather than reporting only favourable results. Review important decisions with your supervisor, then update Chapter 5 with the methods, verified results and your own interpretation. Record material AI assistance and follow applicable institutional disclosure requirements.

Example agent prompt

"Review this notebook and the baseline and current-run results. Identify supported differences, possible data leakage and failure patterns. Refer to the exact code and output files, distinguish observations from hypotheses, and propose one controlled follow-up experiment. Do not invent results or references."

At the Sprint II review, be prepared to show the versioned code, a reproducible baseline or core pipeline, saved experiment evidence, and one example of an AI suggestion that you checked. Explain what you accepted or rejected and why. **You remain responsible for understanding the code and defending the research conclusions**.

Sprint III: Project Completion, Deployment and Final Evaluation

In this sprint, you will move from the working core demonstrated in Sprint II to a **complete, usable and evaluated project**. Complete the agreed functionality, refine the system around normal user tasks, deploy it in an appropriate environment, and finish the report and demonstration materials. The final solution should work coherently from the user's first action to the intended outcome.

Begin by reviewing the remaining tasks and acceptance criteria with your supervisor. **Complete the approved objectives before adding optional features.** Any necessary scope adjustment must be agreed and reflected consistently in the implementation, evaluation and report. Maintain the documentation as you work.

The completion review will take place at the end of Sprint III. Follow the confirmed course announcements for the exact review, submission and final presentation dates.

By the End of This Sprint, I Expect to See

- **Completed functionality linked to the objectives**, with a traceability record connecting each objective or requirement to its implemented component, test or evaluation, result and supporting evidence. Explain any unmet requirement and its effect honestly.
- **A coherent system designed around real user tasks**, with suitable navigation, readable interfaces, consistent interactions, appropriate access controls and helpful feedback. Complete and verify each module before progressing to the next, then test the integrated workflows.
- **A working operational release**, accessible through HTTPS where applicable, with the actual backend, database, model or devices connected. For specialised or hardware projects, agree on an appropriate controlled operating environment with your supervisor.

- **Completed testing and evaluation**, including relevant usability and failure scenarios, saved evidence, correction and retesting of important defects, and a discussion of limitations.
- **A complete report and reproducible handover**, including Chapters 5 and 6, corrections to earlier chapters, the final code/model release, setup and operating instructions, and prepared demonstration materials. Review the whole package with your supervisor before submission.

From System Design to a Complete Working Solution

AI agents may help generate code, but **you remain responsible for the design and behaviour of the system**. A collection of generated screens does not establish a well-designed solution. Return to the requirements and Chapter 4 diagrams and check whether the implemented system supports the people, tasks, decisions and data flows described there.

Start with a normal-use scenario: who is using the system, what are they trying to achieve, what information do they need, and what should happen next? Map the **main user journey and its alternative paths**, including invalid data, unavailable services, cancellation, interruption and recovery. Use screen-flow sketches or a simple interactive prototype to resolve confusion before asking an agent to implement more pages.

Keep a clear separation between **presentation, application rules, data access and model/service integration**. Define the responsibility and input/output contract of each module; enforce business rules and permissions on the server as well as providing helpful checks in the interface. Use data constraints and transactions where needed to preserve valid records. A small, well-structured application can be sufficient; extra services should solve a demonstrated need.

Update the relevant Chapter 4 architecture, data, interaction or state diagrams to match the implementation. Choose views appropriate to the approved paradigm: for example, OOAD interactions and classes, or structured processes and data flows. Explain design changes and retain traceability to the objectives. The diagram, code, interface and report should describe the same system.

Build and Verify One Module at a Time

1. Define the module's purpose and completion criteria. Identify the user role, task, inputs, rules, stored data, outputs and dependencies. Write down what successful use and important failures should look like.

2. Design the complete interaction. Sketch the entry point, fields, actions and outcome. Specify empty, loading, success, validation-error, service-error and permission-denied states where applicable. Plan what the user can do next in each state.

3. Implement a complete vertical slice. Connect the interface, server-side rules and storage or inference service needed for that task. Verify actual persistence and results rather than relying on temporary frontend state or mock responses.

4. Test the behaviour and recovery. Check valid and invalid inputs, boundaries, permissions, duplicate submissions, interruption and dependency failures. Confirm that retrying does not accidentally create duplicate records or inconsistent state.

5. Review with a user, correct and commit. Ask someone representative of the intended users to complete the task without coaching. Record confusion, errors and time taken where useful. Correct the important issues and save a meaningful tested commit.

6. Integrate and regress. Connect the completed module to the next workflow and rerun affected tests. Reuse shared navigation, components and validation patterns so later modules behave consistently.

Example: completing an appointment-booking module. A user should find available slots, select one, confirm the details and receive a clear booking reference. The server must prevent two users from taking the same slot. An unavailable slot should produce a useful explanation and alternatives; a failed submission should preserve the entered details. The booking must survive refresh and appear in the authorised staff view. These behaviours should work before moving on to reminders or optional analytics. This is an illustrative design scenario, not a reported project result.

When working with an AI agent, provide the module's user task, business rules, data/API contract, screen states and acceptance tests before requesting code. Ask for one bounded change, inspect the result and run the workflow yourself. Do not accept a module simply because the agent reports that it is finished.

Human–Computer Interaction: What the System Should Look and Feel Like

The interface should help users complete their work with minimal uncertainty. **Visual polish should support task completion**: clear hierarchy, readable content and predictable interactions matter more than adding decorative dashboards or numerous screens. Apply the following checks to the implemented system:

- **Organise screens around tasks and roles.** Use clear page titles, familiar labels, predictable navigation and a visible next action. Provide useful search, filters, sorting or pagination when the amount of data warrants them. Show an honest empty state instead of fabricated totals or charts.
- **Use a consistent visual language.** Keep typography, spacing, alignment, controls, status terms and button behaviour consistent. Give the primary action appropriate prominence, distinguish destructive actions and request confirmation when the consequence justifies it. Test responsive layouts on the devices users are likely to have.
- **Prevent errors and explain corrections.** Label fields, show formats and units, distinguish required from optional inputs and validate on both client and server.

Place a specific correction near the affected field and preserve valid entries. Replace generic messages such as “Something went wrong” with a safe, actionable explanation.

- **Make the system state visible.** Distinguish processing, completed, saved, failed and cancelled operations. Prevent unintended repeated submissions, provide a retry or recovery route, and show progress only when it reflects real work. A spinner should not conceal an operation that has already failed.
- **Provide user control and accessible interaction.** Support cancel, back, correction or undo where appropriate. Use keyboard-operable controls, visible focus, readable contrast, meaningful labels and status messages accessible to assistive technologies. Do not communicate a result or error through colour alone.
- **Verify with realistic users and conditions.** Test normal tasks, mistakes, mobile layouts and slower or interrupted connections. Record task success and observed usability problems, correct them and retest. A working function and a usable workflow are separate things to verify.

Use the Pack to strengthen interface checks. The current Polypane offer provides one year of individual access for non-commercial work, useful for responsive layouts and accessibility checks. LambdaTest’s listed student offer supports live browser testing. Redeem an eligible offer, use it against your actual user tasks and record the issues corrected; automated checks do not replace testing with representative users.

Intelligent Data Handling in AI/ML Interfaces

An AI/ML interface should help users **prepare valid inputs, understand the returned result and decide what to do next**. Displaying an unlabelled prediction or a block of raw JSON is rarely sufficient. Design the interface around the task and the model’s evaluated capabilities.

- **Guide and check the input.** State supported languages, formats, file sizes, units and required fields. Preview the uploaded image, text, audio metadata or tabular data as appropriate. Identify missing values, duplicates, incompatible columns or unsupported inputs before inference, using checks that are actually implemented.
- **Make data preparation visible.** For tabular or batch workflows, show column mapping, data types, missing-value counts and rejected rows. Explain important transformations and request confirmation for consequential changes. Keep an authorised original or audit trail where appropriate; do not silently replace missing values, delete records or alter units.
- **Present results in the user's context.** Use meaningful labels, units, summaries and task-relevant visualisations. Connect each result to its input record or request; provide useful filtering, drill-down or export when needed. Distinguish measured, entered, inferred and unavailable values, and show the scope or denominator behind aggregates.
- **Represent uncertainty honestly.** Explain the limitations of the model output. A raw score is not automatically a calibrated probability; do not label an arbitrary percentage as confidence. Where validated, show alternatives or a low-certainty state and provide a human-review route. Never invent a confidence value or a supporting explanation.
- **Handle inference as a real workflow.** Show queued, processing, completed, failed and cancelled states as relevant. For batch work, preserve successful results, identify failed records and offer a safe retry. Associate returned results with their request so a late response cannot overwrite a newer input's result.
- **Keep provenance and feedback meaningful.** Record the relevant model version, result time and permitted input/source reference. For generated answers, distinguish retrieved evidence from generated text and show traceable sources where the system actually has them. Let users correct or flag a result; feedback needs consent and review before becoming training data.

- **Protect the user's data and decision.** Minimise collection, restrict history and exports by role, explain retention and allow correction or deletion where required. Keep tokens server-side and sensitive content out of logs. In consequential applications, make the intended human oversight and escalation route clear.

Example: an image-classification workflow. Preview the submitted image, explain supported formats, show upload and processing status, then present the predicted class with a task-relevant explanation and any justified uncertainty information. If the implemented checks detect an unsupported or unusable image, explain how to replace it. Offer a review or correction action and preserve the relationship between the image, model version and result. The interface should help the user act on the output without implying that every prediction is correct.

Preparing for Deployment and Model Serving

Once the modules and integrated workflows are ready, prepare a reproducible release candidate. **Deployment makes the application available; model serving runs the evaluated model for prediction.** Preserve the model and preprocessing artefacts, define the input/output schema, validate requests and return useful status information. A live prediction request should not start model training.

Choose a suitable route with your supervisor, based on the project architecture, model requirements, privacy, expected usage and budget. The three examples below show different strengths and may be combined where justified. You do not need to deploy on every platform. Keep the code, configuration, model version and operational instructions traceable.

Plan credits around the chosen host. The Student Developer Pack includes selected cloud offers, such as Azure credit, which may support an alternative deployment agreed with your supervisor. Credits apply to their named provider and eligible

services; they do not make the AWS example free. Check expiry, resource limits and charges before provisioning, and record a plan for continued assessor access after an offer ends.

Example 1: Namecheap Domain, GitHub main and Cloudflare

Why this example matters. Cloudflare connects domain ownership, DNS, GitHub deployment and a public interface. Students can trace the frontend, gateway and model-service boundaries and verify that the complete application works.

1. Redeem the Namecheap student domain offer. After GitHub Education approval, open Namecheap's offer through the Student Developer Pack and select an eligible available .me domain. The listed benefit covers one year of .me registration; other extensions and later renewals may cost money. Check redemption terms and the final checkout amount before completing registration. Keep ownership, recovery details and renewal settings under your control. Use this domain with Cloudflare in the next steps; the domain benefit itself does not provide the model service.

2. Add the domain to Cloudflare. Review the imported DNS records, including email records if the domain is already used. In Namecheap, open Domain List → Manage → Nameservers → Custom DNS and enter the two nameservers assigned by Cloudflare. Save the change and wait for Cloudflare to confirm activation. Keep the registration at Namecheap and manage authoritative DNS in Cloudflare after delegation.

3. Prepare the GitHub repository. Keep the deployable frontend, gateway code, dependency files and configuration in clearly identified folders. Test changes on a development branch and merge reviewed, working changes into main. Exclude credentials, private datasets and unsuitable large model files from Git.

4. Connect Cloudflare Pages to GitHub. Authorise the intended repository and explicitly select main as the production branch. Set the frontend root directory, framework preset, build command and output directory to match the project. For a

standard Vite frontend, npm run build and dist are common settings; verify them against your own project. Check the build logs and test the pages.dev deployment before adding the custom domain.

5. Attach the domain in Pages → Custom domains. Add the apex domain and, if required, www; choose a consistent address and redirect. Confirm DNS and the HTTPS certificate are active. Complete the Pages custom-domain setup rather than only adding a DNS record. Test direct navigation and page refreshes on application routes.

6. Deploy the API gateway separately. For this example, create a Cloudflare Worker, connect its code through Workers Builds and select main as its production branch. Configure the correct gateway folder and build/deploy settings; the Pages frontend deployment does not automatically deploy this separate Worker. Attach an API subdomain such as api.yourdomain.example using the domain you actually own.

7. Connect the trained model service. Deploy the model API on a suitable runtime, such as the AWS service in Example 2. Configure its HTTPS address and service credential in the Worker; store the credential as a secret, never in frontend code. The Worker validates the user request, checks the applicable user/session permissions and calls the model API, which also validates inputs and authenticates the service request. Restrict browser cross-origin access to the intended frontend where needed; CORS alone is not authentication.

8. Verify and record the release. Test the live domain end to end, gateway/model logs, invalid inputs, expired sessions, outages and timeouts. Record URLs, code commits, deployment IDs and model version; confirm outputs come from the deployed model.

Pages serves the frontend, not an arbitrary Python checkpoint. Serve the trained model on a compatible host. Workers AI uses supported models; any substitution must fit the methodology. Document justified browser or edge-inference alternatives.

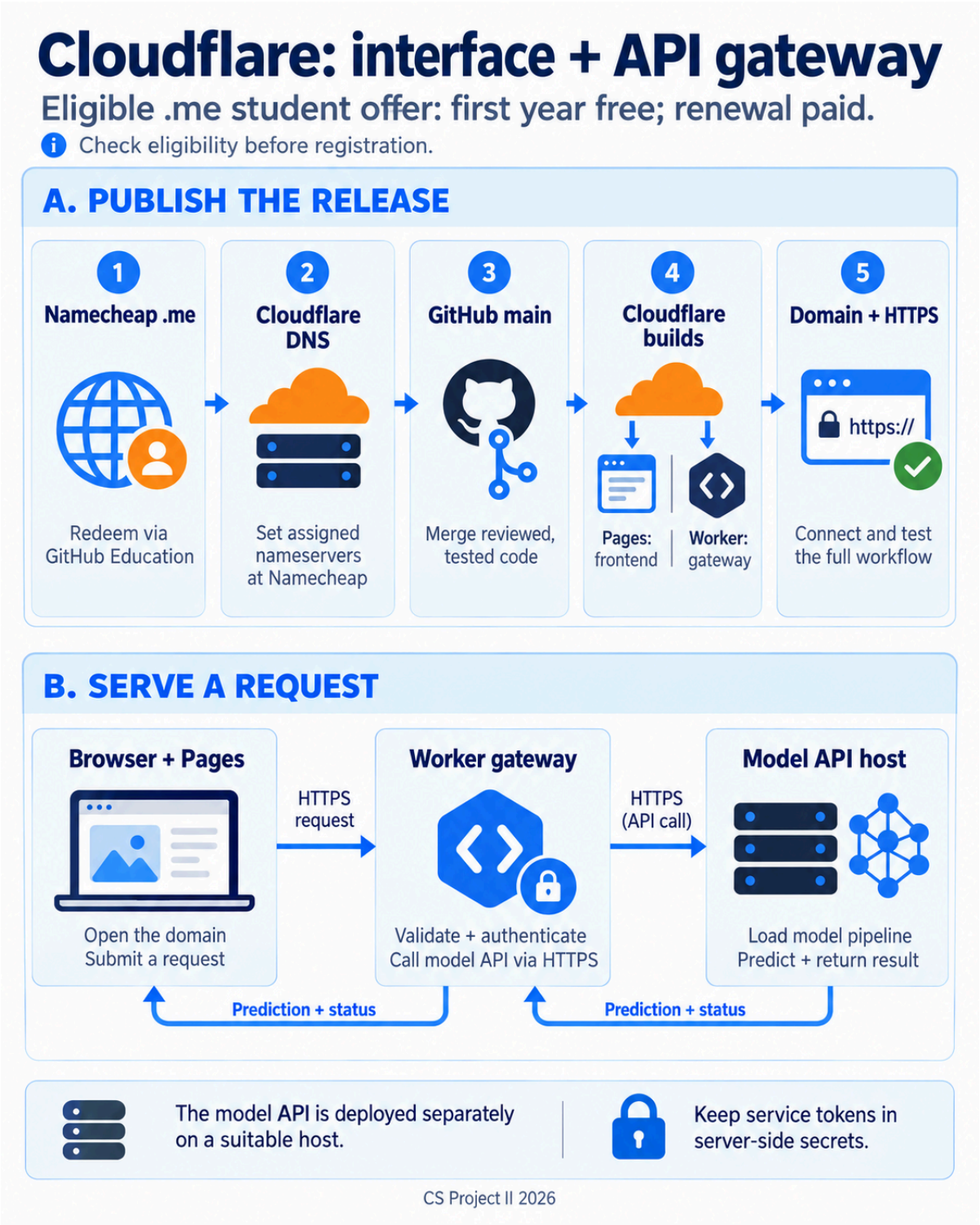


Fig 2: A Namecheap domain and GitHub main release feed the Cloudflare interface and gateway. Predictions return through the gateway; the lower arrow summarises the return path.

Read the diagram in two parts: the upper flow publishes the release; the lower flow shows a live request. The model API must be deployed and tested separately before the complete workflow can succeed.

Example 2: GitHub main, Amazon ECR and ECS Express Mode

Why this example matters. A container packages the application and runtime dependencies for repeatable deployment. AWS provides an example of operating that release with explicit permissions, resources, health checks, logs and persistent storage, helping students connect architecture decisions to reliability and cost.

This route serves stateless applications and CPU-compatible models. ECR stores the image; ECS Express Mode runs it on Fargate with HTTPS routing. GPU-dependent models need a suitable alternative. App Runner is closed to new customers.

1. Prepare the application and model. Provide a defined prediction endpoint, such as POST /predict, and a lightweight health endpoint, such as GET /health. Load the selected model and preprocessing artefacts at startup and report readiness only after successful loading. Validate inputs, return useful errors and identify the model version. For a small project, the same service can also serve the built frontend.

2. Build and test a container. Include a Dockerfile, pinned dependencies and the intended startup command. Bind the server to 0.0.0.0 on the configured container port, for example 8080. Match the image CPU architecture to the service. Include the permitted model artefacts or retrieve a fixed, checksummed version from private S3 storage using an appropriate task role. Do not download an unspecified latest model on each prediction.

3. Configure the AWS account and costs. Select a suitable Region, enable account MFA and use scoped IAM roles. Establish a budget alert and sensible scaling limits before provisioning. ECS Express Mode uses billable resources, including compute and

load balancing; do not assume that student access or a budget alert imposes a spending cap.

4. Publish from GitHub main to ECR. Configure a GitHub Actions workflow triggered by changes to main to run tests, build the container and push it to a private ECR repository. Tag the image with the Git commit and record its digest. Use GitHub OIDC to assume a restricted AWS deployment role, with trust limited to the intended repository and branch or protected environment; avoid long-lived AWS keys.

5. Create the ECS Express Mode service. In the ECS console, follow the Express Mode creation flow and select the tested ECR image. Configure the task execution and infrastructure roles, application task role if needed, container port, health-check path, CPU/memory, network settings and scaling bounds. Confirm the required VPC/subnets and permissions are available. Keep credentials in the supported secret configuration.

6. Test the assigned HTTPS URL. Wait for the service to become healthy, then test the interface and prediction API through the generated application address. Use CloudWatch logs to investigate startup, model-loading or request failures. Store persistent uploads and records in S3 or an appropriate managed database; container-local storage is not the project's permanent record.

7. Release updates deliberately. A push to ECR alone does not replace the running service. Update the Express Mode service to the new release image through the console, or add and verify an explicit service-update step in the workflow. Confirm deployment health and run live smoke tests after each update. Keep the previous working image and a documented rollback procedure.

8. Record ownership and lifecycle. Save the service URL, Region, release image, code/model versions, configuration and operating costs. Keep the deployment available for assessment, then remove unneeded billable resources, including retained storage, images, logs and networking.

Fig 3: A versioned container runs the application and model on AWS. Responses return through the load balancer; the lower arrow summarises the return path.

Example 3: Hugging Face for Model Hosting and Serving

Why this example matters. Hugging Face connects model artefacts and documentation to an interactive demonstration or a dedicated prediction service. This can help assessors inspect the model's intended use and try representative inputs, while allowing the main application to use the same frozen model release.

Begin with the Model Hub. Create an appropriately private or public repository for model weights, preprocessing files, configuration and a model card. Document intended use, data provenance, evaluation, limitations and licence. Record a specific revision and load that release in the serving code. A model repository stores artefacts; uploading a checkpoint alone does not start a prediction service.

Use the Strathmore collaboration workspace. Hugging Face lists a Strathmore University organisation at SU-students-ai-321. With your supervisor's and the organisation administrator's agreement, request membership and use an approved project repository there for the model, model card and permitted supporting artefacts. A project Space may also be organised there when the organisation's hosting plan supports it. A shared namespace can help supervisors locate releases, review work and maintain continuity after a student's personal account is no longer the main point of access.

Keep project ownership and permissions clear. Agree on repository names, visibility, contributors and handover responsibilities before uploading. The administrator should assign access appropriate to the project and its data restrictions. Keep sensitive data and secrets out of public repositories. If organisation access is unavailable, continue in an approved personal or project workspace rather than delaying the sprint.

Route A: an interactive Space. For a focused demonstration, create a Gradio Space, or use a Docker Space when a custom runtime is needed. Add the app code and dependencies, load the pinned model revision and choose suitable hardware. Inspect the build and runtime logs, then test representative inputs and error handling. A push

to the Space repository rebuilds the app; a GitHub main change reaches the Space only if you configure and verify a sync workflow.

Route B: a dedicated Inference Endpoint. For integration into a separate application, check that the evaluated model and preprocessing are supported by the serving engine or an appropriate custom container. Select the model repository and revision, hardware/Region, access policy and scaling settings. Wait for the service to become healthy, test its request/response contract, then call it from the application backend or gateway. Store the access token as a server-side secret.

Use it with the other examples. The Cloudflare Worker in Example 1 can call a suitable Hugging Face endpoint over HTTPS. An AWS application can also load permitted artefacts from the Hub at a fixed revision or call an endpoint. Keep model storage, application hosting and inference responsibilities clear; a Space and a dedicated endpoint are alternatives, not compulsory consecutive steps.

Check academic access first. Organisation membership and paid academic benefits are separate. The public Strathmore organisation profile does not establish an Academia Hub subscription, included compute credits or free model hosting. Ask its administrator or university IT to confirm the actual plan, quota, billing and permitted use before provisioning a Space or endpoint. Where an Academia Hub subscription exists, its published benefits include storage, limited compute credits and higher access limits. Confirm which entitlements apply to your account; membership alone does not establish free PRO access or unlimited GPUs.

Consider eligible ZeroGPU access. Current ZeroGPU documentation allows free personal accounts in good standing, with a verified email and an account older than 30 days, to host up to two ZeroGPU Spaces. Check framework compatibility and the actual account eligibility before planning around this route. Shared compute has daily quotas and queues; this can support a suitable Gradio demonstration but should not

be presented as an always-available service. This is a general eligibility route, not an automatic GitHub Pack entitlement.

Check operating limits and costs. Review hardware, sleep/restart behaviour, temporary storage, visibility and access restrictions. Budget for dedicated endpoints or upgraded compute beyond included entitlements, and account for cold starts when scale-to-zero is enabled. Preserve the project's evaluated model and preprocessing when changing the serving environment.

Finish with evidence. Preserve licences and access restrictions, keep tokens in secrets and arrange assessor access securely. A public Space exposes its running app and source; a protected Space may hide code while leaving the app public. Keep private data out of public demos. Record the model and app revisions, hardware, URL, representative predictions, failure tests and limitations. A convenient demo still needs the system-design and HCI checks above.

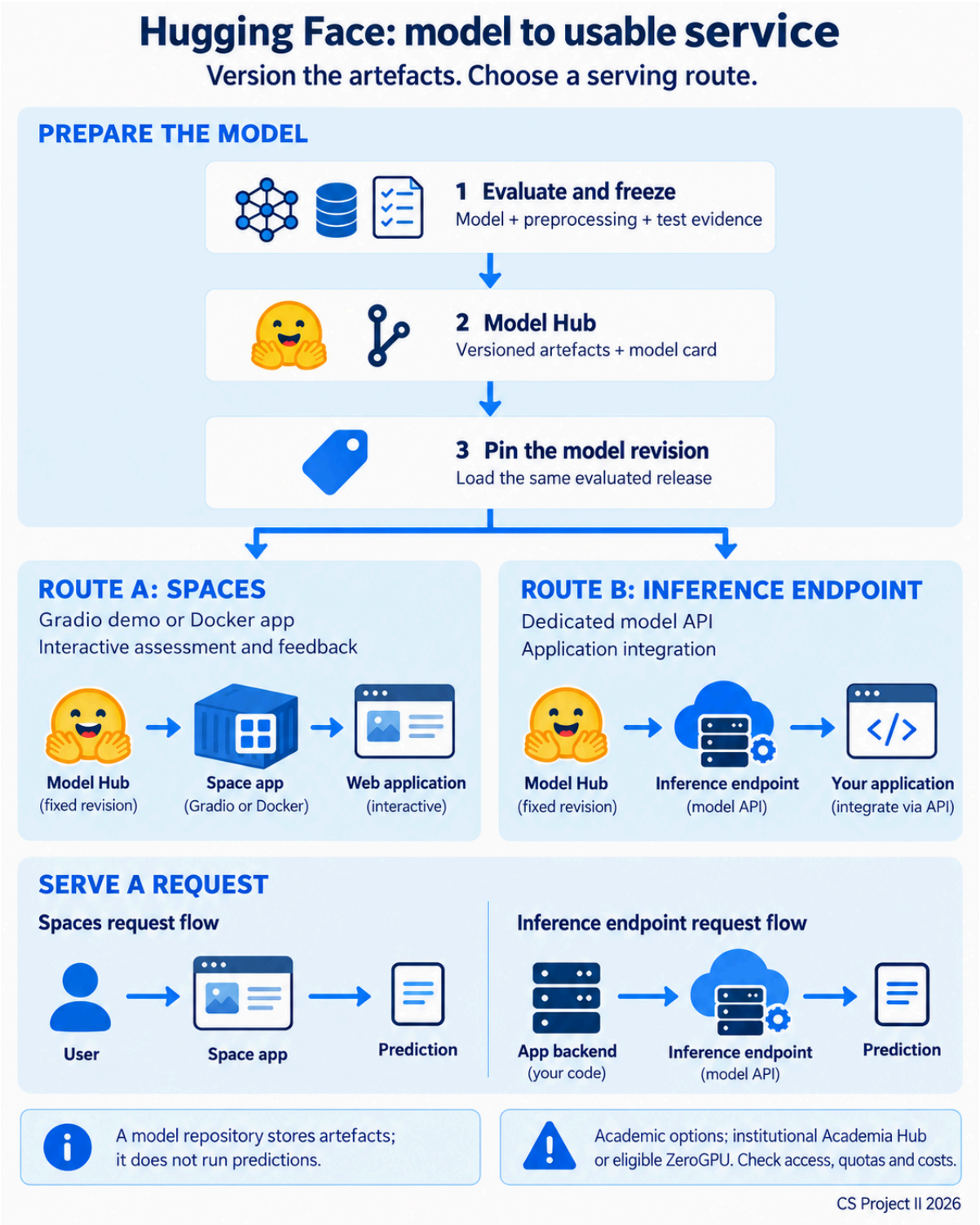


Fig 4: A versioned Hugging Face model can support a Space demonstration or a dedicated inference service. Choose and verify the route appropriate to the project.

Serving Your Model Reliably

- **Deploy the evaluated pipeline**. Preserve the exact feature order, preprocessing, tokenisation, label mappings and thresholds used during evaluation. Record the model identifier, version or checksum and dependencies, and check representative predictions against the local reference pipeline.
- **Keep evaluation valid**. Freeze model and configuration choices before final held-out testing; keep training, development and test data separate. If test results influence a revision, disclose that use and arrange an appropriate independent evaluation. Report relevant baselines, error analysis and limitations.
- **Protect the service and its users**. Apply input-size limits, authentication/authorisation where required, rate or usage limits and safe error messages. Keep secrets server-side, avoid recording sensitive inputs in logs, and use non-sensitive demonstration data. Projects involving consequential decisions need appropriate human oversight.
- **Demonstrate operational behaviour**. Record representative response times, failure rates, resource use and any cold-start or concurrency limitations. Test a restart, an unavailable dependency and recovery. For long-running inference, justify an asynchronous job/queue design rather than allowing requests to fail silently.

Mobile Applications: Testing, Model Serving and Store Publication

For a mobile project, complete the same engineering chain described above, then package and distribute the application for the intended devices. **Google Play and the App Store distribute the app; the backend and model service still need an operating environment.** Choose Android, iOS or both according to the approved scope, users, available hardware and budget. A cross-platform framework such as Flutter or React Native still requires separate platform builds, signing and testing.

Plan store access early in Sprint III. Agree with your supervisor who owns the developer account, application identifier, signing credentials and future maintenance responsibilities. Complete any required identity or organisation verification before depending on publication. Use an institutional account only through an authorised account holder; do not register yourself as the university without authorisation.

Publishing an Android Application on Google Play

1. Establish the publishing account and budget. Register through Google Play Console using the appropriate personal or organisation account. Google currently lists a US$25 one-time registration fee; confirm payment, local charges and verification requirements during registration. Complete any identity, contact or device checks requested. GitHub Education membership does not by itself establish a waiver of store fees.

2. Prepare a signed release. Select a stable application ID, set the app version and an increasing version code, configure the production API address and build a release Android App Bundle (.aab). New Google Play apps use this publishing format. Configure app signing and protect the upload credentials; keep signing material and passwords out of Git. Confirm the current target API and SDK requirements in Play Console and test the release build on real supported devices.

3. Complete the store information. Create the app record, provide the title, description, icon and screenshots, select the audience and countries, and complete content rating, app-access instructions, ads declarations where applicable and Data safety information. Provide a publicly accessible privacy policy consistent with the app, backend and integrated SDKs. If reviewers need to sign in, provide working review access and keep the required backend available.

4. Test before requesting public release. Use internal testing for early checks and closed testing for the intended group. For personal developer accounts created after 13 November 2023, Google currently requires at least 12 testers opted in continuously for 14 days before applying for production access. Recruit genuine testers, collect feedback and demonstrate the corrections made. Completing the minimum period does not automatically grant production access or publication.

5. Submit, respond and release. Resolve pre-launch report issues, apply for production access when eligible and submit the completed release for Google's review. Respond to requests and correct rejected items. After approval, verify installation from the intended store/testing link, authentication, permissions, predictions and updates on a clean device. Use a staged rollout where suitable and monitor crashes and backend errors.

Account and data deletion. Where the app enables account creation, implement the required deletion routes rather than merely removing the sign-in screen. Google Play requires an in-app route and a web resource for requesting account and associated-data deletion. Explain any justified retention in the privacy information and ensure the backend honours the user's request.

Trying iOS Through TestFlight and the App Store

1. Confirm resources and membership. Students can attempt the App Store when an iOS release fits the project scope and they can access the required Apple build tools and test devices. Plan for a supported Xcode/macOS build environment or an appropriate build service. Apple allows development and limited own-device testing with an Apple Account; TestFlight and App Store distribution require Apple Developer Program access.

2. Check the university route before paying. Apple currently lists US$99 annual Developer Program membership, in local currency where available. Eligible accredited educational institutions can request a fee waiver subject to Apple's conditions. Ask the course team or university IT whether an authorised institutional account and access are available; no Strathmore membership or waiver is assumed here. Individual students do not automatically receive this institutional waiver.

3. Prepare the iOS release and test it. Create the App Store Connect record and matching bundle identifier, configure signing and provisioning, archive a release build and upload it through the supported tooling. Use TestFlight to distribute the beta to authorised testers and collect feedback. External testing may require beta review. TestFlight builds are available for up to 90 days, so ensure the assessed build will still be usable when required.

4. Prepare and submit the public version. Complete the product page, screenshots, age rating, privacy details, support/privacy URLs and review information. Address current SDK, permissions and any applicable encryption/export questions, provide review access and keep services running. Submit the chosen build for App Review, respond to feedback and release after approval. A TestFlight installation demonstrates beta distribution; describe public App Store publication only after it actually occurs.

Protect personal data sent to AI services. Apple's review guidance requires clear disclosure and explicit permission before sharing personal data with third-party AI. Make the consent and data flow match the implemented service. Check current account-deletion and other applicable review requirements. Where relevant, review additional requirements for children, health functions, user-generated content or digital purchases before choosing those features.

Serving a Model to a Mobile Application

Option 1: call your hosted inference service. Reuse the earlier cloud architectures: mobile app → HTTPS application backend or gateway → AWS or Hugging Face inference service → a structured result. A Cloudflare Worker may provide the gateway. Authenticate the user, authorise the operation, validate the input on the server and enforce usage limits. Keep provider access tokens and privileged service credentials on the server; a compiled mobile app is not a safe place to hide them. Use a reachable production HTTPS address and test over mobile data as well as Wi-Fi; localhost on the phone refers to the phone itself, not your development laptop.

Keep the request and response contract stable. Define supported input types and limits, result fields, error codes, request IDs and the model/API version. Return enough information for the interface to explain the result. For slower inference, return a job identifier and let the app check status rather than leaving the screen blocked. Use bounded timeouts and safe retries; prevent duplicate processing and ensure a late result cannot overwrite a newer request.

Option 2: run a compatible model on the device. For suitable tasks, use an appropriate mobile runtime, such as LiteRT or Core ML. Package the evaluated model or obtain a controlled version through an approved update mechanism. This can reduce network dependence and keep inference inputs on the device, but feasibility depends on model compatibility, storage, memory and compute. Re-evaluate converted or quantised models for accuracy, latency, battery use and behaviour on representative target phones.

Choose the route using evidence. Hosted inference can support larger models and central updates but depends on connectivity, latency, service availability and operating cost. On-device inference can support offline use but has device and model-size constraints. A justified hybrid may use local checks and remote inference. State clearly

which operations work offline, and distinguish a cached result from a newly computed prediction.

Fig 5: Mobile release routes and two model-serving options. Store review, backend deployment and model evaluation are separate parts of delivering the complete solution.

Mobile Interaction, Reliability and Handover

- **Design for actual phone use.** Use suitable touch targets, readable text, accessible labels, predictable navigation and appropriate keyboard/input types. Preserve important form state across rotation, backgrounding or interrupted use, and test the task on the devices and screen sizes your users have.
- **Handle connectivity and permissions deliberately.** Explain uploads and processing, request camera, microphone, photo or location access only when needed, and handle denial gracefully. Show offline, timeout, failed and completed states with a useful recovery action. Store or queue sensitive data only when the project's consent, protection and retention arrangements support it.
- **Test the installed release end to end.** Cover first installation, sign-in and expiry, permissions, poor connectivity, background/resume, model errors and app upgrades. Test representative lower-resource phones, OS versions, accessibility settings and input sizes. A successful emulator run alone is insufficient evidence of the intended mobile experience.
- **Keep app, API and model releases compatible.** Record the app build, backend release, API contract and model version used for evaluation. Mobile users may retain older builds, so maintain a compatible API or provide a controlled update route. Back up signing/account recovery information securely and document the maintenance and hosting costs.
- **Prepare review and assessment evidence.** Provide the repository, signed build or authorised testing invitation, installation instructions, usable demonstration account, privacy information, device test records, inference results and current review status. Keep assessor access and the backend available for the agreed period.

Agree on the assessment route with your supervisor. If public-store approval is still pending, demonstrate the completed release through an agreed legitimate test-distribution route or supervised device installation and report the actual status. A pending review should have evidence of submission and follow-up; it should not conceal unfinished functionality. Do not promise an external approval date or claim that the app is published when it is only in testing.

Store fees, eligibility, SDK requirements and review policies can change. The linked provider guidance was checked on 19 October 2026; verify the live requirements again before registration or submission.

Final Worked Example: An Image Classification Web or Mobile Application

The deployment examples above describe where a solution runs. This final example brings them together into one complete user journey: sign in, select several images, correct unsuitable inputs, run the model, understand the results and decide what to do next. Consider an illustrative plant-image application that recognises only the plant conditions represented in its evaluated dataset. Adapt the subject, classes and recommendations to the approved project; this example is a design guide, not evidence that a particular model can diagnose plant disease.

1. Authenticate the User and Control Access

Use a maintained identity service rather than inventing a password system. Offer passkeys where supported, or university/Google sign-in through OpenID Connect. For a native mobile app, use the system browser and the authorisation-code flow with PKCE. A passkey uses a device-supported credential; the application should not collect the user's fingerprint or face image. Provide a clear recovery route, sign-out and session-expiry feedback; require stronger verification for privileged administration.

Authentication establishes identity; authorisation determines what that person may access. The backend must validate the session or token and check ownership on every upload, job, result, download and deletion request. Keep credentials and model-provider keys server-side. Use secure, HttpOnly, appropriately SameSite cookies and CSRF protection for cookie-based web sessions; use platform-protected credential storage on mobile. A hidden button is not an access control. A student should never retrieve another student's images by changing an identifier.

2. Select Multiple Images and Help the User Correct Mistakes

Explain the supported subjects, file formats, batch limit and photo-taking requirements before selection. Use the system photo picker where possible, requesting only the access needed. Display a thumbnail and stable image identifier for every selected item, with accessible Remove, Replace and Retry actions. Warn about duplicates without silently discarding distinct views. Let the user rotate or crop a copy, and show exactly which version will be analysed.

Validate each image independently in the interface and again on the server. Check an allowlist of formats, actual file signature and safe image decoding, file size, pixel dimensions and resource limits; an extension or declared MIME type alone is insufficient. Reject corrupt or unsupported files with a specific correction. Apply suitable malware checks and safe decoding or re-encoding, generate storage names and keep originals private. Remove unnecessary location/EXIF metadata, preserve orientation and avoid treating a filename or embedded text as an instruction.

Separate file validity from suitability for the model. A sharp photograph of a shoe may be a valid image but is outside a plant classifier's scope; a leaf may be too dark or blurred to assess. Use evaluated quality and subject checks where feasible, while acknowledging that these checks can also fail. Show "Not analysed: unsupported subject" or "Retake: image too blurred" instead of forcing a class. Preserve the accepted images and ask the user to confirm the corrected batch before analysis.

3. Upload Privately, Process Reliably and Track Each Image

A practical design is web/mobile client → authenticated HTTPS backend → private image storage and job queue → inference worker → results store. The worker loads a fixed model version from the project's controlled release and runs on the chosen AWS or Hugging Face service. An on-device variant can use the mobile route discussed above. Begin with a simple backend and worker; introduce separate services only when scale or reliability justifies them.

Return a batch/job identifier promptly for longer work. Track each image as uploading, queued, analysing, completed, rejected, failed or cancelled, and show progress such as "3 of 5 processed". Preserve the mapping between thumbnail, input, model version and result even if responses arrive out of order. Use bounded concurrency, timeouts and retries; request identifiers/idempotency controls should prevent duplicate jobs or charges after a double tap or connection loss. Allow safe cancellation and retry of failed items without rerunning successful ones. **Distinguish service failure from model uncertainty.**

4. Decide Whether the Model Can Support a Result

Use the same preprocessing and label mapping as the evaluated model release. Agree and validate the acceptance/abstention policy before the final test: consider image suitability, supported classes, relevant model scores and observed errors. A closed-set classifier may assign a high score to an unfamiliar object; a softmax score alone does not prove that the subject is supported or that the prediction is correct. If displaying confidence, explain what it represents and evaluate calibration. Do not present an arbitrary percentage as diagnostic certainty.

For an accepted result, show the image, predicted class, relevant limitations and an approved next-step recommendation. Where the evidence is insufficient, return a distinct "**Not classified**" or "Needs review" state. Do not turn a timeout into "healthy",

combine unrelated images into one prediction or describe “no detected known condition” as proof that no problem exists. Explain which additional photo or context would improve the assessment.

5. Use Tool Calling to Support the Next Step

Tool calling is a controlled application workflow: an orchestration model can request a named function, but the server validates and executes it. A conventional image classifier does not automatically acquire tool-calling capability. First use simple rules for predictable actions such as requesting a clearer image. Add an orchestration model only where interpreting context or retrieving relevant guidance gives a measurable benefit.

For an uncertain but suitable image, the backend can offer a constrained lookup in a supervisor-approved knowledge base, retrieve a photo-taking guide or find an appropriate expert-review route. Example tools are retrieve_guidance(topic, crop, locale) and get_photo_checklist(subject). A request_review function should create a referral only after the user chooses to share the required information. The tool may suggest the next step; it must not replace “Not classified” with an invented model diagnosis.

Give tools narrow schemas, allowlisted sources and least-privilege credentials. Validate arguments, user permissions and returned data outside the language model; enforce call, time and cost limits. Treat uploaded content and retrieved text as untrusted data, including instructions hidden within them. Send only necessary information, disclose any external AI processing and obtain the applicable consent. Do not allow arbitrary URLs, commands or unrestricted browsing. Read-only guidance should be the default; consequential actions require an explicit user decision.

Ground any generated advice in retrieved evidence and show its source, retrieval date and limits. Label it “**Tool-assisted next-step guidance**”, separately from “Model prediction”. If retrieval is irrelevant, sources conflict or the tool is unavailable, state that

clearly and use a safe fixed fallback: retake the image or request qualified review. For the plant example, do not generate unsupported treatment instructions or pesticide dosages. Tool use should reduce uncertainty about the next step, not hide uncertainty about the image.

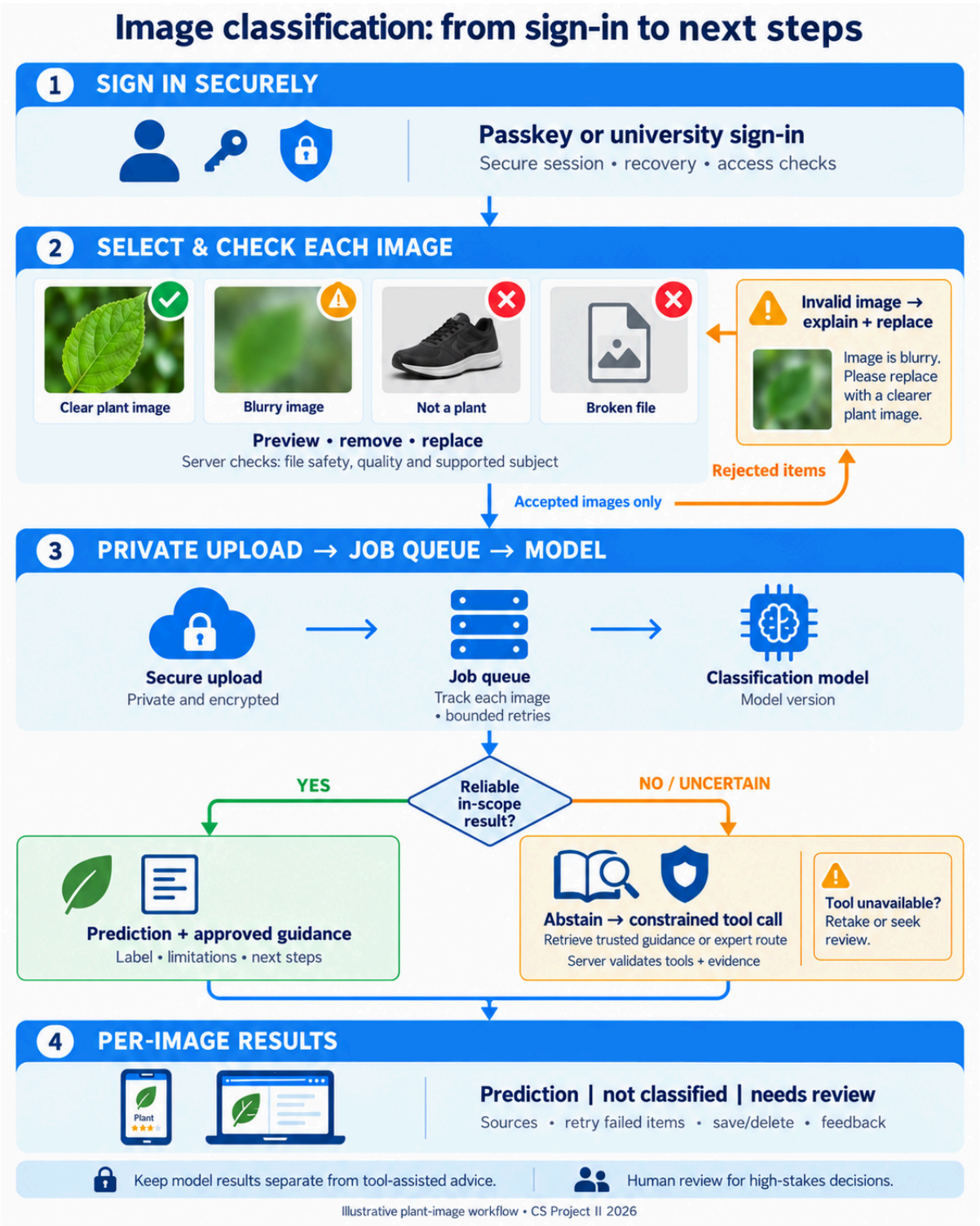


Fig 6: An authenticated image-classification journey. Invalid inputs return for correction; accepted images receive either a supported prediction or an explicit abstention with constrained, source-backed next-step guidance.

6. Present Useful Results and Close the Feedback Loop

Use one result card per image, with its thumbnail, status, explanation, recommendation, provenance and next action. Provide a batch summary such as “2 classified, 1 needs review, 2 require replacement”; keep partial success visible. Announce progress and errors to assistive technologies, support keyboard and touch use, and combine text/icons with colour. Ask only for context the recommendation needs; distinguish optional details from required fields. Never leave the user with a spinner that has no recovery path.

Allow the user to save an authorised history, export a result, delete images/results or report a suspected mistake. Explain retention, sharing and whether inputs are used for research or future training. Offer separate consent for reuse; do not train automatically on uploads or assume that a user correction is a verified label. A supervisor or qualified reviewer should validate feedback before it becomes a new dataset or model release. Keep minimal audit records of versions, errors and tool calls without logging raw private images, tokens or unnecessary personal data.

7. Build the Journey in Modules and Demonstrate the Failure Paths

Implement and verify **one complete module at a time**: identity and ownership; selection and validation; upload/job tracking; inference and abstention; recommendations/tool use; history, deletion and feedback. Define each module’s inputs, outputs, errors and visible user feedback before asking an AI coding agent to implement it. Verify a small end-to-end vertical slice early, then extend it. Keep the architecture, data flow and tests aligned with Chapter 4 and the evaluation evidence in Chapter 5.

Demonstrate a deliberately mixed batch: a usable in-scope image, a blurred image, an unrelated object, a corrupt file and a suitable image for which the model abstains. These are test cases, not measured outcomes. Show the expected correction or result

for each, including a successful guidance lookup and a failed lookup. Also test expired sessions, cross-user access attempts, oversized/deceptive files, network interruption, duplicate submission, out-of-order responses, provider failure, prompt injection, deletion and recovery. Record actual outcomes and fix defects before claiming completion.

Evaluate the classifier and the complete system separately: per-class precision/recall/F1 and confusion patterns on held-out data; abstention coverage and errors among accepted predictions; suitability checks; guidance relevance and source support; end-to-end latency, batch success, operating cost and user task completion. Include realistic devices, connections and representative users. Agree project-specific acceptance criteria with the supervisor. A convincing demonstration shows how the system behaves when it cannot answer, as well as when it can.

Why this example matters: the project is complete when identity, data handling, inference, recommendations, recovery and user understanding work together. The model is one part of the system. A clear, honest "I cannot classify this image; here is the supported next step" can be a better outcome than a confident but unsupported answer.

Complete Testing and Evaluation

Use the acceptance criteria agreed with your supervisor. For each test, record the requirement, input or conditions, expected result, actual result, pass/fail outcome, evidence and any correction/retest. Include end-to-end workflows, permissions, invalid inputs, service failures, data persistence and the deployed configuration.

Select evaluation evidence appropriate to the project: held-out metrics, baselines and error analysis for AI/ML; task completion, usability/UAT and performance for information systems; calibration, reliability and latency for IoT; and correctness, defined threat scenarios, overhead or transaction behaviour for cybersecurity and blockchain.

Use authorised participants, data and controlled testing environments. Report actual sample sizes, conditions and limitations rather than unsupported claims of success.

Complete the Report and Final Submission Package

- **Chapter 5:** document the implemented system, environment, deployment architecture, model/service versions, tests, evaluation results, discussion and limitations. Use labelled figures and readable evidence, explain what each result means, and connect it to the relevant objective.
- **Chapter 6:** answer the objectives and research questions using the evidence already reported; state the contribution, justified recommendations and prioritised future works. Keep future enhancements separate from functionality required to satisfy the approved scope.
- **Whole-report review:** align Chapters 1–4 with the completed work; check the abstract, contents, figure/table lists, numbering, captions, cross-references, APA 7 citations and reference list. Follow the current class template and submission instructions, and include the required declarations and approvals without inventing signatures.
- **Reproducible handover:** provide the repository and final release tag, live URL or controlled operating instructions, dependency/configuration guidance, an example environment file without secrets, database setup/migrations where applicable, model/data access instructions, test records, user guide and known limitations. Share assessor access securely. Record offer expiry dates, renewal charges and the plan for maintaining or closing the deployment after assessment.
- **Final presentation and demonstration:** explain the problem, objectives, design decisions, completed workflow, evaluation, contribution and limitations. Prepare representative test inputs and a backup recording/screenshots for connectivity failures; label backup evidence clearly and keep the live demonstration ready.

- **Supervisor review and submission:** review the working release, final report and evidence together, resolve required corrections and submit through the officially confirmed channel and deadline. Reopen the submitted files and verify links and access permissions. Record what was submitted and retain a backup.

At the Final Sprint Review

Be prepared to demonstrate the complete project from a user or technical input to its final output, show the deployed release and evaluation evidence, explain how each objective was addressed, and present the completed report and handover materials. Demonstrate that another person can follow the operating instructions without depending on undocumented steps on your laptop.

Sprint III is complete when the agreed project scope, operational release, evaluation, report and handover meet the supervisor-reviewed acceptance criteria. A successful deployment alone does not establish project completion. Any unresolved limitation must be stated honestly, with its effect on the objectives and final conclusions.

Deployment and Interface Design References

Use the official references below alongside Chapters 4–6 and the current class template. Provider interfaces, access conditions, limits and pricing can change; these resources were checked on 19 October 2026.

Namecheap: change domain nameservers

Cloudflare Pages: Git integration

Cloudflare Pages: production branch controls

Cloudflare Pages: custom domains

Cloudflare Workers: Git builds and deployment

Cloudflare Workers: secrets

Cloudflare Workers AI: supported models

AWS: ECS Express Mode overview and charges

AWS: create an ECS Express Mode service

AWS: update an ECS Express Mode service

GitHub: OpenID Connect with AWS

AWS: App Runner availability change

Hugging Face: Spaces overview and account eligibility

Hugging Face: model cards

Hugging Face: downloading versioned model artefacts

Hugging Face: synchronising Spaces with GitHub Actions

Hugging Face: creating an Inference Endpoint

Hugging Face: endpoint security

W3C WAI: form labels and instructions

W3C WAI: success, error and status notifications

W3C WAI: page structure and navigation

Microsoft Research: Guidelines for Human–AI Interaction

GitHub Education: eligibility and student access

GitHub Student Developer Pack: current partner offers

Namecheap: student domain terms and renewal

Hugging Face: institutional Academia Hub benefits

Hugging Face: ZeroGPU hosting eligibility and quotas

Polypane: GitHub student offer

Strathmore University: Hugging Face organisation

Hugging Face: organisation membership and access control